

סיכום הרצאות 2026 – מערכות הפעלה

מרצה: פרופסור דוד סרנה

© גיא יער-און

ייתכן שנפלו טעויות בעת כתיבת הסיכום, ולכן השימוש בסיכום על אחריותכם בלבד.

תוכן עניינים:

הרצאה 1: חזרה על מבנה מחשב, מבוא למערכות הפעלה

הרצאה 2: ניהול זיכרון, מבנה מערכת ההפעלה, system calls

הרצאה 3: מכונה וירטואלית, תהליכים (Process)

הרצאה 4: המשך תהליכים, Sockets, Threads

הרצאה 5 : CPU Scheduling

הרצאה 6 : Synchronization

הרצאה 7 : Deadlocks

הרצאה 9 : Main memory

הרצאה 10: Virtual Memory

הרצאה 11: Files

הרצאה 1

מהי מטרת הקורס? עד היום התעסקנו ברמת תוכנית בודד ותהליך בודד: סינטקס של קוד, אלגוריתמים בהם משתמשים, כתיבת קוד נכון ויעיל וכו'. בקורס – אנחנו נתמקד בעולם בו פועל התהליך שלנו והאינטראקציה של אותו תהליך עם הסביבה, וכמובן נדון בכמה וכמה תהליכים. במהלך הקוד נדון בכמה וכמה שאלות:

- מה הן מערכות הפעלה?
- מה הן יודעות לעשות?
- כיצד מערכות הפעלה בנויות?
- מרכיבים עיקריים של מערכות הפעלה – תהליכים, Threads, CPU-Scheduling, וכו'.

נעיר מראש שבמהלך ההרצאות נדון בעקרונות כלליים של מערכות הפעלה – ולכן: בהרבה מהפעמים התשובה תהיה 'לא ניתן לדעת', שכן זה יהיה מאוד תלוי במערכת ההפעלה שתרוץ. **בתרגול:** נדון במערכת הפעלה ספציפית: Linux.

חזרה על מבנה מחשב:

- ארכיטקטורת Van-Neumann: משמשת כבסיס לרוב מערכות המחשבים המודרניות, היא מאוד מאוד פופולרית היום בעקבות יתרונותיה – אנחנו שומרים את הקוד ואת הdata שלנו באותו מקום בזיכרון, מה שהופך את התכנון והיישום של תוכנה וחומרה לפשוטים יותר. כמו כן, הארכיטקטורה נתמכת ע"י רוב שפות התכנות והכלים המודרניים. עם הזמן – התגלו בשנים האחרונות מגבלות בארכיטקטורה זו.
מגבלותיה במערכות מודרניות:
- 1. מחשוב מקבילי – מתאימה יותר למחשוב סדרתי, בעוד מערכות מודרניות דורשות יותר יכולת של תכנות מקבילי עם כמה וכמה מעבדים בו זמנית.
- 2. אבטחת מידע – העובדה שהקוד והנתונים נשמרים באותו מקום בזיכרון הוא פתח עבור "פולשים" שיוכלו לגנוב לי קוד.

במערכת ישנם 4 מודולים מרכזיים:

1. Memory – זיכרון: מאחסן את כל ההוראות ואת כל הdata של התוכנית שלנו. יש לנו בו את Control-Unit שאחראי על הרצת התוכנית שלנו ואת הAlu: אחראי על פעולות חשבון ולוגיקה לפי דרישות התוכנית וכן יש לנו בו את הIO לתקשורת עם "העולם החיצון".
- הזיכרון הוא אוסף של הרבה מאוד תאי זיכרון, כאשר גודל כל תא הוא קבוע. יחידת הזיכרון הבסיסית היא bit ותא זיכרון לרוב הוא byte בודד. הוא יקר ביחס לדיסקים וכדומה.
- הזיכרון עליו אנחנו מדברים נקרא RAM (Random Access Memory): מדובר בזמן גישה זהה לכל תא זיכרון. מדוע זה טוב לנו? כי זה גורר שלא אכפת לנו היכן התוכנית תשב בזיכרון – זמן הגישה זהה.
- WORD:** אוסף כל כמה ביטים (בהתאם לארכיטקטורה של המעבד), מדובר ביחידת המידע שמועבדת בפעולה בודדת. זהו גודל שלרוב זהה לגודל הרגיסטרים ולBUS. לרוב מדובר ב8 ביטים.
- לכל תא זיכרון ישנה כתובת:** את הכתובת אנחנו צריכים על מנת שנדע היכן הנתון נמצא בזיכרון. את הכתובות מייצגים באמצעות מחרוזת של ביטים. בהינתן n ביטים, נוכל לתמוך בזיכרון בגודל של $2^n - 1$ כתובות. מדובר בכמות המקסימלית של זיכרון שנוכל לשים במערכת שלי, כיוון שלא נוכל לייצג עוד זיכרון בשיטה זו.

סיכום מערכות הפעלה | © גיא יער-און

- את גודל הזיכרון מייצגים בבייטים – KB, MB, GB, TB וכו'.
- Hz הוא מס' הגישות לזיכרון שאנחנו יכולים לעשות בשנייה. כלומר 1 Hz הוא מחזור (Cycle) אחד בשנייה. מהו סייקל? יחידת העבודה הקטנה ביותר של זיכרון. הזיכרון יכול לבצע פעולה בכל רגע נתון. ככל שמס' הסייקלים גדול יותר פר שנייה הזיכרון עובד מהר יותר כי הוא מספיק לבצע יותר פעולות.
- מהירות הוא הנתון שאומר כמה סייקלים הזיכרון מבצע בשנייה אחת.

במקום לענות על השאלה: "כמה סייקלים יש בשנייה", נוכל לחשב כמה שניות לוקח סייקל אחד בודד:

$$\text{Time Per Cycle} = 1/\text{Hz}$$

לצורך ההמחשה: אם משהו קורה פעמיים בשנייה, כלומר 2 Hz, אז די ברור שכל פעם כזו (סייקל) לוקח חצי שנייה. כלומר, $\frac{1}{2}$.

פעולות על הזיכרון:

מערכת הזיכרון מופעלת באמצעות:

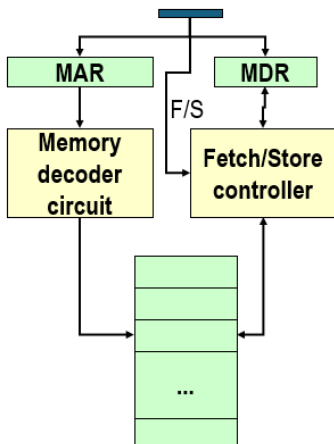
1. רגיסטר כתובות זיכרון – MAR – בעת שליפה ישמש לקרוא את הערך שאנחנו קוראים מהזיכרון.
2. רגיסטר נתוני זיכרון – MDR – ישמש לקרוא את הערך שנרצה לכתוב בזיכרון.
3. אות לשליפה/אחסון – אות חשמלי על מנת להבין האם הפעולה הבאה תהיה שליפה או אחסון.

- **שליפה – Fetch:** שליפת עותק של תוכן בתא הזיכרון עבור הכתובת שצוינה. מדובר בפעולה לא הרסנית כי היא משמרת את הערך בתא הזיכרון.

1. טוענת את הכתובת לתוך MAR

2. מבצע Decode לכתובת הזו: יחידת הזיכרון מפענחת את הכתובת שבMAR כדי לאתר פיזית את התא המתאים בזיכרון, היא תעתיק את התוכן של תא הזיכרון לתוך MDR
- אחסון – Store:** אחסון הערך שצוין לתוך תא הזיכרון עבור הכתובת שצוינה. פעולה הרסנית, דורסת את הערך הקודם בתא הזיכרון.

1. טוען את הכתובת לתוך MAR: המעבד מציב שם את כתובת התא אליו הוא רוצה לכתוב.
2. טוען את הערך לתוך MDR: המעבד מציב שם את הערך החדש שהוא רוצה לאחסן.
3. מבצע Decode של הכתובת לMAR: יחידת הזיכרון מפענחת את הכתובת בMAR על מנת לדעת להיכן לכתוב.
4. מעתיק את התוכן של MDR לתוך תא זיכרון עם הכתובת הספציפית שנבחר וכך מחליף את המידע הישן.



מודול Input/Output (IO):

מטפל בהתקנים שמאפשרים למערכת המחשב לתקשר וליצור אינטראקציה עם העולם החיצון – לתקשר וליצור אינטראקציה עם העולם החיצון (מסך, מקלדת), לאחסן מידע.

לכל התקן של IO ישנו התקן בשם **IO CONTROLLER**: הבעיה היא שמהירות התקני IO נמוכה מאוד ביחס למהירות הRAM – ולכן בכל שימוש בIO נעכב את כל המערכת, לכן הפתרון הוא שימוש בהתקן זה. מדובר במעבד ייעודי שכולל **רכיב זיכרון קטן (IO Buffer)**: יש בו לוגיקת בקרה לניהול התקן הקלט/פלט: הוא ינהל עבורנו בצורה עצמאית את האינטראקציה עם המכשיר עצמו. למשל: הזזת זרוע הדיסק. היתרון: כיוון שהוא עצמאי, אפשר לבצע את הפעולות שלו במקביל למעבד שלי שמבצע פעולות של התוכנית. בסיום ניהול האינטראקציה

סיכום מערכות הפעלה | © גיא יער-און

עם device, ה-CONTROLLER יודיע לי על כך. ישנם כמה וכמה Controller's: אחד ייחודי לכל סוג של מכשיר. למשל – ישנו Device Controller עבור המסך שיכול לנהל כמה וכמה מסכים.

מודל ה-ALU:

מיועד לבצע פעולות מתמטיות, לוגיות. במחשבים של היום ה-ALU משולב כבר בתוך ה-CPU. הוא מורכב ממעגלים לביצוע הפעולות האריתמטיות/לוגיות, רגיסטרים לאחסון תוצאות חישוב הביניים, BUS שמחבר בין השניים.

מודל ה-Control Unit (יחידת הבקרה):

- התוכנית מאוחסנת בזיכרון כפקודות בשפת מכונה (לאחר שהקומפיילר הפך אותם לפקודות בשפת מכונה).
- תפקידה של יחידת הבקרה הוא לבצע תוכניות על ידי חזרה על השלבים הבאים:
 1. שליפה – Fetch: שליפת הפקודה הבאה לביצוע מהזיכרון
 2. פענוח – Decode: קביעת הפעולה שיש לבצע
 3. ביצוע – Execute: ביצוע הפעולה ע"י שליחת אותות מתאימים ל-ALU, לזיכרון ולתתי המערכות של הקלט/פלט. **תמיד שלב זה מתבצע מבחינת הקורס שלנו.**

הוא ממשיך לבצע את 3 הפעולות הללו, אחת אחרי השנייה: עד לפקודת עצירה.

פקודות בשפת מכונה:

פקודה בשפת מכונה מורכבת מ:

1. קוד פעולה: מציין איזו פקודה עלינו לבצע.
2. שדות כתובת – מציינים את כתובות הזיכרון של הערכים שעליהם הפקודה שלנו תפעל.

Opcode (8 bits)	Address 1 (16 bits)	Address 2 (16 bits)
00001001	0000000001100011	0000000001100100
קוד פעולה 9	כתובת זיכרון 99	כתובת זיכרון 100

לדוגמה: add x,y: אנחנו צריכים לקרוא את כתובת הזיכרון של x ואז את הכתובת של y וכן יש את ה-OPCODE של הפקודה. וזה יראה כך:

PC (Program counter) – רגיסטר ששומר את הכתובת של הפקודה הבאה בתור לביצוע. בעת שפקודה נשלפת מהזיכרון, הוא מתעדכן כדי להצביע על הפקודה הבאה בזיכרון.

IR (Instruction Register) – רגיסטר שמחזיק את הפקודה עצמה שנשלפה כרגע מהזיכרון, שומר את ה-OPCODE שעשינו לו Fetch מתוך הזיכרון.

Instruction Decoder: מפענח הפקודות. יחידה לוגית שמקבלת את הפקודה מה-IR ומפענחת אותה. הוא מתרגם את הקוד הבינארי של הפקודה לאותות חשמליים שמפעילים את הרכיבים הרלוונטיים. הוא הרכיב החומרתי שמופעל בעת ביצוע Decode לפקודה.

מבנה מערכת המחשב:

ניתן לחלק את מערכת המחשב לארבעה מרכיבים:

- אפליקציות (Applications): מגדירים כיצד יבוצע השימוש במשאבי המערכת על מנת לפתור בעיות חישוביות של משתמשים. היא יכולה להיות קומפיילר, משחק וידאו, אתר וכו'.
- משתמשים: לא בהכרח מדובר באנשים – גם מחשבים מרוחקים הם משתמשים. למשל עבור web server ישנם מחשבים שפונים אליו כבקשות ומוגדרים כמשתמשים.

סיכום מערכות הפעלה | © גיא יער-און

- מערכת ההפעלה: נמצאת מעל החומרה. היא שולטת על ומתאמת את השימוש במשאבי החומרה בין התהליכים והמשאבים השונים. מדובר על מערכות שיש בהן הרבה מאוד תהליכים שרצים – ואיננו יודעים מי מבין התהליכים יותר חשוב או יקר. עלינו רק לשלוט ולנהל אותם.
- חומרה (Hardware): ה"ברזלים". אילו משאבי מחשב בסיסיים יש לנו? CPU, Memory, IO Devices וכו'.

הגדרה: מערכת הפעלה היא תכנית אשר משמשת כחוצץ בין המשתמש של מערכת המחשב והחומרה של מערכת המחשב. מערכת ההפעלה נמצאת במכונות, במכשירים ביתיים, בטלפונים, מחשבים אישיים ותעשייתיים וכו'. מטרותיה: הרצת תוכניות המשתמש בצורה קלה, הפיכת השימוש במחשב לנוח יותר ושימוש יעיל יותר בחומרת המחשב.

נשים לב: מערכת ההפעלה מוגדרת כתוכנית גם כן – ולכן דורשת משאבים גם בעצמה. כלומר: גם כשלכאורה המחשב נראה שלא עושה כלום, המעבד מריץ קוד של מערכת ההפעלה.

שאלה. האם יכולה להיות מערכת מחשב ללא מערכת הפעלה? (כלומר, אם אפשר לקנות מחשב ולא להתקין עליו מערכת הפעלה).

תשובה. בתכלס, כן, אבל. זה ייצור עבורנו הרבה מאוד בעיות:

- נצטרך לכתוב תוכניות בקוד מכונה.
- לא נוכל להריץ יותר מתוכנית אחת בו זמנית.
- נצטרך לטפל בעצמנו בכל המקרים והתגובות – שגיאות, תזוזת עכבר, לחיצה על מקשים וכו'.
- נצטרך לנהל אינטראקציה עם החומרה בעצמנו.

מתי כן נראה את זה? במערכת סגורה – שבה אין אינטראקציה עם האדם, שהופך את התנהגות התוכנית ללא דטרמיניסטית. הרי, אם המערכת סגורה וידועה והכל בה דטרמיניסטי אז אין לנו צורך במערכת הפעלה (שדורשת הרבה משאבים).

סוגי קטגוריות מחשוב:

- **Supercomputer:** מחשב על. מחשב נמצא בחזית יכולת החישוב, במיוחד במהירות החישוב. מחשב מאוד יקר אותו מדינות/אוניברסיטאות קונות.
- **Mainframe:** מחשבים חזקים (מבחינת אמינות, זמינות וביצועים). משמשים בעיקר ארגונים גדולים ליישומים חשובים – כמו רישום אוכלוסייה, ניהול מלאי וכדומה. מהירות פחות גבוהה משל מחשב העל, אך עדיין גבוהה.
- **Personal Computer (PC):** כל מחשב רב תכליתי שגודלו, יכולותיו ומחירו המקורי הופכים אותו לשימושי עבור משתמשים פרטיים. אתה מוכר אותו עבור אדם ואינך יודע מה הוא יעשה בו – אם הוא ישתמש בו עבור מוזיקה בנטפליקס, או שיכתוב בו קוד.
- **Handheld:** דומה ל-PC, מכשיר מחשב אך עם סוללה שניתן לתפעול ביד אחת. הוא כולל בדרך כלל מסך תצוגה עם קלט של מגע. (כמו טלפון סלולרי).

לכל מערכת הפעלה יש מטרות ועיצוב שונים. למשל:

- **Mainframe:** הפוקוס של מערכת ההפעלה יהיה על ניצול מקסימלי של משאבי חומרה – פחות נוחות המשתמש, יותר ניצול מקסימלי של החומרה.
- **PC:** תמיכה מקסימלית בהרצת תוכניות של המשתמש – נוח עבור המשתמש.

UI-User Interface: המעטפת הוויזואלית והאינטראקטיבית שדרכה המשתמש מתקשר עם המכונה (כמו כפתורים, תפריטים ועיצוב המסך).

סיכום מערכות הפעלה | © גיא יער-און

- **Handheld**: שיהיה נוח להריץ אפליקציות, כי אין מקלדת/עכבר. נרצה גם ביצועים טובים פר יחידת ניצול של הסוללה.

בבירור – הנוחות מגיעה על חשבון היעילות. כמו תמיד, נאלץ לוותר על חלק מהיעילות עבור הנוחות ולוותר קצת על הנוחות עבור היעילות.

תפקידיה של מערכת ההפעלה:

- **Resource allocator**: בדומה לממשלה, שמקצה כמה מהתקציב ילך לכל תחום (חינוך/בריאות וכו'), מערכת ההפעלה מנהלת את המשאבים, היא מחליטה במצבים של קונפליקט על מנת שהשימוש במשאבים יהיה יעיל והוגן.
- **Control program**: שולטת על ההרצה של התוכניות השונות על מנת למנוע שגיאות, ולמנוע שימוש מוטעה במערכות המחשב.

תפקידיה חשובים מאוד בעיקר כאשר ישנם כמה משתמשים שמחברים לאותו mainframe.

שאלה. האם צייר, סוליטייר וכו' נחשבים כחלק ממערכת ההפעלה?

תשובה. אין תשובה אחת. יש כאלו שסוברים שכן – כל מה שהגיע בהתחלה כחלק מדיסק ההתקנה של מערכת ההפעלה הוא חלק ממערכת ההפעלה, ויש כאלו שסוברים שלא, והם לא חלק ממערכת ההפעלה.

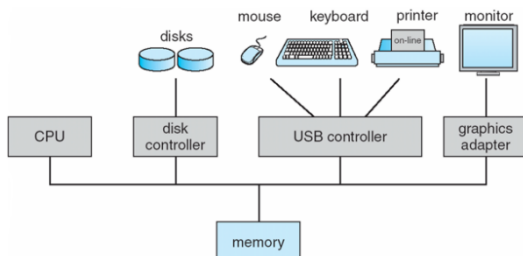
הגדרה: התוכנית/תהליך שרץ כל הזמן במערכת המחשב נקרא kernel. כל שאר התהליכים שרצים ע"פ דרישה מקוטלגים כ system programs (תוכניות שהגיעו יחד עם מערכת ההפעלה). למשל – הצייר מקוטלג כך. הוא הגיע עם מערכת ההפעלה, אך לא רץ כל הזמן. לעומת זאת application programs אלו דברים כמו chrome, הורדנו למחשב אך זה לא הגיע עם מערכת ההפעלה. כאשר אפליקציה רוצה לקרוא קובץ, להפעיל מצלמה, להקצות זיכרון – kernel מטפל בזה. הקרנל נותן יכולות בסיסיות, ותוכניות המערכת הופכות אותן למשהו נוח לשימוש.

Middleware frameworks: מדובר בשכבת ביניים שמטרתה היא לתת למפתחי אפליקציות שירותים מוכנים מבלי שיצטרכו להתעסק ישירות בפרטים הנמוכים של הקרנל. זה עדיין יעבור פונקציונאליות של הקרנל.

הפעלת המחשב: נניח וקנינו מחשב חדש, עליו כבר מותקנת מערכת ההפעלה. לחצנו על כפתור ההדלקה. מה קורה? ברגע שהדלקנו את המחשב ישנו תהליך של reboot. עולה **reboot program**: מדובר בתוכנית דטרמיניסטית שסגורה על עצמה ויודעת להתקדם קדימה ללא עזרה מהמשתמש – היא מאוחסנת ב-EPROM (רכיב חשמלי, לא נמחק בניגוד ל RAM, בעת שנסגור את המחשב ונדליק יום למחרת הוא יישאר שם). תוכנית זו מאתחלת את כל מרכיבי המערכת – תוכן הזיכרון, הרגיסטרים של המעבד וכו'. בנוסף: היא זו שמעלה את kernel של מערכת ההפעלה ומעבירה אליה את השליטה. בנקודה זו – מערכת ההפעלה הופכת לעצמאית, כל הקוד שלה טעון ומרכיביה מאותחלים. ואז – מערכת ההפעלה לא עושה כלום: היא מחכה, עד שיהיה אירוע כלשהו: שהמשתמש יפעיל את העכבר, יזיז משהו וכו'.

מעבדים:

מערכת ההפעלה מאופיינת ע"י מעבד (אחד או יותר, כיום יש כמה וכמה) Device 1
controllers: כמו Disk controllers controller וכו'. כל אחד מהם אחראי על סוג מסוים של devices. הם מחוברים באמצעות common bus שמאפשר להם גישה לזיכרון משותף, כך:



כל device controller אחראי על סוג מסוים של device. המחשב לא יודע כיצד לדבר ישירות עם המנוע של הארד דיסק, ולכן הוא מדבר עם הבקר והבקר יודע כיצד לנהל את המכשיר הספציפי שמוטאם לו.

סיכום מערכות הפעלה | © גיא יער-און

המידע מ ואל device מנוהל באמצעות Local Buffer: המעבד מעביר מידע מ/אל הזיכרון הראשי אל הלוקאל באפר. כיוון שהתקני IO איטיים מאוד בהשוואה למעבד, המידע לא יעבור ישירות מהדיסק למעבד. הלוקאל באפר הוא זיכרון מקומי קטן. המידע מהמכשיר נאסף קודם כל בבאפר של הבקר, כשהמידע מוכן הוא מועבר אל הזיכרון הראשי, ומשם למעבד (ולהפך). במקום שהמעבד ישב וישאל כל הזמן "סיימת? סיימת?" – הוא בונה על כך שבעת שהdevice controller יסיים הוא ישלח interrupt: לכן הוא יכול להמשיך לבצע פעולות אחרות תוך כדי.

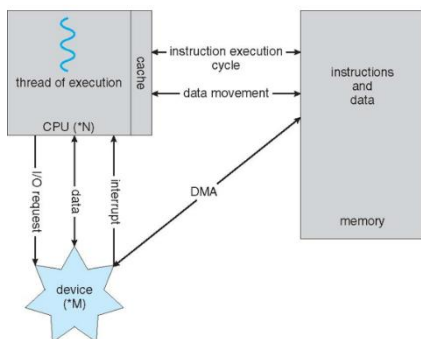
ברוב המערכות נמצא לפחות מעבד אחד מסוג **general purpose**: מעבד שיועד לעשות הכל, הוא לא נבנה בצורה שנותנת יתרון ליישום מסוים – הוא נבנה בצורה כללית ויועד לבצע את הכל. לצד מעבד זה, מוסיפים מעבדים נוספים מסוג **special purpose**: למען מטרה מסוימת. למשל – GPU (מעבד עבור גרפיקה). לצד המעבדים הנוספים, תמיד נהיה חייבים את המעבד הכללי.

מערכות **Multiprocessors** הן מערכות בהן יש יותר ממעבד אחד – ישנה תקשורת בין המעבדים, שחולקים את אותו bus ולעיתים גם את הזיכרון. מה היתרונות של מערכת שכזו?

- מהירות, כמובן.
 - כלכלי: אך למה שלא נקח מעבד פי 2 יותר מהיר, במקום לקחת 2 מעבדים? זה מגיע כמובן מבחינה כלכלית – לרוב ליצור מעבד מהיר פי 2, הרבה יותר יקר מאשר ליצור 2 מעבדים במהירות שקולה.
 - בעת שיש תקלה, אם יש לנו רק מעבד אחד תקלה במעבד משביתה את המערכת כולה. בעת שיש כמה מעבדים, הם יכולים לכפות על המעבד שהתקלקל – המהירות תרד, אך המערכת לא תושבת.
- ישנן שתי ארכיטקטורות של ריבוי מעבדים:

- **Asymmetric Multiprocessing**: משימות מסוימות מוקצות למעבדים מסוימים בלבד. בפרט, יתכן כי מעבד יחיד יהיה אחראי על טיפול בכל interrupts במערכת ואולי אף יבצע פעולות קלט/פלט.
 - **Symmetric Multiprocessing**: מתייחס לכל רכיבי העיבוד במערכת באופן זהה – אפשר לפנות לכל המעבדים בעת דרישה.
- בכל פעם שנאמר בקורס core: הכוונה למעבד. (זה לא כך בפועל, אך זה הדיון במסגרת הקורס).

- **Clustered systems**: דומה לריבוי מעבדים, אך בריבוי מעבדים ישנם כמה מעבדים שחולקים אותו לוח אם ואותו זיכרון: כאן מדובר במערכות נפרדות (בזיכרון, לוח אם) שעובדות יחד. הן חולקות אחסון משותף דרך רשת שנקראת SAN. היתרון הגדול הוא הזמינות – גם אם אחת המכונות קורסות, השירות לא יקרוס. בדומה, ישנן 2 שיטות לגישה זו:
1. **Asymmetric Clustering**: ישנה מכונה אחת שהיא במצב "Hot-StandBy": היא לא עושה כלום, והיא רק מחכה אם אחת השרתים יקרוס: היא תכנס במקומו.
 2. **Symmetric Clustering**: כל המכונות עובדות ומריצות אפליקציות במקביל, אם אחת נופלת הן מתחלקות בעומס שלה. זה יעיל יותר כמובן כי כל החומרה מנוצלת.



איך מחשב מודרני פועל? יש לו מעבד – שמדבר גם עם הזיכרון, וגם עם device. בזיכרון – יש לנו גם פקודות, וגם דטה. המעבד יהיה זה שייקח דטה מהזיכרון, ויעביר אותו אל device: באמצעות device controller.

אם יש בקשה של device מהמעבד לבצע פקודות IO הוא ישלח בקשה: IO REQUEST, לשם כך לפעמים יהיה צריך להעביר data בין CPU לdevice. בסיום – device ישלח interrupt, עליה נרחיב כעת:

Interrupt: פסיקה – מדובר באות חשמלי שמתקבל במעבד מרכיב החומרה (דיסק, עכבר וכו'). נוצר ע"י device. הוא מעביר את השליטה ל **interrupt service routine (ISR)**. זו הדרך של device לומר "סיימתי": זו הדרך שלנו לעצור את מה שעושה המעבד. המעבר שעצר, יטפל כעת בinterrupt וכשהוא יסיים נוכל לחזור לעשות מה שעשינו קודם. הוא מחייב שמירה של ה כתובת של ה interrupted instruction ושל תוכן הרגיסטרים (על מנת שנוכל לחזור בהמשך למצב שבו היינו – עם ערכי הרגיסטרים שהיו לפני ביצוע ה interrupt). להכל יהיה interrupt: בכל פעם שה device controller יסיים משהו – הוא ישלח. נבחין כי בעת לחיצה של מקש במקלדת – ה device controller יטפל בכל התהליך – יזהה איזה מקש במקלדת הוקש וכו', ורק בסוף הוא יבצע interrupt.

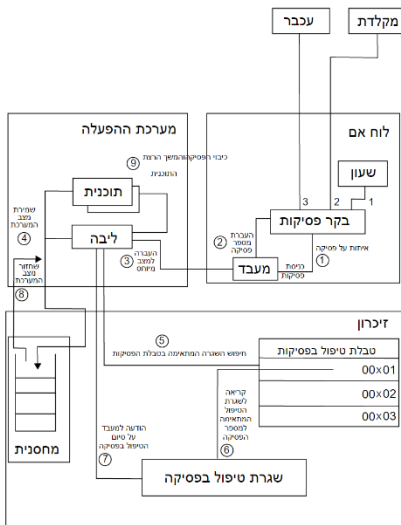
אם נסכם את התהליך, נעזר בתמונה משמאל להבנת התהליך:

שלב א': זיהוי – בלוח האם: התקן חיצוני (מקלדת/עכבר) שולח אות חשמלי לבקר הפסיקות. הבקר מעביר את האות אל המעבד. בקר הפסיקות שולח למעבד את ה"זהות" של הפסיקה (מספר הפסיקה. למשל: "פסיקה מספר 2 הגיעה דרך המקלדת")

שלב ב': החלפת הקשר – במערכת ההפעלה: המעבד מפסיק את ריצת התוכנית הרגילה ועובר ל kernel mode, על מנת שיוכל לגשת למשאבי מערכת מוגנים. כמו כן, המערכת שומרת את הנתונים הנוכחיים של התהליך (רגיסטרים, כתובות וכו') בתוך המחסנית על מנת שנוכל לחזור אליהם אחר כך ללא איבוד מידע.

שלב ג': איתור וביצוע – בזיכרון: חיפוש בטבלת הפסיקות: המעבד נגש לטבלת וקטורי הפסיקות בזיכרון ומחפש את הכתובת של הקוד שמתאים למס' הפסיקה שהתקבל בשלב 2. המעבד קופץ לכתובת שבטבלה ומצא, ומריץ את שגרת הטיפול בפסיקה (ISR), זהו הקוד שבאמת מטפל בלחיצת המקלדת או בתזוזת העכבר.

שלב ד': חזרה לשגרה – הודעה על סיום, המערכת מסיימת ומעדכנת את המעבד שהטיפול הושלם. שחזור מצב המערכת – שולפת מהמחסנית את הנתונים ששמרה ומחזירה אותם לרגיסטרים. המעבר חוזר למצב User Mode וממשיך להריץ את התוכנית בדיוק מהנקודה שבה נעצרה.



הרצאה 2

Trap: דומה לinterrupt, הוא לא אות חשמלי אלא הודעה שמעביר קוד שלנו. הוא software-generated interrupt. פונקציונלית, הוא גורם למערכת ההפעלה להתעורר ולעשות משהו, לכן הוא דומה לinterrupt, הוא נגרם בעקבות שגיאה/בקשה של תוכנית של המשתמש.

Interrupt Handling: כאשר נקבל interrupt ישנן 2 גישות עיקריות כיצד לטפל בו, ראשית עלינו להבין מה סוג interrupt שקרה. הגישות:

- **Polling:** יצרנו מראש קוד גנרי שיודע לטפל בכל interrupt שיגיעו (יודע לתת את כל התגובות לכל המקרים). ואז – כשנקבל interrupt נריץ את הקוד הגנרי ונבין באמצעות הקוד הגנרי מה קרה.
- **Vectored interrupt system:** יחד עם interrupt נקבל מס' כלשהו (למשל: מס' 7 הוא הודעה של המדפסת, מס' 3 הוא הודעה מהשעון וכו'). באמצעות המס' הנ"ל אנחנו יכולים ללכת אל טבלה בזיכרון – להגיע אל המקום המתאים בטבלה, ושם נמצא את כתובת הקוד שצריך לטפל בinterrupt הנ"ל.

בבירור, לכל interrupt יהיה קטע קוד רלוונטי (routine), ששמור בזיכרון. מהם היתרונות/חסרונות של כל אחת מגישות הטיפול?

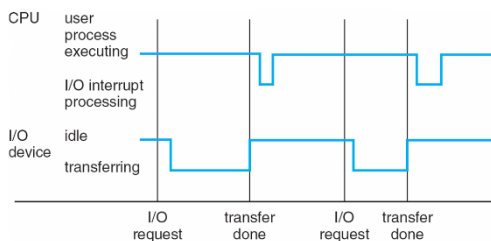
בגישת Vectored: נקבל שזמן הטיפול יהיה הרבה יותר מהיר – שכן נמצא את קוד הטיפול הרבה יותר מהיר (זו מעין "טבלת האש"), אך – כנראה שיש הרבה כפילויות: טיפול בinterrupt שהגיע מדיסק מסוג מסוים A וinterrupt שהגיע מדיסק מסוגים B – יתכן חפיפה בקוד שלכאורה יכולנו לאחד אותם אם היה בידינו קוד מסוג Polling, ולכן אנחנו משתמשים בהרבה יותר זיכרון כי יש קודים נפרדים לכל אחד מהמקרים.

כיוון שהמהירות היא קריטית (המעבד עוצר, מחכה..) – רוב המערכות עובדות בגישת Vectored.

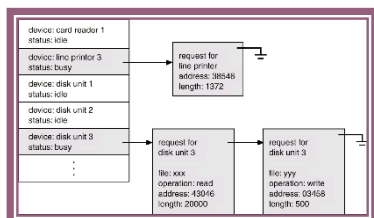
טיפול בIO:

סינכרוני: ברגע שהתוכנית של המשתמש מתחילה לרוץ, אנחנו מעבירים את השליטה לKernel שיפעיל את device controller ובאמצעותו את device עצמו, ועד שלא סיימנו את הטיפול בIO התוכנית של המשתמש לא תמשיך לרוץ

אסינכרוני: מבקשים את IO ממערכת ההפעלה באמצעות device controller ומיד מחזירים את השליטה לתוכנית של המשתמש – ברגע שהdevice יסתיים הוא ישלח interrupt. אך, נבחין כי יתכן ויהיה תלות להמשך קוד המשתמש בIO: ולכן במקרים אלו נעבוד בצורה סינכרונית. אם רק רוצים להדפיס למסך למשל – ואין תלות בהמשך הקוד בIO: נעבוד בגישה אסינכרונית.



משמאל – דוגמה למצב אסינכרוני: המשתמש רץ עד לקו השחור הראשון, שם המשתמש מבקש IO (למשל: לפתוח קובץ). במצב זה – מדובר בsystem call והשליטה עוברת למערכת ההפעלה – היא מריצה קוד בו היא מעבירה לdevice controller את המידע לרגיסטרים שהוא זקוק (זה החלק בין שני השחורים ש"נופל"), במקביל – כיוון שאנחנו באסינכרוני, תוכנית המשתמש ממשיכה לרוץ, עד לקו השחור ששם device controller סיים ושולח interrupt למעבד – ועובר להריץ קוד שמטפל באותו interrupt (זה המעבד) – זה ה"נפילה" בין הפס השחור השני לשלישי. לאחר מכן – התוכנית ממשיכה לרוץ (וקורים תהליכים דומים).



- מה קורה במצב אסינכרוני שבו לא סיימנו לטפל בבקשה שהגיעה, וכבר הגיעה עוד בקשה לdevice? לשם כך משתמשים במעין "תור", ישנה טבלה: Device-Status Table – כמו באיור משמאל. לכל device תהיה מעין שורה בטבלה, ובה הפניה ל"תור" בקשות.

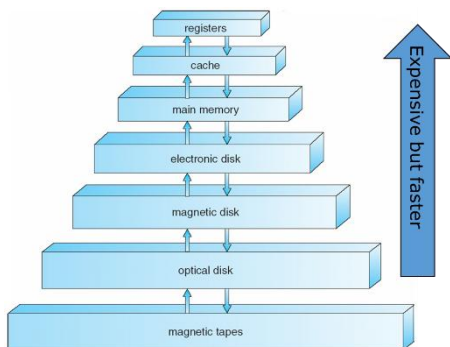
ניהול הזיכרון:

מדוע הזיכרון כל כך חשוב? instructions חייבות להיות בזיכרון על מנת שנוכל לבצע אותן, וגם data חייב להיות בזיכרון. ניהול זיכרון עוסק בהחלטה על מה יהיה/ישהה בזיכרון בכל רגע ורגע. מדוע שלא נשמור הכל בזיכרון בכל רגע נתון? כי הזיכרון ממש יקר – ומוגבל במקומו, ולא נוכל להחזיק הכל בזיכרון בכל רגע נתון.

תפקיד מערכת ההפעלה בנוגע לניהול הזיכרון:

- היא רושמת ועוקבת על החלקים בזיכרון שכרגע נמצאים בשימוש.
 - מחליטה על העברה של תוכן זיכרון שמשויך לתהליך מסוים – מ/אל הזיכרון: זה נקרא swapping. למשל: להעביר אל דיסק. מדוע שנעשה זאת? כי לפעמים אין מספיק מקום בזיכרון.
 - מקצה זיכרון לתהליכים, ולוקחת זיכרון מתהליכים, ויכולה לשפר זיכרון לתהליכים (לתת עוד זיכרון).
- Mass Storage Management:** פרט לזיכרון הראשי (אמצעי אחסון הגדול היחיד שהמעבד יכול לגשת אליו בצורה ישירה), ישנם דיסקים. מדוע שנשתמש בדיסקים?

- נרצה לאחסן נתונים שלא יכולים להכנס לRAM בגלל גודלם.
 - נרצה לאחסן נתונים שיהיו שם גם לתקופה ארוכה – גם אחרי שנכבה את המחשב.
- ניהול נכון של Mass storage (הדיסקים) משפיע מאוד על המהירות הכוללת של פעילות המחשב – כיוון שגישה לדיסק יקרה מאוד. תפקיד מערכת ההפעלה בהקשר: מנהלת את המקום הפנוי, מקצה מקום אחסון ומתזמנת פעולות דיסק.



משמאל: היררכיית Storage. ככל שנעלה בהיררכיה, התקן האחסון שנפגוש יהיה יותר מהיר, העלות שלו תהיה הרבה יותר יקרה פר כמות אחסון. משלב מסוים גם הזיכרון הופך לנדיף – הרגיסטרים והקאש נדיפים (נדיף הוא זיכרון שזקוק לזרם חשמלי רציף כדי לשמור על המידע שנמצא בתוכו. ברגע שמכבים את המחשב או שיש הפסקת חשמל – כל המידע שלא נשמר על הדיסק הקשיח נמחק לצמיתות).

שאלה. האם יתכן ואדם יגיע יום אחד – ויאמר שהצליח ליצור רמה בהיררכיה שנמצאת גבוהה יותר שהיא גם מהירה יותר וגם זולה יותר?

תשובה. כן: יכול להיות, אבל מה ההשלכות? היא תעלים מההיררכיה את כל מי שנמצא מתחתיה – כי לא נצטרך להשתמש בהם יותר. הרי היום משתמשים בדיסקים כי הם זולים יותר, למרות המהירות הלא טובה שלהם. אם נמצא זאת: לא נצטרך יותר להשתמש בכל מי שמתחתיו בהיררכיה. הערה: הגודל הפיזי של הרכיב – גם עשוי להיות שיקול (שכן אם יגיע אדם ויציע טכנולוגיה כזו – רק שהיא דורשת חדר שלם לאחסון, זה יהפוך לשיקול).

Caching: העתקת מידע למערכת אחסון מהירה יותר. למשל: אם נרצה לקחת מקובץ שנמצא בדיסק פעולת חישוב 2+3, זה יעבור מהדיסק לזיכרון הראשי, אל הקאש ואל הרגיסטרים.

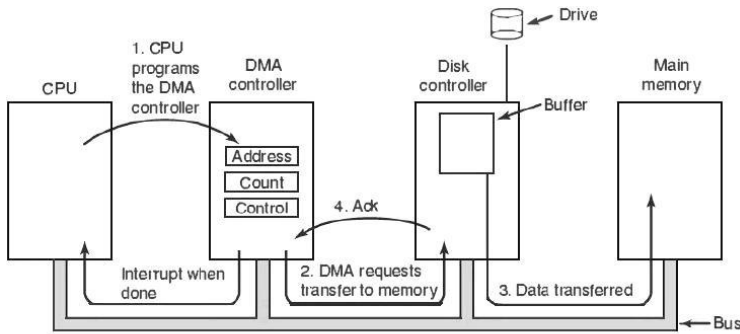


Figure 1. Operation of a DMA transfer.

נזכר בתמונה מעמוד 7 – הופיע שם DMA: כעת נסביר עליו.

המטרה של DMA היא "לעקוף" את המעבד. בשיטה הרגילה – המעבד צריך לקרוא כל בייט מהdevice ולכתוב אותו לזיכרון: מדובר בפעולה בזבזנית. כעת, dma מאפשר להתקן קלט/פלט להעביר בלוקים של מידע ישירות לתוך RAM מבלי שהמעבד יצטרך לגעת בנתונים בדרך.

בתמונה משמאל נראה תיאור של התהליך בו המידע עובר ישירות מהdevice אל הזיכרון הראשי.

שלב ראשון: המעבד לא רוצה להתעסק בהעברה של כל בייט ובייט – לכן הוא ניגש אל הDma controller ומטעין אותו בנתונים – כתובת (לאיזה כתובת בזיכרון לשפוך את המידע), כמה מידע להעביר, וcontrol: סוג הפעולה (קריאה/כתיבה).

שלב שני: הDma controller פונה לDisk controller ומבקש ממנו להתחיל להעביר נתונים.

שלב שלישי: בקר הדיסק שולף מידע מהDrive ושם אותו בבאפר: משם המידע עובר על הbus ישירות לזיכרון הראשי. המעבד לא רואה את המידע הזה עובר ולא נוגע בו – הנתונים נשלחים ישירות לזיכרון.

שלב רביעי: Ack – האישור: בכל פעם שבלוק מידע עובר בהצלחה מהבקר לזיכרון נשלח סינגל Ack חזרה לDma controller בשביל לעדכן את count (כמה נותר להעביר). כאשר count=0 הDma controller שולח Interrupt למעבד – רק עכשיו המעבד עוצר לרגע, מבין שהמידע מוכן בזיכרון הראשי ויכול לעבוד איתו.

Multiprogramming: היום, במחשב שלנו בבית אנחנו מריצים כמה תהליכים בו זמנית (למשל: פותחים סביבת פיתוח ומתכנתים, ותוך כדי וורד פתוח, ומאזינים לשיר בנגן וכו') – הרבה פעמים נוצר מצב בו כמות התהליכים גדולה מכמות המעבדים. זהו מצב של multiprogramming. כאשר יש תוכנית בודדת שרצה, אי אפשר לשמור על המעבד והתקנים השונים בנצילות מלאה. אנחנו מעוניינים לשפר את הנצילות של המעבד – ולכן אידיאלית נרצה שבכל רגע נתון תהיה תוכנית שרצה על המעבד. multiprogramming מאפשר כמות תהליכים גדולה מכמות המעבדים, בצורה של שיתוף: כשתהליך מסוים למשל פונה IO או מסיים הרצה, מתפנה מקום לתוכנית אחרת. ולכן ישנו מעין תור של תהליכים – צריך לנהל זאת כמובן: אך בכל פעם תהליך אחר מקבל זמן מעבד.

Timesharing: עדיין מדובר בהרצה של כמות תהליכים שגדולה מכמות המעבדים, אך דואגים להחליף אותם על גבי המעבדים בתדירות כל כך גבוהה כך שכל התהליכים שהמשתמש מריץ ברגע זה רצים כרגיל – והוא לא רואה אף אחד מהם נתקע או מחכה שיתפנה זמן מעבד. תחושת המשתמש היא שכל התהליכים רצים ברגע נתון, גם אם הם לא באמת מקבלים כל שניה זמן מעבד. נדגיש כי:

- זמן התגובה חייב להיות פחות משנייה אחת.
- לכל משתמש יש לפחות תוכנית אחת בהרצה בזיכרון.
- אם לא ניתן להכיל את כל התהליכים בזיכרון – מתחיל swapping בין התהליכים.

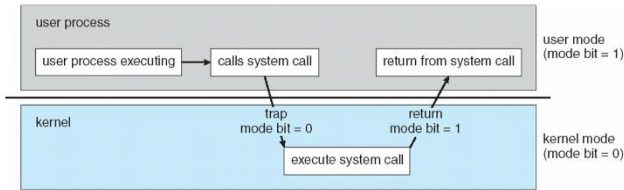
מעבר בין User mode לKernel mode:

מערכת ההפעלה צריכה לפעמים להריץ קוד שלה על מנת לבצע משהו עבור תוכנית של המשתמש – למשל בsystem call. הקוד של מערכת ההפעלה הוא יותר בטיחותי כי ישבו עליו הרבה ודיבגו אותו והוא לא ינסה לעשות דבר זדוני. היינו רוצים שהקוד הזה יעשה יותר. ישנן פקודות שנרצה להגדירן privileged: פקודות שרק מערכת ההפעלה יכולה לבצע – המשתמש לא.

סיכום מערכות הפעלה | © גיא יער-און

כיצד נדע האם אנחנו ב-User Mode? ישנו ביט מיוחד mode bit: אם ערכו 1 אנחנו ב-user mode, ואחרת אם הוא 0 אנחנו ב-kernel mode. אם אנחנו ב-user mode – לא נאפשר להריץ פקודות שהוגדרו כprivileged.

בעת שנקרא ל-system call, נשלח trap למערכת ההפעלה וה-bit mode הופך להיות 0. מכאן: השליטה עוברת למערכת ההפעלה. בסיום, ה-bit mode הופך חזרה להיות 1: וחוזרים ל-user mode.



מבנה מערכת ההפעלה

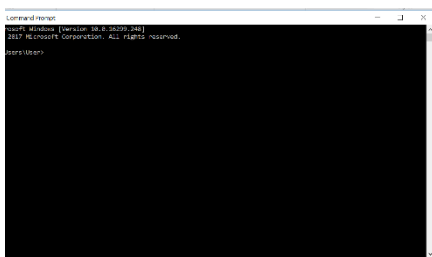
שאלה. מה הופך מערכת הפעלה להיות "מערכת הפעלה"? הרי – לכל מכשיר יש מעין ממשק למשתמש.

תשובה. איפה שיש ממשק משתמש בלבד – כמו בכפתורים במיקרוגל או במכונת הכביסה, במקומות כאלו הממשק הוא מעין שכבת תקשורת בין משתמש למכשיר. הממשק הנ"ל לא מייצר ניהול משאבים – כפי שנותנת מערכת ההפעלה. למעשה, מכשירים פשוטים יכולים לפעול עם תוכנה דטרמיניסטית פשוטה שמנהלת מס' פעולות מוגבלות מוגדרות בראש ואין צורך שיתעסקו בניהול משאבים. כלומר: למערכת הפעלה יש עושר בניהול משאבים ובמורכבות של ניהול המשאבים.

Director: סט של הוראות מחשב – שמגדיר פעולה רציפה שמורכבת מכמה וכמה פעולות. מדובר בראשית של מערכות ההפעלה. לפני שהופיע director הכל היה מורכב מהתערבות ידנית של מפעילים אנושיים – היה צריך להזין/לפקח/לקבוע דברים שונים. לאחר כניסת director (שנות 60) – היה שינוי גדול, היה אפשר להפעיל דברים הרבה יותר מורכבים שקודם בני אדם לא יכלו לפקח עליהם (או שהיה קשה לפקח עליהם).

Services שמציעה מערכת ההפעלה

ממשק משתמש – User interface (UI): כמעט לכל מערכת הפעלה יש UI (למערכות ישנות אין). לרוב נראה כמה וכמה סוגים בכל מערכת הפעלה. וישנם כמה סוגים שונים:



- **CLI: Command Line interface** – המשתמש מזין פקודה, ה-CLI לוקח את הפקודה (fetch) ומבצע אותה – הופך אותה לרצף של system calls.

כיצד מממשים CLI?

1. כחלק מ-kernel: תמיד נמצא בזיכרון, וה-command interrupter קופץ למקום המתאים בזיכרון. מהירות גבוהה מאוד! קפיצה ישירה לכתובת בזיכרון. אך התחזוקה והגמישות נמוכה – שינוי דורש עדכון של טבלת הכתובות בקרנל. כאשר משתמש מקליד פקודה, הקרנל מזהה שמדובר בפקודה ומבצע קפיצה לכתובת בזיכרון שבה נמצא הקוד של אותה פקודה בתוך הקרנל. אין כאן קונטקסט סוויץ', שכן ה-shell הוא תוכנית רגילה. הביצוע מידי לכן המהירות גבוהה מאוד. החיסרון: חוסר גמישות – הקרנל הוא "קוד קדוש". אם רוצים להוסיף פקודה חדשה, לעדכן את הממשק או לתקן באג ב-CLI יש לקמפל מחדש את הקרנל כולו ולבצע Reboot למערכת.
2. לכתוב תוכניות, כך שכל תוכנית מממשת פקודה אחרת (ושם התוכנית יהיה זהה לשם הפקודה) ולשמור את התוכנית בדיסק. למשל – אם ב-Unix נבצע `rm file.txt` – המערכת תחפש קובץ בדיסק שקוראים לו `rm`: בתוך הקובץ היא תמצא תוכנית – תעשה loading לתוכנית לזיכרון, ותשתמש ב-`file.txt` כקלט לתוכנית שרצה. המהירות איטית – צריך ללכת אל הדיסק לחפש קובץ. אך הגמישות גבוהה: ניתן להוסיף פקודות מבלי לשנות את הקרנל.

- לעיתים נמצא מס' interpreters במערכת הפעלה: ואז נקרא להם shell.
- GUI: ממשק משתמש גרפי, הוא נותן את המטאפורה של "שולחן עבודה", עכבר, קיצורי דרך (גרירה עם העכבר, משיכה, עבודה עם המקלדת וכו').

ברוב המערכות נמצא גם CLI וגם GUI – מדוע בעצם? ה-CLI הרבה יותר מהיר – ובאמצעות GUI צריך לבצע הרבה יותר פעולות על מנת לבצע את אותה פקודה פשוטה דרך ה-CLI. אך: ה-GUI הרבה יותר פשוט ואין סינטקס.

- ישנם עוד ממשקי משתמש רבים – למשל VUI: Voice User interface, ממשק משתמש מבוסס מגע, ממשק משתמש מבוסס מציאות מדומה – למשל VR Interface וכו'.

היכן עוד מסייעת מערכת ההפעלה למשתמש?

- הרצת תוכניות (טעינת תוכנית לזיכרון, הרצת התוכנית, סיום הרצה. כאשר התוכנית מסתיימת מעצמה, או כאשר התוכנית מסתיימת בצורה לא טבעית – יש להצביע על השגיאה ולטפל בה).
- טיפול ב-IO – נדרש קלט/פלט בתוכנית, נרצה שהתוכנית לא תגש ישירות כמובן – תמיד לפנות למערכת ההפעלה (הגנה על המשאב, ויעילות השימוש במשאב).
- טיפול בקבצים (נדון על כך בהמשך הקורס – קריאה מתוך קבצים, כתיבה לקבצים וכו').
- תקשורת בין תהליכים: העברת מידע בין תהליכי שפועלים באותו מחשב / שפועלים במערכות מרוחקות ומקושרים באמצעות רשת.
- טיפול בשגיאות – לא רק שגיאות תוכנה, אלא גם שגיאות חומרה (בעיה פיזית בזיכרון, שגיאה בהתקני IO: למשל נגמר הנייר במדפסת / נפלה הרשת וכו'). בכל סוג כזה של שגיאה – מערכת ההפעלה צריכה לצאת לפעולה על מנת לעזור לנו בצורה היעילה יותר.
- לרוב מספקת מערכת ההפעלה Debugging facilities שמקלות על התכנית להבין מה קרה ולדבג.

כיצד מערכת ההפעלה שומרת על יעילות המערכת?

- הקצאת משאבים: כאשר ישנם כמה וכמה משתמשים מחוברים במקביל / תהליכים שרצים במקביל – מנהלת בהתאם.
- Accounting: ניהול רישום של השימוש במשאבים השונים לפי משתמשים לאורך ציר זמן – כל פעם שתהליך של המשתמש צריך להשתמש באיזשהו משאב, נרשם בצד באיזה משאב השתמש המשתמש שלי, מתי, וכמה זמן הוא השתמש. מדוע צריך זאת? משתי סיבות: 1. Billing – כלומר לחייב בהתאם לשימוש 2. עבור סטטיסטיקות עתידיות, שישמשו את המעבד בצורה מושכלת יותר בעתיד (על מנת להכיר את המשתמש).
- הגנה ואבטחה על משאבים:

Protection: בכל פעם שניגשים למשאבים של המערכת – ניגשים בצורה שהיא תקינה, יש בקרה של המערכת. למשל: אם משתמש רגיל שאיננו מנהל מערכת ינסה למחוק קובץ מערכת.

Security: אימות משתמש, הגנה על משאבי IO מגישה לא חוקית או זדונית. למשל: כשפותחים מחשב, נדרשים להזין סיסמה/טביעת אצבע.

System calls

מנגנון שמאפשר לתוכניות שרצות על המחשב לבקש שירותים מסוימים ממערכת ההפעלה. למשל: open, read, write, fork, exit וכו'. נרצה שמימוש של פקודות אלו יהיה דרך מערכת ההפעלה. לרוב הם כתובות ב-high level language: C או C++.

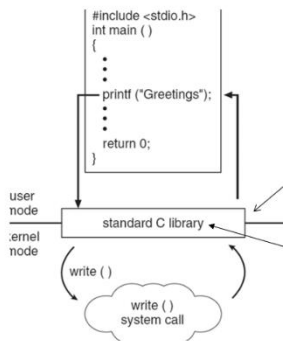
סיכום מערכות הפעלה | © גיא יער-און

לכל system call יוצמד מספר, ותוחזק טבלה שמאנדקסת לפי מס' זה ואז בעת קריאה ל system call נגיע לשורה זו בטבלה. בעת הקריאה, יפעיל system call interface את ה system call המתאים ב kernel ויחזיר את סטטוס הביצוע (ואת ערכי הפלט המתאימים).

דוגמה. המשתמש לוקח קובץ, ומעתיק את התוכן שלו לקובץ יעד. כמה system calls יתבצעו?

תשובה. כרגיל, ה"כן, אבל" יהפוך ללא ניתן לדעת: בשלב ראשון, יש לשאול את המשתמש מיהם הקבצים של היעד והמקור. זה כבר דורש IO (הדפסה למסך), לאחר מכן צריך לבצע open, אם בכלל הקובץ קיים – אם לא: או שנסיים את התוכנית וזה גם system call, או שנחזור למשתמש ונבקש (IO) שוב מהמשתמש שם תקין. נניח והגענו למצב שצלחנו את זה – בידינו הקבצים הרלוונטיים: אנחנו נעבוד בלופ, נקרא קצת מהמקור ונכתוב ליעד, אך יתכן וגם בשלב זה נתקל בהרבה בעיות – אין מקום בדיסק, וכו'... נניח וגם את זה סיימנו – נצטרך לעשות close לקובץ, ולבסוף גם exit. מסקנה: יתכן שלפעולה כזו פשוטה נצטרך מאות עד אלפי system calls.

לרוב לא נקרא ישירות ל system call מהתוכנית שלנו, אלא נקרא לקריאה שנמצאת ב API (Application programming interface): מגדיר את סט הפונקציות שזמינות לאפליקציה שלנו ובכלל את הפרמטרים שנדרשים כקלט לפקודה והערכים שמוחזרים להם ניתן לצפות – והוא יקרא ל system call. כלומר: אנחנו לא צריכים לדעת את המימוש עצמו של system calls אלא רק לדעת מה נדרש ממני לשלוח. נחדד, כי אם היינו קוראים ישירות ל system call – היינו צריכים לדעת את המס' המדויק של הקריאה שיכול להשתנות בין מערכות הפעלה, את אופן העברת הפרמטרים, הפורמט המדויק של הפלט וכו'. לכן נרצה להשתמש ב API. יתרה מזאת – אם יש לנו שתי מערכות מחשב שונות שמשתמשות ב API: נצטרך רק להכיר סינטקסית API, זה מאפשר לנו להעביר קוד ממערכת למערכת ולקמפל אותן ולא יהיה בעיה. ללינוקס (Unix) יש את posix api.



לדוגמה, משמאל: קוד C פשוט. בפועל, בקריאת printf מדובר בקריאה ל API של ספריית C, כעת ב API: הפונקציה printf תקח את הטקסט ותעבד אותו, תכין אותו ואת הפרמטרים הנדרשים להדפסה ובסוף תקרא ל write system call. נעיר כי ה API קורה כמובן ב user mode.

```
#include <unistd.h> // עבור פונקציית syscalls
#include <sys/syscall.h> // סביל את המגדרות של מספרי הקריאות (SYS_*)

int main() {
    char* msg = "Hello\n";
    syscall(SYS_write, 1, msg, 6);
    return 0;
}
```



```
write(1, "Hello\n", 6);
```

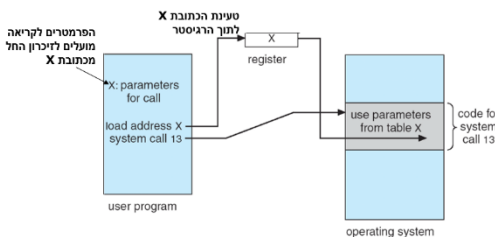
לצורך העניין, יכולנו גם לבצע את קריאת המערכת בעצמנו בצורה הבאה:

או להשתמש גם בפונקציית המעטפת write:

כאשר קוראים ל system call לרוב צריך להעביר פרמטרים ואף סט של פרמטרים. ישנן 3 צורות כלליות להעברת פרמטרים למערכת ההפעלה:

1. העברה באמצעות רגיסטרים. אך – הרבה פעמים צריך להעביר יותר פרמטרים מכמות הרגיסטרים שיש לי.
2. שמירת הפרמטרים כבלוק בזיכרון, והעברת הכתובת של תחילת הבלוק באמצעות רגיסטר.
3. שימוש במחסנית: הפרמטרים יוכנסו למחסנית באמצעות התוכנית הרצה ויימשכו ע"י מערכת ההפעלה. (pop)

לרוב, נראה את הצורה של בלוק בזיכרון. כך זה יראה בפועל (משמאל):



מערכת MS-DOS

מערכת זו פותחה לפני 60 שנים, והיא מסוג single tasking: משתמש יחיד ופרוסס יחיד.

כאן, נבחין גם לפי התמונה כי בעת ש-process עלה לזיכרון, command interpreter קטן. מדוע? כי בעת שעלה תהליך מסוים – יש הרבה ב-command interpreter שהופך להיות לא רלוונטי, לכן אפשר להקטין חלק ממנו לטובת זיכרון עבור התהליך. בעת שהתהליך הנ"ל יסתיים, command interpreter יטען מחדש חזרה לגודלו.

עם זאת, עברו מהר מאוד למערכת שתומכת במס' משתמשים. ישנם הרבה אתגרים שנוספו כתוצאה מכך:

- אתגרי אבטחה – מניעת גישה לא מורשית למשאבי משתמשים אחרים.
- ניהול משאבים – קודם לכן היה משתמש יחיד, כעת יש כמה משתמשים ותהליכים שונים וצריך לחלק את המשאבים בצורה הוגנת.
- ניהול זיכרון מורכב הרבה יותר.
- הפרדה בין תהליכים על מנת למנוע התנגשות / הפרעה בין תהליכים.
- לשמור על יעילות הביצועים.

על אלו באה לענות UNIX:

כאן, כבר רצים multiprocessing, ואי אפשר להקטין את גודל command interpreter, שכן יתכן שתהליך B לא זקוק לחלק מהמידע שם, אך תהליך C כן צריך.

System programs

מדובר בתוכניות שהגיעו עם מערכת ההפעלה, ישנם המון סוגים שכאלו שמגיעים עם מערכת ההפעלה. כל מיני תוכניות שקשורות בניהול קבצים – סייר הקבצים למשל, ישנן תוכניות שנועדו לתת אינפורמציה ממערכת המחשב (למשל: שעון, לוח שנה וכו'), שינוי תוכן של קבצים (כמו notepad וכו'), קומפיילרים, דיבאגריים, browsers שמקבלים עם מערכת ההפעלה וכו'.

הרצאה 3

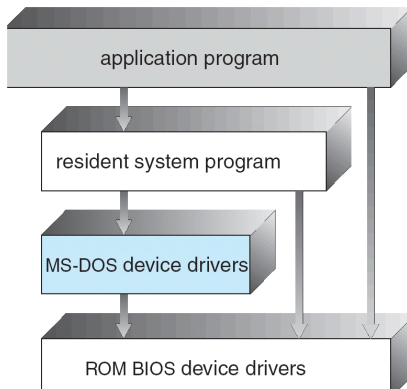
מימוש מערכות הפעלה

היכן נכתוב את הקוד של מערכת ההפעלה? הרי לבסוף, מערכת ההפעלה היא תוכנית.

- במקור, מערכות ההפעלה פותחו באסמבלי. בהמשך, עברו לשפות קצת יותר מתקדמות. כיום: רוב מערכות ההפעלה נכתבות ב-C++ או C. אך עדיין, ישנם חלקים במערכת ההפעלה שעדיין כתובים באסמבלי – החלקים שעדיין קרובים יותר לחומרה ודורשים את המהירות שמספקת אסמבלי. system programs (תוכניות שהגיעו עם מערכת ההפעלה) יהיו כתובות לרוב ב-C++.
- כאשר אנחנו באים לתכנן מערכת הפעלה – אנחנו צריכים לייצר ארכיטקטורה מסוימת. זו שאלה שאינה פתורה: יכולים להיות הרבה עיצובים למערכת ההפעלה. אבל – ישנם עקרונות כלליים שיכולים לעזור לנו שבאים לידי ביטוי בעיצוב של מערכות הפעלה, וכן יש עקרונות שנוגעים ספציפית בנושאים מסוימים שרלוונטיים למערכות מסוימות.
- **Batch**: פונקציונליות בה אנחנו מבקשים לבצע פעולה מסוימת (למשל: פעולת חיבור) אך לא מעוניינים לראות את התהליך עצמו שחישב אלא רק את התוצאה הסופית.
- **Time shared**: ההפך, כלומר כאן נרצה לראות את התהליך עצמו שחישב בנוסף לתוצאה הסופית.

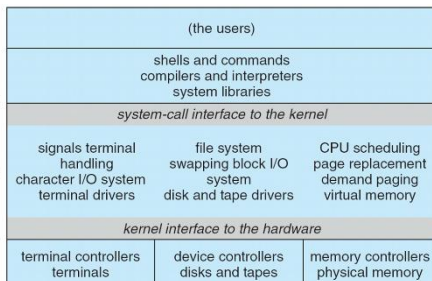
דוגמה ראשונה למבנה של מערכת הפעלה: MS-DOS

- מערכת מאוד ישנה, נוצרה בשביל לספק פונקציונליות מקסימלית במינימום משאבים. נועדה למשתמש יחיד, לא היה אפשר להריץ כמה תוכניות במקביל, נועד לשימוש ללא חיבור לרשת וכו'.
- הארכיטקטורה שלה משמאל. apd דיברו עם תחילתה של מערכת ההפעלה – והתוכנית הנ"ל העבירה את הדברים דרך device drivers לתוכנה עצמה.
- מה הבעיה בארכיטקטורה הזו? החיצים בין הרמות שלא סמוכות זו לזו. למשל, apd ל device drivers. שכן: היה אפשר לגשת ישירות לחומרה. הבעייתיות בכך, שיכולנו למחוק למשל מהדיסק את מערכת ההפעלה עצמה. אפשר להשתמש בחומרה בצורות שלא ייעודיות לכך מלכתחילה – ולנצל בצורה זדונית. ההנחה הייתה: שרוב הדברים שנעשים נעשים בצורה תמימה ונכונה – לאט לאט הופיעו איומים והבינו שעלינו להתמודד מול תוכנה זדונית.



דוגמה שנייה: Unix

- תוכנה לתמוך בכמה משתמשים שיריצו כמה תוכניות בו זמנית. כיצד נראית ארכיטקטורה זו? משמאל. עבדה ב"שכבות". בשכבה הראשונה – קומפיילרים, interpreters, בשכבה הבאה – kernel וכו'.



Layered approach: גישה למימוש מערכת הפעלה ב"שכבות". כל שכבה תלויה בפונקציונליות של השכבה לפנייה – כאשר שכבה 0 היא כבר פנייה ישירות לחומרה ושכבה N זו הממשק למשתמש. מה יתרונותיה של גישה זו?

סיכום מערכות הפעלה | © גיא יער-און

- Debug – ישנה הפרדה מלאה בין השכבות. ניתן להתחיל לדבג בשכבה מסוימת, ואם עדיין יש את הבאג הוא בהכרח בשכבות שמעליו. ניתן לצמצם את טווח החיפוש של הבעיה, ולמצוא מהר את הבעיה.

מה חסרונותיה?

- העיצוב עצמו, לרוב זה מסובך לחלק פונקציונאליות של מערכת הפעלה לשכבות מסוימות. שכן, יתכן שתוך עיצוב מערכת ההפעלה נגלה שאנחנו זקוקים לפונקציונליות משכבה מעל.
- יעילות – בכל פעם צריך לגשת לכל השכבות שלפניה.

Modules

רוב מערכות ההפעלה מפותחות בשיטה זו כיום, משתמשת בגישת תכנות מונחה עצמים. הרעיון הוא: במקום לבנות קרנל אחד ענק שכולל הכל מראש – או קרנל מינימליסטי שמעביר הכל למשתמשים, משתמשים במודולים:

- גישה דמוית תכנות מונחה עצמים – הקרנל מורכב מרכיבים נפרדים (כמו אובייקטים), ולכל רכיב יש תפקיד מוגדר וממשק ברור דרכו הוא מתקשר עם שאר המערכת.
- כל רכיב ליבה הוא יחידה עצמאית.
- טעינה דינמית: לא חייבים לטעון את כל הדרייברים כשהמחשב עולה, המודולים נטענים לזיכרון הליבה רק כאשר צריך אותם.

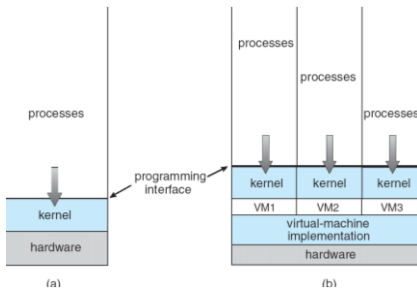
השוואה נחמדה: כשמקבלים טלפון אנחנו מקבלים את תוכניות המערכת עליו, ומתקנים בהתאם אפליקציות שנרצה. כך זה עובד בקרנל כאן – הליבה הבסיסית קטנה ומכילה רק מה שבאמת צריך, והמודולים הם קבצים נפרדים שיושבים בצד. אם למשל חיברנו מדפסת חדשה – הקרנל קורא למודול של המדפסת ומוסיף אותו לתוך עצמו בזמן אמת.

Virtual Machines

מדובר בתוכנית שתדמה מערכת הפעלה שיושבת מעל החומרה שלנו – למרות שהיא עצמה יושבת מעל מערכת הפעלה. הרעיון הוא, שבעת שנשתמש ב-Virtual machines נקבל ממשק שיושב מעל החומרה מתחת, ואז נוכל לכאורה להתקין מערכת הפעלה נוספת במחשב שלי.

בתמונה: דוגמה ל-virtual machine, מקבלים הפרדה מלאה.

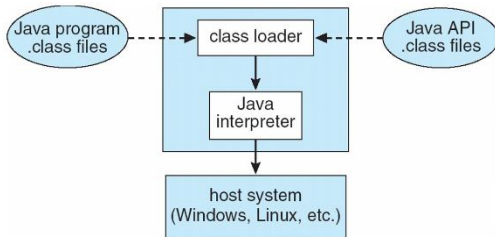
- אנחנו רגילים לחשוב על שימוש בזה על מנת לקבל מערכת הפעלה אחרת במחשב שלנו. אך, ייתכן שנשתמש גם במחשב לינוקס ובכל זאת נוריד מכונה וירטואלית. מדוע? על מנת ליצור הפרדה בין המשתמשים – כל אחד יקבל מעין שרת משלו באמצעות המכונה הווירטואלית.



יתרונות שימוש:

- אפשר לבדוק את אותה אפליקציה שאנחנו מפתחים על אותה מערכת הפעלה מבלי שנצטרך לקנות מחשבים נוספים – לא צריך עוד חומרה. גם לא צריך להעלות מחדש את המחשב (שהרי יכלנו לבצע dual-boot ולהעלות למחשב כל פעם מערכת הפעלה אחרת).
- הפרדה מלאה בין כמה תהליכים שרצים.
- מונע פגיעה במערכת ההפעלה האמיתית – אם היה לנו תהליך שבגלל משהו זדוני שעשה הקריס לי את מערכת ההפעלה, עכשיו הוא לא יכול לעשות את זה. הוא במקסימום יפגע במכונה הווירטואלית ויגרום לי להעלות אותו מחדש.
- אפשר להריץ אפליקציות שונות שפותחו במקור למערכות הפעלה שונות ממה שיש לי על המחשב – מבלי לבצע ריסטארט של המחשב.

- מייצרים עוד "שכבה" שצריך לעבור בדרך – וזה מאט את המהירות.
- לא נותן תאימות מלאה, תמיד ישנו מקום שלא נקבל את ההתנהגות המלאה של מערכת ההפעלה שנכתבה במקור.



JVM: המכונה הווירטואלית של ג'אווה. class loader מקבל קבצי class משני מקורות – הקוד שכתבת וספריות המובנות של ג'אווה. הקבצים הללו מקבלים bytecode שזהו קוד שרק JVM מבין ולא המעבד ישירות, interpreter לוקח את הbytecode ומעדכן אותו תוך כדי תנועה להוראות שמערכת ההפעלה יודעת להריץ. היתרון: בזכות המפרש לא צריך לשנות את הקוד שלנו.

System debugging:

Log files אלו הראיות הראשונות, המערכת כותבת כל הזמן מעין יומן אירועים – אם משהו לא עובד אבל המחשב לא קרס, הולכים ללוגים לראות מה השתבש בדרך. ברמת האפליקציה (core dump) מדובר כבר בקריסה של תוכנה אחת, מערכת ההפעלה מצלמת את הזיכרון שהיה שייך לאותה תוכנה ושומרת לקובץ, כדי שהמפתח יוכל לראות מה היה בתוך המשתנים רגע לפני הפיצוץ. ברמת המערכת (crash dump) זהו המקרה החמור ביותר – כל מערכת ההפעלה קרסה, כיוון שהכל נפל המערכת שומרת צילום של הקרנל כדי שיהיה אפשר להבין איזה דרייבר גרם לכל המחשב להיעצר.

Core dump file – תאונה של תוכנה ספציפית שנכשלה וקרסה, המחשב עצמו עדיין עובד טוב. מערכת ההפעלה מבצעת "צילום מצב" של זיכרון RAM שהיה שייך לאותה תוכנה בדיוק ברגע המוות שלה. המפתח מקבל קובץ שנקרא core – הוא יכול להטעין את הקובץ הזה לתוך דיבאגר, לראות מה בדיוק היה הערך של המשתנים, היכן הקוד עצר ומה גרם לו לקרוס.

Crash dump file - תאונה של כל המערכת. המצב החמור ביותר. כל המחשב קפא. מאחר וכל מערכת ההפעלה קרסה – היא לא יכולה לכתוב לעצמה לוג רגיל. לכן, בעת הקריסה היא מנסה לשמור עותק של מצב הקרנל לדיסק הקשיח. המהנדסים שחוקרים את הקריסה משתמשים בcrash dump הזה כדי להבין מה גרם למערכת לאבד יציבות.

תהליכים – Process

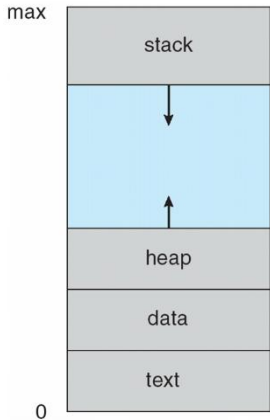
בעבר – מערכות הפעלה אפשרו להריץ תהליך אחד (תוכנית אחת) בו זמנית. כיום: ניתן להריץ כמה בו זמנית.

הערה: אנחנו נתייחס לbatch systems וprocessi – בעוד בפועל – jobs מתייחס לbatch systems. time-shared מסוג (מציג גם את הדרך, לא רק תוצאה סופית).

- איזה תהליכים רצים במערכת? תהליכים של מערכת ההפעלה, ותהליכים של המשתמש שמכילים user code.
- **Program**: מדובר ביישום פאסיבית (היא לא משתנה) ששוכנת על הדיסק, בעוד **Process**: ישות אקטיבית אשר משנה באופן רצוף את מצבה.

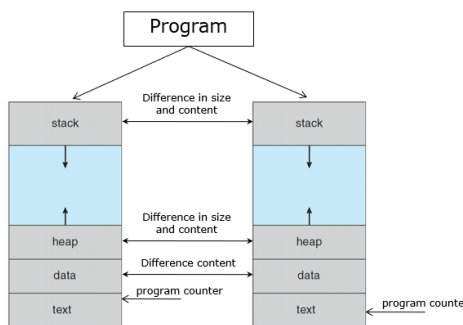
סיכום מערכות הפעלה | © גיא יער-און

- הרצת התהליך מתבצעת בצורה סדרתית – זהו עקרון חשוב שכן בזכותו אנחנו תמיד יודעים איפה הפקודה הבאה שלנו.
- התהליך יושב ב"מרחב זיכרון וירטואלי" – הוא יושב בצורה לוגית מכתובת 0 לוגית (הכוונה היא לכתובת מדומה) עד לכתובת max לוגית. שוב: אלו לא הכתובות האמיתיות, אך מה שרלוונטי לנו הוא המבנה שנשמר בזיכרון ביחד.



1. Text נשמר הקוד.
2. Data נשמרים כל המשתנים הגלובליים – המשתנים שרצים עם התהליך לכל אורך חייו.
3. Heap ישנן ההקצאות הדינמיות שעשינו.
4. Stack נשמרים משתנים לוקליים (כל פעם שקוראים לפונקציה, שיחיו במשך חיי הפונקציה, הם נוצרים במחסנית).

על בסיס אותה תוכנית, ניתן להריץ שני תהליכים או יותר במקביל שלכל אחד יש execution sequence משלו. למשל: כאשר פותחים כמה וכמה קבצי word, לכל אחד מהם יש sequence שונה. משמאל – דוגמה:



נבחין:

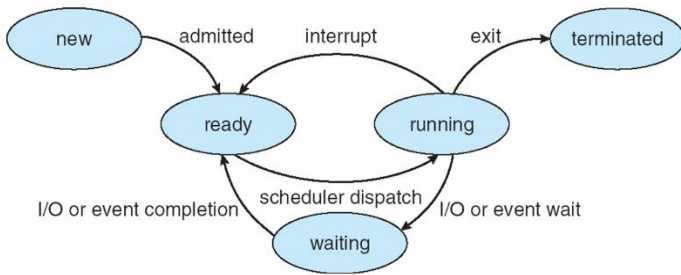
- Text יהיה זהה בשני הsequence אך program counter יהיה שונה שכן יתכן שאנחנו נמצאים במקומות שונים בקוד.
- data יהיה זהה בגודלו, אך שונה בתוכנו (אותם משתנים, אך יתכן שערכיהם שונים).
- heap, stack יהיו שונים זה מזה. (לא בהכרח, אך לרוב).

כל Process במהלך ריצתו משנה את המצב שלו (state). ישנם כמה מצבים:

- New: מדובר בתהליך שנמצא בתהליך יצירה, תהליך שעשינו לו loading. הוא עדיין "תינוק" ללא תעודת זהות, הוא רק מיוצר בשלב זה ולא ניתן להרצה.
- Running: כבר אפשר להריץ הוראות, אנחנו על המעבד.
- Waiting: מדובר במצב של המתנה, ממתינים לאירוע כלשהו שיקרה. למשל – IO של המשתמש שמעכב אותו.
- Ready: התהליך יכול להמשיך ולרוץ – אך אין לו כרגע את המעבד, לכן הוא ממתין.
- Terminated: התהליך קרס / הסתיים, וכעת מערכת ההפעלה צריכה לאסוף את המשאבים אחריו ולנקות אותו.

שאלה. כמה תהליכים יכולים להיות במערכת בכל רגע נתון בכל state?

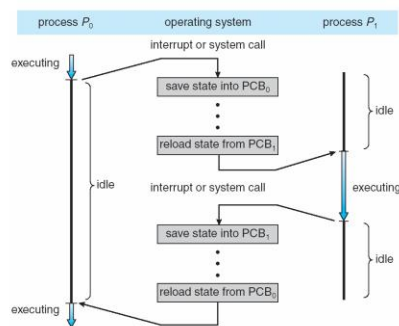
תשובה. לא ניתן לדעת. הרי, בשביל שיהיו לנו כמה תהליכים בתוך new אנחנו זקוקים למס' המעבדים. לכן, הערך הזה קטן שווה ממס' המעבדים ואם נקצין: נאמר שזה 1 ומטה, שכן לא נרצה שיהיו לנו כמה תהליכים שנוצרים בזמנית. בדומה עבור Terminated ועבור running. כמה יהיה לנו waiting ובready? קטן שווה ממס' התהליכים במערכת.



- מתחילים בnew, זה שלב ההיווצרות. בדומה: רק פעם אחת אנחנו terminated.
- משם, אנחנו עוברים אל מצב ready: תהליך זה נקרא **admitted** – נבחין שתהליך לא יכול לעבור ישירות אל running או waiting: שכן בשביל שיוכלו לבחור אותי בהרצה למצב running ויתנו לי את המעבד אנחנו צריכים להכנס למצב ready.
- ממצב ready ניתן לעבור למצב running אם scheduler בחר בי להכנס (מיד נרחיב עליו). אנחנו למעשה יושבים ומחכים.
- ממצב running ניתן להגיע למצב terminated אם ביצענו exit, וניתן להגיע למצב waiting אם ביצענו IO למשל, וגם: ניתן לעבור למצב של ready – אם אנחנו למשל בtime shared והscheduler החליט שהיה לנו מספיק זמן – ואם למשל נכנס קוד של מערכת ההפעלה.
- ממצב של waiting ניתן להגיע אך ורק למצב ready: שכן חיכינו ועכשיו אנחנו מוכנים ומחכים לscheduler.

process state
process number
program counter
registers
memory limits
list of open files
...

Process control Block (PCB): על מנת לתחזק את המעברים בין המצבים של התהליך, אנחנו שומרים מבנה נתונים בשם PCB – שם נשמור אינפורמציה כמו: מצב התהליך, מס' התהליך, הPCB (המקום בו אנחנו נמצאים בקוד), תוכן הרגיסטרים וכו'.

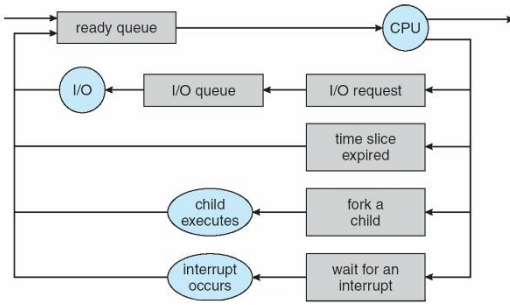


כיצד עוברים מתהליך לתהליך?

Process Scheduling Queues: המטרה שלנו היא שבכל עת יהיה תהליך שיכול לרוץ על המעבד (על מנת למקסם את ניצולת המעבד), לכן נשתמש בprocess scheduler: תורים בהם נעשה שימוש:

- Job queue: סט של כל התהליכים שבמערכת (לתור זה מגיעים כל התהליכים במערכת)
- Ready queue: סט של כל התהליכים אשר: 1. נמצאים בזכרון. 2. ממתינים להרצה.
- Device queue: סט של כל התהליכים שממתינים לIO.

במהלך הרצתו, התהליך עובר בין התורים השונים.



לדוגמה: תהליך נוצר ונכנס לתור ready. הוא יושב ומחכה – עד שהגיע תורו. הוא מבקש IO באמצעות system calls ונכנס לתור של IO, כשהIO שלו הסתיים הוא חוזר לתור של ready. מה עוד קורה לו בהמשך? Time slice expired: כשאנחנו חולקים את הזמן במעבד, נגמר הזמן שהוקצה לו, והוא חוזר שוב למצב של ready. יתכן ובהמשך – התהליך ייצור תהליך בן, ונרצה לבצע wait() ולעצור עד שהבן שלנו יסתיים, כעת נכנס למצב waiting. אפשרות נוספת – שאנחנו מחכים לinterrupt ולכן נמצאים בsleep.

Scheduler: ישנם שני schedulers עם מטרות שונות:

- **Long term scheduler:** בוחר התהליכים שיהיו בready queue – בוחר מי יהיו בזיכרון. ה"יותר חכם", כיוון שאנחנו מפעילים אותו בתדירות יותר נמוכה ביחס לshort-term, הוא "חושב קדימה" וההחלטות שלו מאוד משמעותיות והרות גורל. כל החלטה שלו – מעבירה תהליך מדיסק לזיכרון, לכן מופעל בתדירות די נמוכה. הוא שולט על מס' התהליכים שנמצאים בזיכרון בכל רגע (מושג זה נקרא **degree of multiprogramming** – דודי אמר לזכור מושג זה למבחן).
- **Short-Term scheduler:** בוחר את התהליך שירוצו על המעבד (מבין אלו שנמצאים בready queue). מופעל כל כמה מילישניות, חייב להיות מאוד מהיר אחרת יהיה לנו overhead (תקורה – לא נרוויח ממנו כלום – אך יתפוס לנו משאבים בזמן שלא הרצנו כלום). לכן ישתמש בשיטות די פשוטות להחלטתו כי הוא חייב לקבל החלטות בצורה די מהירה.
- **Overhead:** תקורה. מתייחס לכל משאב של המחשב (זמן מעבד, זיכרון וכו') שנצטרך לא כדי לבצע את המשימה עצמה אלא כדי לנהל אותה.

תהליכים מאופיינים כאחד מהשניים הבאים לרוב:

- **IO bound:** תהליך שרוב הזמן שלו הוא מבלה את זמנו בפעולות של IO – רוב הקוד שלו מריץ הוראות של IO. למשל: גלישה ברשת (הרי מדובר בהרבה הדפסה/קבלה מהמסך וכו'. החישובים קצרים) או העתקה של קבצים גדולים.
- **CPU bound:** תהליכים שרוב הזמן שלהם מריצים הוראות חישוביות – למשל חישוב של pi מבצע הרבה חישובים – ואין שם יותר מדי הוראות IO.

חשוב: Long term scheduler צריך לדאוג שבכל רגע נתון יהיה בזיכרון שילוב של תהליכי IO, CPU, IO. מדוע? כי אם יהיה לנו בזיכרון הרבה תהליכי IO התורים של IO יתפוצצו – והמעבד לא יהיה בשימוש, והנצילות תרד. מצד שני, אם יהיה לנו בזיכרון הרבה תהליכי CPU תור ready יתפוצץ ולא יהיה נצילות ושימוש בIO.

Context Switch

הגדרה: כאשר המעבד עובר לטפל בתהליך אחר, המערכת חייבת לשמור את state של התהליך המסוים ו"לטעון" את state השמור של התהליך החדש – תהליך זה נקרא context switch.

- context switch עצמו שמור בPCB (מבנה נתונים לתחזוק המצב הנוכחי).
- זמן ביצוע הcontext switch הוא overhead שכן המערכת איננה מבצעת עבודה שמשמשת את המשתמש בזמן זה – אלא משמשת את עצמה בהמשך. משך זמן הביצוע של context switch מושפע מהתמיכה בחומרה.

סיכום מערכות הפעלה | © גיא יער-און

שאלה לדוגמה (ממבחן). תהי מערכת מחשב עם N תהליכים שחולקים CPU באמצעות מנגנון בשם RR (כל תהליך מקבל אותו זמן CPU). נניח כי כל context switch אורך S מילישניות וכל time quantum אורך Q מילישניות. מהו הערך המקסימלי של Q כך שהזמן המקסימלי בין זוג instructions של אותו תהליך הוא T מילישניות?

פתרון: למעשה אנחנו נשאלים כמה זמן (חסום ב- T) עובר בין זמן של תהליך x לפעם הבאה שירוצץ במעבד. נבחין כי בין התהליכים ישנם:

- N פעמים context switch: בזמן כולל של SN
- $N-1$ פעמים שיתבצעו תהליכים אחרים: בזמן כולל של $(N-1)Q$

לכן נדרוש: $SN + (N-1)Q < T$ ונקבל: $Q < \frac{T-NS}{N-1}$.

Process creation: כיצד יוצרים תהליך? משהו צריך להריץ פקודה בשם create על מנת להריץ פרויקט. לכן – לכל תהליך יהיה תהליך סבא שידאג ליצור אותו. למעשה אנחנו נקבל מעין עץ תהליכים, כאשר ראש העץ הוא scheduler שמנהל את התהליכים (אותו יקצה boot program). כל תהליך צריך לקבל משאבים/זיכרון וכו'. מי ייתן לו אותם? ישנם כמה גישות:

- האבא נותן לתהליך הבן את המשאבים שלו – שיחלקו אותם יחדיו.
- האבא מקצה חלק מהמשאבים שלו שיוקצו עבור הבן.
- האבא והבן לא יחלקו משאבים – לבן יהיה משאבים משלו (זה מסוכן לנו: שכן ייתכן שיהיה תהליך שייצור בכוונה הרבה תהליכים בנים על מנת לקבל עוד משאבים עבורו).

הרצאה 4

כפי שאמרנו בסיום ההרצאה הקודמת, תהליך נוצר ע"י מישור אחר. בלינוקס – זה ע"י פעולת `fork()` והBoot program יוצר את התהליך הראשון. ומה לגבי ההרצאה עצמה? תהליך האב יכול לרוץ במקביל לתהליך הבן – בזכות multiprocessing. אך לא תמיד הם ירצו לרוץ במקביל: למשל, אם נרצה ליצור תהליך בן שישאל את המשתמש כל מיני שאלות, עד שהמשתמש לא יענה – נרצה שבזמן הזה תהליך האב יחכה. בשביל זה ישנה פעולת `wait()`.

- מרחב הזיכרון (memory space) של הבן תהיה זהה לתמונת הזיכרון של האב בעת שנוצר תהליך הבן, אך הם יהיו נפרדים.
- ניתן גם לטעון תוכנית חדשה לחלוטין לתוך תהליך הבן, באמצעות פעולה `exec()`.
- כיצד מבדילים בין האב לבן? באמצעות ערך החזרה של פעולת `fork()`. ערך זה הוא `pid`. אם `pid < 0` אזי תהליך יצירת הבן נכשל. אם `pid = 0` אזי מדובר בתהליך הבן. אחרת, `pid > 0` ומדובר בתהליך האב.

Process termination

בסיום פעולתו של כל תהליך הוא נדרש להריץ את הפקודה `exit()` – מדובר בsystem call. במצב זה מערכת ההפעלה:

- מוחקת אותו מרשימת התהליכים הקיימים (job queue)
- כל המשאבים שהוקצו לתהליך יוחזרו למערכת ההפעלה, והיא תרשום שהמשאבים הללו אינם מוקצים לאף תהליך.
- אם התהליך הנ"ל היה תהליך בן, אזי אם הייתה בקשת `wait()` מהאב יוחזר לו כי סיימו.

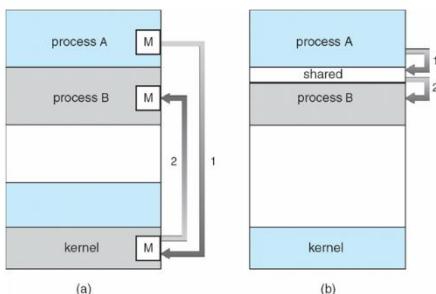
כמו כן, תהליך אב יכול לגרום להפסקת פעולת תהליך הבן באמצעות `abort`. מתי נרצה לעשות זאת?

- הבן משתמש ביותר מדי משאבים ממה שתכננו שהוא השתמש.
- המשימה שבשבילה נוצר הבן – כבר לא רלוונטית עוד. למשל: אנחנו סרבר, ויצרנו תהליך עבור בקשה מסוימת שהפכה ללא רלוונטית עוד. אז נרצה לבטל את התהליך הנ"ל.

Cascading termination: יש מערכות הפעלה שבעת שאבא ביצע `exit()` הורגות את כל העץ תחתיו – כל הבנים.

Interprocess Communication

תהליכים צריכים לשותף ולמצוא דרך לחלוק מידע בניהם. בין אם מדובר בתהליכים שרצים באותה מערכת או במערכות שונות. התמיכה בשיתוף המידע היא באמצעות IPC - interprocess communication. ישנם שני מודלים של IPC:



- **Shared memory**: מוקצה אזור בזיכרון הפיזי ששני התהליכים (או יותר) יכולים לגשת אליו. התהליכים קוראים וכותבים לאזור הזה כאילו הוא חלק ממרחב הכתובות הפרטי שלהם. זהו המנגנון המהיר ביותר כי הוא לא דורש התערבות של הליבה (Kernel) לאחר ההקמה, אך הוא דורש מהמתכנת לנהל סנכרון (כדי למנוע מצב ששני תהליכים כותבים בו-זמנית). בתמונה: מימין.
- **Message passing**: מערכת ההפעלה בונה מנגנון שמעביר "הודעות" בין התהליכים. היתרון שלו הוא שאין שום צורך לנהל דברים – זה תחום אחריותה של מערכת ההפעלה. החיסרון שלו הוא שהוא איטי יותר כי בגלל המעברים לkernel בכל הודעה הוא נדרש לעצור ולחכות. בתמונה: משמאל.

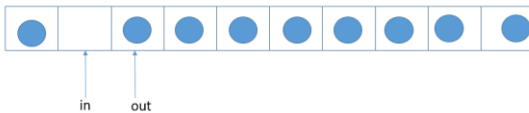
Producer-Consumer Problem: פרידגמת היצרן-צרכן. מושג שנשתמש בו הרבה במערכות הפעלה. מודל קלאסי לשיתוף פעולה בין תהליכים, שבו תהליך אחד מייצר מידע ותהליך אחר משתמש בו. דוגמה נפוצה היא קומפיילר שמייצר קוד Assembly, אשר נצרך לאחר

מכאן על ידי ה-Assembler. כאשר משתמשים בפתרון של Shared Memory (זיכרון משותף) למימוש הפרדיגמה הזו, נהוג לדבר על שני סוגי חוצצים (Buffers) המשמשים להעברת המידע:

1. **Unbounded-Buffer (חוצץ ללא הגבלה):** במודל זה, אין הגבלה מעשית על גודל החוצץ (המקום אליו הם כותבים על מנת לתקשר בניהם) ולכן מבחינתם הוא אינסופי.

2. **Bounded-Buffer (חוצץ מוגבל):** מודל זה מניח שיש לחוצץ גודל קבוע ומוגדר מראש.

כיצד מממשים זאת? באמצעות שני מצביעים - in, out. במעין צורה ציקלית מעגלית. in הוא פוינטר שמצביע על התא הריק הבא שבו היצרן יכניס מידע חדש, out הוא התא שבו נמצא המידע הבא שהצרכן צריך לקרוא. ישנם שני מקרים:



- $in == out$: הצרכן יודע שאין לו מה לקרוא והוא צריך להמתין.
- $Out = (Buffer\ size) \% (in + 1)$: היצרן מתקדם ומגיע למצב שבו התא הבא שלו הוא out, המחסן מלא ולכן הוא צריך לחכות שהיצרן יוציא לפחות "תא אחד" ויתפנה מקום.

Message passing

- יוצרים משתנים משותפים ששני התהליכים מחזיקים.
- IFC Facility: מספק שתי פעולות, $send(message)$ ו- $receive(message)$. אם זוג תהליכים רוצים לתקשר בניהם עליהם ליצור communication link בניהם ולהעביר הודעות באמצעות הפקודות. מימוש ה-communication link הוא באמצעות חומרה.
- את תהליך ההודעות ניתן לחלק לשני אפשרויות: direct – בצורה ישירה. indirect – התהליכים לא שולחים הודעות אחד לשני אלא משתמשים במתווך שנקרא Mailbox (תיבת דואר). כלומר – במקום שתהליך א' יצטרך להכיר את השם של תהליך ב' – שניהם צריכים בסך הכל להכיר את תיבת הדואר. מערכת ההפעלה צריכה לספק פונקציות כדי לנהל את התיבות הללו כמו יצירה של תיבה, שליחה וקבלת הודעות והריסת תיבה.
- השוואה בין תקשורת ישירה לעקיפה:

Interprocess Communication –
Message Passing (cont.)

	Indirect Communication	Direct Communication
method	Primitives are defined as: $send(A, message)$ – send a message to mailbox A $receive(A, message)$ – receive a message from mailbox A	Processes must name each other explicitly: $send(P, message)$ – send a message to process P $receive(Q, message)$ – receive a message from process Q
Link established	only if processes share a common mailbox	established automatically
Link vs. process	link may be associated with many processes	link is associated with exactly one pair of communicating processes
Process pair vs. link	Each pair of processes may share several communication links	Between each pair there exists exactly one link
direction	Link may be unidirectional or bi-directional	The link may be unidirectional, but is usually bi-directional

Synchronization

המנגנון של message passing יכול להיות מאופיין כblocking or non-blocking: כלומר, האם השולח חוסם / לא חוסם את עצמו עד שההודעה מתקבלת. תצורת blocking נחשבת סינכרונית.

Blocking send: השולח מבצע block לעצמו עד שההודעה מתקבלת בצד השני.

Blocking receive: המקבל מבצע block לעצמו עד שההודעה זמינה לקבלה.

Non-blocking send: השולח שולח את ההודעה וממשיך בפעולתו – ללא חשיבות אם הצד השני קיבל אותה או שלא.

Non-blocking receive: המקבל מקבל את ההודעה גם אם היא invalid, הוא לא יושב ומחכה להודעה.

Buffering: המקום בו אנחנו משאירים את ההודעות ומחכים שיגיעו לצד השני. כמות המקום (ההודעות) שניתן לאחסן בו משתנה ויש כמה אפשרויות -

סיכום מערכות הפעלה | © גיא יער-און

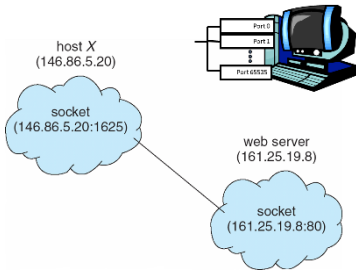
Zero capacity: לבאפר אין מקום להודעות שישבו ויחכו – בעת שליחת הודעה הצד השני צריך למשוך אותה, השולח חייב לבצע block לעצמו עד שהצד השני יקבל את ההודעה, הוא מחכה למקבל, המקבל מושך את ההודעה.

Bounded capacity: יש כמות סופית של הודעות שיכולות לשבת ולחכות, וכשהיה מלא אי אפשר לשלוח יותר הודעות.

Unbounded capacity: המקום בבאפר לא מוגבל – תמיד אפשר לשלוח הודעות. המצב הכי טוב לכאורה – מצריך הכי פחות ניהול.

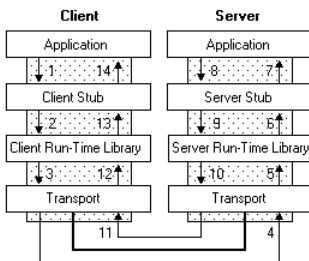
Socket – לא למבחן (לפי דודי)

עד כה, דיברנו על תקשורת בין תהליכים שהיו באותה מערכת הפעלה (אותו מחשב). ומה עם תקשורת בין תהליכים שנמצאים במחשבים שונים?



- לכל מכונה ברשת יש כתובת IP: X.X.X.X, שבאמצעותה ניתן לזהות את המחשב עצמו.
- זה לא מספיק לזהות את המחשב עצמו – לשם כך יש **port**: ערך מספרי שמייצג את האפליקציה בתוך המחשב. לצורך המחשה: יש בניין דירות, שמיוצג ע"י אותה כתובת, אך הport הוא מספרי הדירה.
- בעת שנשויך את port לIP ניצור socket. בעת ששני התהליכים מעוניינים לדבר – הם יצרו connection.
- ישנם שני פרוטוקולים שתפקידם להעביר הודעות בין אפליקציות שרצות על מחשבים שונים ברשת.
 1. UDP: "שולחים", לא אכפת לי מה יקרה" – טוב לשידורים חיים וכו'. לא מובטח שההודעה תשלח.
 2. TCP: אמין, מובטח שיגיע לנו הודעה.
- במהלך השנים התפתחו מוסכמות לport מסוימים. למשל הוחלט כי למטרה מסוימת יוקצה port מסוים. ואז בשביל לבצע את אותה מטרה כל מה שצריך לדעת הוא את הIP. למשל PORT 80:HTTP.

RPC – Remote procedure calls: לא למבחן (לפי דודי)



- מנגנון ה-RPC (Remote Procedure Call) נועד לגשר על הפער שנוצר כשעובדים עם Sockets – שבהם צריך לנהל דינאית את פתיחת הקשר, אריזת הנתונים (Serialization) ופענוח ההודעות. ה-RPC גורם לקריאה לפונקציה שנמצאת במחשב מרוחק להיראות ולקוד שלך בדיוק כמו קריאה לפונקציה מקומית. הוא "מסתיר" את כל הבלאגן של הרשת (פתיחת Sockets, אריזת נתונים ושליחה) מאחורי קוד מתווך שנקרא Stub.
- התוכנה שלנו קוראת לפונקציה רגילה, קוד מקומי דרך תיווך (stub) אורז את הפרמטרים, תופס את הקריאה ושולח אותם לשרת. בצד השני: server stub פורק את החבילה, מפעיל את הפונקציה האמיתית בשרת ומחזיר את התוצאה באותה דרך.

RMI (Remote Method Invocation) הוא הגרסה של Java לעולם ה-RPC, והוא מאפשר לאובייקט שרץ ב-JVM (מכונה וירטואלית של ג'אווה) אחת להפעיל מתודות של אובייקט שנמצא ב-JVM אחרת, אפילו על מחשב שונה.

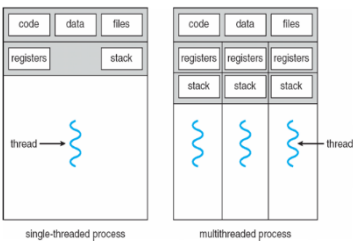
- **Busy waiting:** לרוץ בלולאה אינסופית ולבדוק תנאי – מבלי להמשיך. כאשר אנחנו במערכת עם מעבד יחיד – ביצוע busy waiting הוא חסר הגיון ואף מזיק. במערכת בעלת מעבד יחיד רק מעבד אחד יכול לרוץ בכל רגע נתון. אם תהליך א' מבצע busy waiting הוא תופס את המעבד ב-100%. הבעיה היא – תהליך א' מחכה לו יכול להשתנות רק ע"י תהליך ב' אבל כל עוד תהליך א' תקוע על המעבד בבדיקה שלו, תהליך ב' לא יכול לרוץ ולבצע את הפעולה שתשחרר את החסימה. המערכת תצטרך לחכות עד שיגמר זמן הריצה של א', ותבצע קונטקסט סוויץ' כדי שתהליך ב' יוכל לעבוד. בזמן הזה המעבד שרף זמן יקר.

Threads

עד כה הנחנו שכל תהליך מריץ תוכנית כך שבכל רגע נתון מתבצעת פקודה אחת בלבד של התהליך, אך ברוב האפליקציות המודרניות אנחנו רואים שימוש ב-multithreading (שימוש בכמה חוטים בו זמנית, וכך מתבצעות כמה פקודות בכל רגע נתון).

לדוגמה: קליינט שולח בקשה לסרבר שמאזין לו, ובעת שמגיעה הבקשה הסרבר מייצר thread חדש שייתן את השירות לאותו קליינט וישוב להאזין לport שהיה בו. נשאל את השאלה: למה שלא ייצור תהליך חדש? מיד נבין זאת.

- threads רצים במסגרת התהליך והאפליקציה הבודדת.
- הרבה מהמשימות של אותו תהליך לרוב יכולים להיות ממומשים ע"י threads שונים. למשל: בעת שימוש בword נרצה שיהיה thread נפרד לטיפול בתצוגה, אחר לטקסט עצמו, תגובה להקשה על מקשים, הבאת מידע/שמירת מידע, וכו'.
- kernel של מערכת ההפעלה ממומש גם כן כמולטי-טרד.



הthread מורכב מ: thread id, program counter, register set, stack. עם זאת, הthread חולק עם שאר threads של אותו תהליך את:

- Code section
- Data section
- משאבים אחרים של מערכת ההפעלה כמו קבצים וכו'.

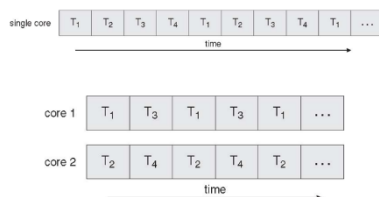
יתרונות של threads:

אז למה שלא ליצור עוד תהליך במקום תרד?

- הרבה יותר כלכלי ויעיל לבצע context switch לthread ולא לprocess שכן הם חולקים את משאבי התהליך. מדובר על פי 5 זמן בערך. כמו כן, הרבה יותר יעיל (פי 30) ליצור thread מאשר תהליך.
- מאפשר לתוכנית להמשיך לרוץ גם כאשר חלקים ממנה הם block (למשל יצאנו לIO) או שמבצע פעולה ארוכה (שהרי תרד רץ על חלקים שונים בתוכנית).
- threads חולקים את אותו הזיכרון והמשאבים של התהליך.
- אותו תהליך יכול לרוץ על מס' מעבדים במקביל באמצעות שימוש בthreads (כל אחד על מעבד שונה).

Multicore Programming

מערכות שהן multicore מביאות איתן אתגרים חדשים:



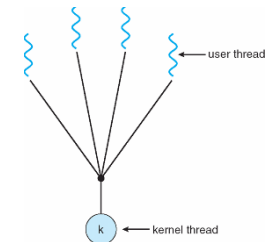
- קשה למצוא תחומים/פעילויות שניתן לחלקם בצורה נקיה למשימות שיכולות להתקדם במקביל.
- חלוקת data ויצירת מבני נתונים מתאימים לתמיכה בהרצה במקביל.
- קושי משמעותי בtesting וdebugging. שכן הקוד הפך מדטרמיניסטי – לקוד multithread שכבר לא דטרמיניסטי – יתכן שהבאג קרה כי תרד אחד התקדם מהר מדי מתרד אחר: וכשנריץ שוב יתכן שזה כבר לא יקרה.

כיצד זה נראה בפועל? משמאל בתמונה. יהיו כמה ליבות (cores) שיעשו time-sharing. כשיש single core זו לא באמת ריצה במקביל לעומת המקרה שלמטה – שם זה ממש ריצה במקביל.

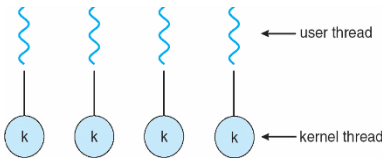
סיכום מערכות הפעלה | © גיא יער-און

Kernel threads: התרדים שמכירה מערכת ההפעלה. כלומר, threads שמכיר scheduler. כשהמתזמן מחליט מי ירוץ עכשיו, הוא בוחר ישירות קרנל תרד.

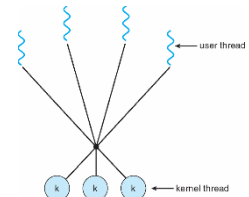
User threads: התרדים שמנוהלים באמצעות ספרייה (כמו pthread) בשם threading library ומשובצים ומנוהלים ע"י הספרייה עצמה. היא זו שמחליטה איזו תרדים כרגע רצים. מערכת ההפעלה לא יודעת שהם קיימים – היא רואה רק תהליך אחד גדול. הספרייה היא זו שמנהלת "לוח זמנים פנימי" ומחליטה מי מבניהם ירוץ עכשיו בתוך הזמן שהתהליך קיבל. היתרון הוא שהם מהירים מאוד, החיסרון הוא שאם תרד אחד מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך! כי היא לא מודעת באמת לכך שיש שם עוד תרדים בפנים. על מנת שuser threads באמת ירוצו על המעבד – הם חייבים "להתלבש" על קרנל תרד. ישנם כמה מודלים של מיפוי - המיפוי הוא הדרך שבה אנחנו מחברים בין העולם הווירטואלי של התוכנה לבין העולם הפיזי של המעבד. ישנם מס' מודלים:



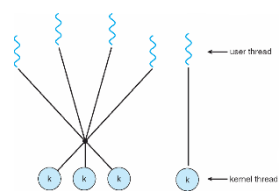
1. **Many-to-one:** מספר user-threads של התהליך ממופים כולם לkernel thread אחד. מה היתרון של הדבר הזה? זה מאוד קל, וכל הניהול של threads ינוהל בספרייה עצמה (זה יעיל מבחינת משאבים ותקורה). החיסרון הגדול באמת זה שאם אחד התרדים מבצע פעולה חוסמת מערכת ההפעלה חוסמת את כל התהליך, ואז גם אם יש תרדים אחרים שיכולים להשתמש במעבד כרגע הם לא יכולים לרוץ כרגע כי המערכת חסומה. כמו כן: אין כאן באמת הרצה במקביל, גם אם למעבד יש כמה ליבות.



2. **One-to-one:** לכל user-thread ישובך kernel-thread. נקבל מקביליות אמיתית בהרצה, אין בעיות של חסימה וכו'. אבל: כמובן שיש חסרונות – זה יקר מאוד! ויש גם overhead שגורם לניהול של הקרנל תרדס ומביא הרבה מגבלות. יהיו הרבה תהליכים שלא יוכלו לרוץ בפועל כי נגיע למגבלה על כמות התרדים המקסימלית.



3. **Many-to-many:** במקרה זה ישנם m kernel threads וישנם n user threads. ומתקיים $m \leq n$. גם כאן פתרנו את החסימה.



4. **Two-level model:** במקרה זה, לחלק מהuser threads שחשובים לנו אנחנו נעניק להם kernel thread ייחודי להם. ולשאר threads אנחנו נעבוד במודל many-to-many.

Java Threads

גם בjava ניתן ליצור threads. ניתן להרחיב את המחלקה Thread, או לממש ממשק בשם Runnable. המתודה start() מקצה זכרות ומאתחלת תרד חדש במחשב.

Threading Issues

יש המון בעיות שנוצרות מתרדים. נדון בכמה מהם:

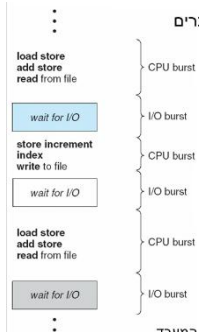
- נניח וביצענו fork(). הוא צריך לשכפל רק את thread הקורא (זה שיצר את התהליך החדש) או את כל התהליך? העיצוב הנכון הוא בהתאם למטרת היצירה. אם מיד לאחר fork נבצע exec הרי כדאי לשכפל רק את התרד הקורא (נחסוך במשאבים ובזמן, הרי במילא נדרוס הכל). אחרת, נשכפל את הכל.
- נרצה לפעמים לסיים פעולתו של thread לפני שהוא סיים את פעילותו. למשל – אם מס' תרדים מחפשים בבסיס נתונים מידע ואחד מהם מצא את המידע. נרצה להפסיק את האחרים. ישנן שתי גישות עיקריות לעצירה:
 1. גישה אסינכרונית: "להרוג" את התרד. לא בהכרח נצליח לשחרר את המשאבים כמו שצריך.
 2. Deferred cancellation: מאפשר לתרד המטרה לבדוק באופן תקופתי האם עליו להתבטל – באמצעות בדיקה של דגל flag.

סיכום מערכות הפעלה | © גיא יער-און

- סינגלים מועברים לתהליך ונעשה שימוש בהם על מנת להעביר לתהליכים מידע לגבי אירועים רלוונטיים (למשל: חלוקה באפס או גישה לכתובת לא חוקית). מה הבעיה? לאיזה תרד נעביר את הסינגל הנ"ל? אפשר להעביר לכל התרדים של התהליך (ואז תקורה), אפשר להעביר לסט של תרדים שנקבעו מראש, אפשר להשתמש בתרד ייעודי שאליו יועברו כל הסינגלים, ואפשר להעביר לתרד שהסינגל הכי רלוונטי אליו (אך זה מחייב הבנה של לוגיקה).
- Threads pool: הרעיון הוא יצירה של מס' תרדים מראש שנמצאים במאגר ומחכים למשימות. יש לכך יתרונות כמובן:
 1. מהירות – מהיר יותר לטפל בבקשה לרוב באמצעות תרד קיים מאשר ליצור תרד חדש מאפס
 2. מאפשר להגביל את מס' התרדים באפליקציה לגודל המאגר הקבוע מראש. הסיכון ללא המאגר הוא שכמות בלתי מוגבלת של תרדים עלולה לכלות את כל משאבי המערכת.

הרצאה 5

בשיעורים הקודמים אמרנו שישנם שני סוגים של schedulers. בפרט, short term scheduler קובע מי יהיה בכל רגע נתון על המעבד (וה Long קובע מי יכנס לready queue). נציג את הרעיון של CPU Scheduling שהוא הבסיס למultiprogramming.



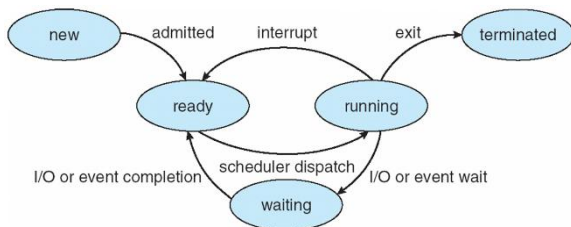
מחזור החיים של כל תהליך מורכב מרצף של מעברים בין הרצת דברים על המעבד, לבין ביצוע פעולות של IO, כאשר burst הוא רצף. כלומר הוא מחליף בין CPU burst to IO burst, ככה לסירוגין.

- תמיד מתחיל CPU burst ותמיד מסתיים בו. מדוע? כי הפקודה הראשונה והאחרונה שצריכים להריץ – תמיד רצים על גבי המעבד (מסתיים ב()exit שהרי מתבצעת על גבי המעבד).

בהינתן תהליך, ניתן לבנות היסטוגרמה של השכיחות היחסית של כל burst. ניתן לראות מכל היסטוגרמה כזו שבדרך כלל יש הרבה burst קצרים – ומעט מאוד burst ארוכים (אלו למען חישובים ארוכים לרוב, שלא קורים יותר מדי).

בהינתן ההבנה הזו, נבחין כי המעבד מתפנה יחסית די מהר – כל כמה מילישניות. אנחנו לא רוצים שהמעבד יהיה פנוי ולא יעשה כלום – זה יוריד את נצילות המעבד. לכן, נרצה בזמן זה (שהמעבד מתפנה) להכניס תהליך אחר לקבל זמן מעבד. כלומר – בעת שתהליך מסוים עבר לwaiting הוא יפנה את מקומו לתהליך אחר. מי שיחליט על התהליך הבא שיכנס – יהיה Short Term Scheduler.

מבין כל התהליכים שנמצאים בready queue המעבד יבחר מי כעת יקבל זמן מעבד.



מתי יש להפעיל את הCPU Scheduler?

- כאשר תהליך עובר ממצב running-> waiting
- כאשר תהליך עובר ממצב running->ready
- כאשר תהליך עובר ממצב waiting->ready (התהליך חיכה למשל לקריאה מתוך קובץ, קיבל interrupt, וכעת הוא מוכן. במצב זה נרצה לזמן לבדוק אם ניתן להריץ אותו).
- כאשר תהליך מסיים את חייו.

המצב הראשון והרביעי – שונים מהשני והשלישי. מדוע?

- במצב הראשון והרביעי – בעת שמתפנה המעבד, במצבים אלו נקרא לזה: nonpreemptive – זה אומר שהscheduler לא עוצר תהליכים באמצע על גבי המעבד. כלומר: רק כאשר התהליך עצמו מחליט לעצור – scheduler יעצור אבל הוא לא יעצור תהליך באמצע. במצב זה – המעבד מתפנה כי לתהליך אין ברירה אלא לעצור (כי נגמר התהליך, או שיוצאים למצב המתנה!).
- במצבים השני והשלישי, מתבצע preemptive – המעבד יכול לעצור תהליך באמצע למרות שהוא יכול להמשיך לרוץ.

Dispatcher: הוא זה שמבצע את החלטות scheduler. הוא מעביר את השליטה לתהליך שנבחר – מבצע context switch, עובר לuser mode, טוען את הPC של התהליך החדש וממשיך משם את הריצה. הזמן שלוקח לו לבצע את ההחלפה (לעצור את התהליך שרץ ולהפעיל במקומו תהליך אחר) נקרא Dispatch latency.

קריטריוני scheduling:

- ניצולת המעבד – אחוז הזמן בו המעבד עסוק בהרצת פקודות (ולא במנוחה או המתנה). נשאף לכמה שיותר גבוה אך זה נדיר שנגיע למאה אחוז. לעיתים מדובר במדד בעייתי שכן לא מפריד בין זמן מעבד של מערכת ההפעלה לבין זה של המשתמש. **נרצה למקסמו.**
- Throughput (תפוקה): מספר התהליכים שמסיימים ריצה בפרק זמן נתון. **נרצה למקסמו.**
- Turnaround time: הזמן הכולל להרצה של התהליך, כולל את: הזמן שהתהליך המתין עד שנכנס לזיכרון (אשר אותו הכניס לזיכרון ה Long term scheduler), הזמן שהמתין בready queue – **זה היחיד ששולט עליו ה short term scheduler: לכן הוא הכי חשוב!** זמן ההרצה על המעבד, הזמן שבילה בביצוע (או בהמתנה) לIO. **נרצה למזער.**
- Waiting time: הזמן היחיד שבאמת מושפע מהscheduling. **נרצה למזער.**

סיכום מערכות הפעלה | © גיא יער-און

- Response time: הזמן מהרגע שהתהליך ביקש את המעבד, עד לרגע הראשון שבו התהליך קיבל אותו לראשונה. (מדוע לראשונה? זה הזמן שבו יש "התקדמות" לתגובת המשתמש – זה נותן למשתמש את ההרגשה שהתהליך איתו, ולא תקוע). **נרצה למזער.**

מעבר לגישה הפשוטה של למקסם או למזער כל אחד מהמדדים – נרצה לעבוד בminmax: שיקולים של – לא שווה למזער את כולם ואחד המדדים יהיה מאוד גבוה. יתרה מזאת: נרצה למזער את השונות, ולא את הממוצע.

כעת נדון בשיטות Scheduling.

First-Served (FCFS) Scheduling

The simplest to implement!

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

השיטה הפשוטה אומרת את הדבר הבא: אסתכל על כל התהליכים שנמצאים בready queue, ואכניס ראשון את התהליך שהגיע ראשון, שני מי שהגיע שני וכו'. כלומר בהינתן תהליכים שהגיעו בסדר $P_1, P_2, P_3, \dots$ נריץ ראשונה את P_1 ואז P_2 וכו'.

ניצור את Gantt chart (תרשים זמן של התהליכים, כמו שמופיע בדוגמה משמאל):

נבחין כי אנחנו מזניחים בדוגמה משמאל את context switch בין כל החלפה של תהליכים.

נבחין ששיטה זו עובדת בnonprimitive – אין שום כוח בעולם שיוריד תהליך מהמעבד! בהכרח: בגלל שתהליך הגיע במיקום הו ל-ready queue גורר שהוא ירוץ במיקום הו. סה"כ נגדיר לבסוף:

$$\text{Average waiting time} = \frac{\text{waiting-time}(P_1) + \text{waiting-time}(P_2) + \dots + \text{waiting-time}(P_n)}{n}$$

ניצולת המעבד בשיטה זו היא 100%.

Suppose that the processes arrive in the order

P_2, P_3, P_1

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- Convoy effect short process behind long process $\rightarrow$ lower device utilization
- FCFS is nonpreemptive

נבחין שלמרות שהתהליכים כפי שנאמר בשאלה הגיעו בזמן זהה – הסדר שלהם שונה:

מדוע זה כך? ברוב המקרים אם שני תהליכים הגיעו בזמן $t=0$ המתזמן יכניס אותם לפי הסדר שבה מופיע בטבלת התהליכים שכתובה לו. ולכן יופיעו בסדר שונה. סיבה נוספת יכולה להיות קשורה למזהה התהליך – המתזמן יכניס את התהליך עם הPID הנמוך יותר.

הconvoy effect הוא ההשערה לפיה תהליך מאוד ארוך – ימשוך את כל התהליכים אחריו, ויגדיל כתוצאה מכך את Average waiting time. למשל, אם התהליכים (מהדוגמה קודם) היו מגיעים בסדר שונה היינו מקבלים:

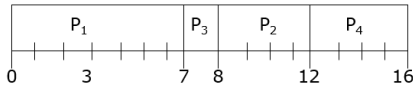
Shortest-Job-First (SJF) Scheduling

שיטה זו בוחרת בכל שלב להריץ את התהליך שהCPU-Burst שלו הוא הקטן ביותר. ישנן שתי סכמות למימוש:

- Nonpreemptive: ברגע שתהליך משובץ על המעבד, לא ניתן לעצור אותו עד שישלים את CPU burst הנוכחי שלו.
- Preemptive: אם מגיע תהליך חדש ברגע נתון עם CPU burst קצר יותר מהburst שנותר להרצה של התהליך הנוכחי, תבוצע החלפה לתהליך הקיים ובמקומו ירוץ התהליך החדש.

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (non-preemptive)



□ Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (preemptive)



□ Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

בתחילה בזמן 0 מופיע רק התהליך הראשון ולכן נכנס. הוא מתבצע כל הזמן שלו כי אנחנו ב nonpreemptive. לאחר מכן – בזמן 7 נמצאים במערכת שלושת התהליכים האחרים.

זה עם הזמן הקטן ביותר הוא תהליך 3 שמגיע תורו. לאחר מכן בהתאם – מופיעים תהליכים 2 ו 3. סה"כ הזמן הוא הממוצע של זמן ההמתנה.

החישובים לכל תהליך מתבצעים לפי הנוסחה מטה.

ואם נעבור ב preemptive נקבל:

בתחילה תהליך אחד הוא היחיד שקיים, רץ למשך 2 שניות וכעת זמן burst שלו הוא 5. בשנייה 2 מגיע תהליך 2 שנכנס כי יש לו פחות, ובשנייה 4 נכנס תהליך 3 שמבצע את כל הזמן שלו. בשנייה 5 השליטה חוזרת לתהליך 2 וכן הלאה.

כיצד מחשבים לכל תהליך את זמן הריצה שלו? לפי:

$$waiting - time = end - start - burst_time$$

מדוע? זמן הסיום פחות זמן ההגעה זה משך הזמן שהתהליך בילה במערכת. מזה נחסיר את הזמן שהוא אכן עבד. סה"כ נקבל את זמן ההמתנה במערכת.

- **טענה:** שיטה זו מבטיחה את ה minimum average waiting time לכל סט נתון של תהליכים.
- **הוכחה:** נניח שיש תהליך ארוך לפני תהליך קצר, נחליף בניהם. התהליך הארוך שהיה לפניו קדימה בגודל התהליך הקטן (לכן הזמן של הקצר שקודם חיכה לארוך – הרוויח את גודל התהליך הארוך!). כיוון שארוך < קצר, הרווח תמיד גדול מההפסד.
- הוכחה זו נכונה כאשר context switch זניח! (שכן יתכן שזמן ההחלפה גדול). וכן: זה נכון כאשר מדובר ב preemptive.

מה הבעיה העיקרית בשיטה? איננו יודעים ויכולים להעריך את משך burst הבא. (אך ניתן לשאול תמיד את המשתמש שידוע נתון זה: אבל זה לא טוב כי הוא ישקר! הרי תמיד מישו יגיד "רק שאלה קצרה – אני נכנס ועוקף את התור", לכן לא נרצה לשאול את המשתמש). שהרי, המעבד לא יכול לדעת כמה זמן ייקח התהליך הבא שירוצ עליו!

במקום זה, אפשר לבצע חיזוי על פי תצפיות עבר. זה שכיח מאוד בעולם מדעי המחשב – וברמה בסיסית במערכות הפעלה. בבסיס השיטה עומד העיקרון הבא:

עקרון הלוואיות: אם קרה משהו לאחרונה, סיכוי סביר שיקרה שוב משהו דומה בעתיד הקרוב.

Exponential Smoothing Methods: שיטת חיזוי לזמן burst. מספקים ממוצע משוקלל נע (עם משקלים שקטנים אקספוננציאלית) של כל תחזיות העבר. זה מתאים לדטה ללא מגמה של ירידה/ עליה. המטרה היא להעריך את הממוצע הנוכחי ולהשתמש בו כאמד לערך העתידי. באופן פורמלי:

$$F_{t+1} = F_t + \alpha(y_t - F_t)$$

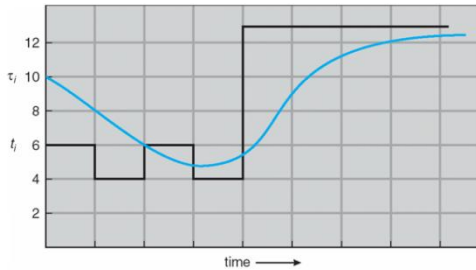
סיכום מערכות הפעלה | © גיא יער-און

- F_{t+1} : התחזית לזמן $t+1$.
- $Y_t(t)$: התחזית האחרונה שראינו בזמן t .
- a : ערך קבוע המקיים $0 < a < 1$.

מדוע מדברים על אקספוננציאלי? אם נפתח את הנוסחה – נקבל את הנוסחה הבאה:

$$F_{t+1} = \alpha y_t + \alpha(1-\alpha)y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \alpha(1-\alpha)^3 y_{t-3} + \dots + \alpha(1-\alpha)^{t-1} y_1$$

הירידה האקספוננציאלית באה לידי ביטוי בחזקה של $(1-a)$: ככל שמתרחקים מהburst הנוכחי החזקה רק עולה. כיוון של $1-a < 1$ זו ירידה אקספוננציאלית.



לדוגמה: כאשר $a = 0.5$

CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12

- את הערך של a נקבע באמצעות חיזוי ערכים שונים של a (למשל $a=0.1, 0.2, \dots, 0.9$). הערך של a שנותן את הroot-mean-square error הקטן ביותר הוא זה שנשתמש בו.

- כיצד נקבע את F_{t+1} ? הרי זמן ההתחלה לא ידוע לנו. ניתן או לקחת את הערך הראשון עצמו בזמן, או ממוצע של כמה ערכים ראשונים בסדרה.

Priority Scheduling

- שיטה נוספת שבה לכל תהליך מצמידים מספר שמייצג את העדיפות שלו. המעבד מוקצה לתהליך עם העדיפות הגבוה ביותר (המספר הגבוה או הנמוך ביותר – אפשר גם ככה וגם ככה).
- גם בשיטה זו ניתן לבצע preemptive, nonpreemptive: אם יגיע תהליך עם עדיפות גבוהה יותר – נשנה את הסדר הרצה על המעבד (נפסיק נוכחי ונחליף).
- גם SJF הוא סוג של משיטה זו שמתעדף לפי burst time.
- מה הבעיה בשיטה זו? אם תהליך עם עדיפות מסוימת מחכה, ובכל שלב מגיע תהליך אחר שעוקף אותו. (למשל: עומדים בתור למשרד הפנים ובמשך שעות מגיעים כל הזמן אנשים עם פטור מתור). זה נקרא starvation. פתרון אפשרי לבעיה זו היא Aging: משפרים את העדיפות של התהליך ככל שהוא מחכה יותר זמן.

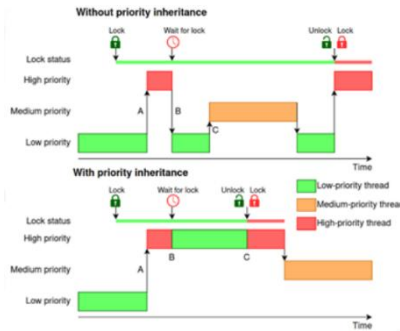
דוגמה: בתחילה ירוץ כאן תהליך 2 שהוא בעל העדיפות הגבוהה ביותר, לאחר מכן תהליך 5, תהליך 1, 3, 4 וכו'. זה כמובן בnonpreemptive.

Process	Burst Time	Priority (1=highest)
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2 msec



Priority Inversion (inheritance): היפוך עדיפויות. נניח מצב שבו תהליך פחות חשוב מצליח לעכב תהליך מאוד חשוב. זה נשמע לא הגיוני במערכת עם עדיפויות – אך קורה בגלל משאבים משותפים. נניח וישנם 3 תהליכים, L נמוך, M בינוני ו-H גבוה. מתבצע התרחיש הבא:

- L מתחיל לרוץ ותופס מנעול על משאב מסוים (קובץ נניח).
- H מתעורר, כיוון שיש לו עדיפות גבוהה הוא מעיף את L מהמעבד ומתחיל לרוץ.
- בשלב מסוים, H צריך את אותו משאב של L מחזיק. לכן H נחסם ומחכה של ישחרר את המנעול.

- עכשיו L חוזר לרוץ, מסיים, משחרר את המנעול. ואז: תהליך M מתעורר – כיוון ש-M חזק יותר מ-L הוא מעיף את L מהמעבד.
- התוצאה: תהליך H החשוב מחכה ל-L אבל L לא יכול לרוץ כי M תפס לו את המעבד – כלומר תהליך בינוני מעכב תהליך גבוה.

על מנת לפתור את זה משתמשים בירושת עדיפויות – בעת שתהליך H מנסה לתפוס מנעול שמוחזק ע"י L המערכת אומרת כי ניתן לזמנית את העדיפות של H. כעת L חשוב מאוד – וכש-M יתעורר הוא לא יכול להעיף את L מהמעבד. אם במקרה M גדול מ-H, ירושת עדיפויות גם לא תעזור כי באמת M אמור לרוץ (הכי חזק כרגע).

Round Robin (RR)

- שיטה זו בעיקר למערכות מסוג time-sharing.
- כל תהליך מקבל זמן יחסית קצר (time quantum) על גבי המעבד. בד"כ בין 10-100 מילישניות. זה מתבצע בצורה ציקלית (מעגלית) על פני תור מסוים.
- שיטה זו מבטיחה כי אם ready queue ישנם n תהליכים והtime quantum הוא q אז כל תהליך יקבל $1/n$ מזמן המעבד באינטרוולים רצופים של q יחידות זמן ואף תהליך לא ימתין יותר מ-q(n-1) (כל קודמיו, כפול q הזמן שלוקח) יחידות זמן.
- אם q גדול: זה כמו FIFO.
- שיטת RR מבטיחה שאף תהליך לא ישאר "רעב" (starvation).

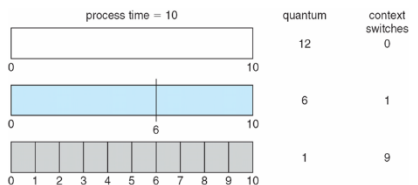
לדוגמה: אם $q=20$, שיטה זו תראה כך (כאשר המספרים שכתובים מעלה הם כמה זמן נותר לכל אחד מהתהליכים לרוץ). חוזרים בכל שלב בצורה ציקלית – לאלו שנותרו עוד בתור (remaining time > 0).

□ The Gantt chart is:

53	33	33	33	33	13	13	13	0	0	0
17	17	0	0	0	0	0	0	0	0	0
68	68	68	48	48	48	28	28	28	8	0
24	24	24	24	4	4	4	0	0	0	0

P ₁	P ₂	P ₃	P ₄	P ₁	P ₃	P ₄	P ₁	P ₃	P ₃	
0	20	37	57	77	97	117	121	134	154	162

□ Typically, higher average turnaround than SJF, but better response



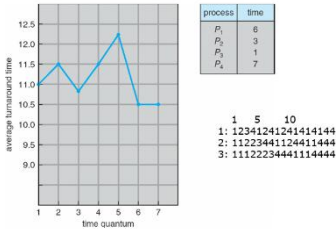
נתבונן משמאל – ישנה דילמה חשובה מאוד בבחירת זמן הtime quantum בשיטת RR: הקשר בין אורך הקוונטום לבין overhead (תקורה) של המערכת. נתבונן בתהליך שאורך 10 יחידות זמן.

1. מקרה ראשון: הקוונטום גדול מהזמן שהמעבד צריך. $10 < 12$. התוצאה היא שהתהליך נכנס למעבד ומסיים את כל העבודה שלו בפעם אחת. מספר context switch הוא 0 עבור אותו תהליך כתוצאה מכך. כלומר: ככל שהקוונטום גדול יותר, RR מתנהגת יותר ויותר כמו FCFS – מערכת יעילה ללא בזבז זמן על החלפות.
2. מקרה שני: קוונטום ששווה ל-6. קטן מ-10. התוצאה היא שהוא רץ 6 יחידות, מועף לסוף התור לכן מתבצע קונטקסט סוויץ' אחד ולכן שילמנו על זה קצת.

סיכום מערכות הפעלה | © גיא יער-און

3. מקרה אחרון: $q=1$. כעת יש 9 קונטקס סוויץ', יש overhead גבוה מאוד! המעבד יבלה יותר זמן בלהחליף מאשר לבצע עבודה בפועל.

הכלל שנגזר מדוגמה זו: נרצה ש $time_context_switch > q$.



עם זאת, כדאי שנשים לב לנתון הבא: זה לא תמיד נכון שאם נגדיל את q זמן השהייה הממוצע במערכת ישתפר. הקשר איננו לינארי! (למרות הדוגמה הקודמת). נתבונן בגרף משמאל שממחיש זאת. מדוע זה קורה? זמן השהייה הממוצע במערכת תלוי בשאלה "האם הקוונטום מאפשר לסיים תהליכים מהר?". בדוגמה משמאל, יש לנו תהליכים באורך 1 3 6 ו7. אם נבחר קוונטום של 6 – כולם פרט לאחרון יסיימו בסבב אחד. זמן השהייה הממוצע כן משתפר כאשר רוב התהליכים מסיימים את burst שלהם בתוך קוונטום אחד. אם הקוונטום קטן מדי ביחס לאורך התהליכים – יצטרכו הרבה קונטקס סוויץ'.

Multilevel Queue

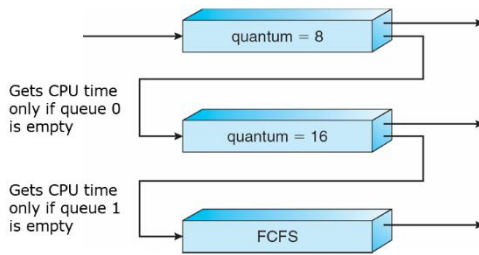
- חלוקת ready queue לכמה תורים נפרדים. למשל: תור עבור סוג תהליך. מדוע זה טוב? כי יש תהליכים שחשובים יותר, אותם ניתן להכניס לתור עבור סוגי תהליכים חשובים.
 - לכל תור נקבע אלגוריתם scheduling משלו (לא בהכרח זהים!).
 - כעת ישנן כמה גישות למי מהתורים יקבל את המעבד כרגע:
1. עדיפות גבוהה – Fixed priority: שיטה היררכית נוקשה. תור Foreground (תהליכים אינטראקטיביים מול המשתמש) תמיד יקבל עדיפות – כל עוד יש שם תהליך אחד התורים האחרים לא יקבלו כלל את המעבד. ישנה כאן סכנה של starvation: אם המשתמש כל הזמן מזיז את העכבר או המקלדת, תהליכי הרקע לא ירוצו.
 2. פריסת זמן – Time slice: גישה "דמוקרטית" יותר. המעבד מחלק את הכוח שלו לפי אחוזים. למשל מתוך כל שנייה של עבודה – 80% מהזמן לתהליכים בכוח החשוב, ו20% לחלק הפחות חשוב.

Multilevel Feedback Queue

זו השיטה הכי נפוצה במערכות הפעלה מודרניות. כאן, עובדים עם תורים נפרדים לפי מאפייני cpu burst שלהם. הרעיון המרכזי מתבסס על ענישה על שימוש מוגזם במעבד. השיטה הזו מתעדפת תהליכים קצרים ואינטראקטיביים על פני תהליכים כבדים. תהליך יכול לעבור בין מגוון תורים. תהליך תמיד נכנס לתור העליון – אם מסיים את עבודתו מהר, הוא נשאר בעדיפות גבוהה. אם הוא חמדן ומנצל את כל הקוונטום שהוקצה לו בתור הזה – הוא יורד קומה לתור נמוך יותר עם אלגוריתם אחר – אבל תמיד המערכת תדאג שאם יש מישהו בתור עליון, הוא יתבצע קודם. ישנה היררכיה. בשביל להשתמש באלגוריתם זה על המערכת לקבוע:

כמה תורים יהיו, איזה אלגוריתם רץ בכל תור, מתי מעבירים תור לתור נמוך יותר (לרוב כשמסיים קוונטום שלם), מתי מחזירים תהליך למעלה? (אם תהליך מחכה יותר מדי – לא נרצה שירעב, לכן לפעמים נעלה אותו למעלה), לאיזה תור נכנס תהליך כשהוא רק נוצר (לרוב להכי גבוה).

סיכום מערכות הפעלה | © גיא יער-און



לדוגמה: כל התהליכים נכנסים לתור הראשון, ש time quantum הוא בדיוק 8. אם התהליך צריך 8 או פחות יחידות זמן, הוא יסיים את חייו בתור הראשון. אחרת: אם burst שלו היה גדול ממש מ-8, הוא יכנס לתור הבא. קודם ירוצו כל התהליכים בתור הראשון. כשהוא יתרוקן – יתחילו לרוץ התהליכים בתור השני וכן הלאה. העדיפות של שיטה זו היא שהתהליכים הקצרים יחסית ירוצו מהר, ואם התהליך יחסית כבד, הוא כל פעם יתקדם קצת. בתור האחרון למשל ננקוט ב-FCFS.

קשר בין האלגוריתמים שראינו

- הקשר בין Priority to SJF: SJF הוא למעשה אלגוריתם של תזמון עדיפות לפי burst.
- הקשר בין FEEDBACK QUEUE ל-FCFS: הרמה הנמוכה ביותר של שיטת התורים היא FCFS, שכן תהליכים כבדים שוקעים לתורים התחתונים. בתור האחרון והנמוך ביותר, בדרך כלל כבר לא משתמשים ב-RR (אין טעם לקטוע תהליך שכבר הוכח לנו קודם שכבד מאוד) אלא נותנים להם לרוץ אחד אחרי השני לפי סדר ההגעה עד שסיימו בשביל לחסוך ב-overhead (שיגורם כתוצאה מהרבה קונטקס סוויץ').
- הקשר בין Priority to FCFS: FCFS הוא תזמון לפי סדר ההגעה.

Multiple-Processor Scheduling

כאשר ישנם כמה מעבדים Scheduling הופך להיות מסובך הרבה יותר. למעשה אנחנו צריכים לשאול את עצמנו אם אנחנו עדיין עובדים בתור אחד או כמה. ישנם שני מצבים:

- Asymmetric multiprocessing: רק מעבד אחד מנהל את התזמון בין התהליכים.
- symmetric multiprocessing: כל מעבד מתזמן לעצמו (כל המעבדים ניגשים לready queue משותף או שכל אחד מהם מנהל תור כזה משל עצמו – אבל אז זה פוגע ביעילות).
- **Processor affinity:** זה צמידות. ישנו רצון שנהיה צמודים למעבד עליו אנחנו רצים כרגע. שכן, זה קשור ל-cache: תהליך שרץ על גבי מעבד ויעבור למעבד אחר עלול לאבד את cache שהוא בנה – ואז יצטרך מחדש לבנות cache שם וזה בזבז.
- Soft affinity – בקש ונשתדל לעשות.
- Hard affinity – התהליך דורש: לא מוכן לעבור למעבד אחר. תמיד דרישתו תענה.

Real-Time Scheduling

מערכות מסוג hard-real-time (כמו למשל "טיל כיפת ברזל") מאפשרות להשלים משימה קריטית בתוך פרק זמן מוגדר מראש. התהליך מבקש לקבל זמן מעבד עד לדדליין מסוים. scheduler מבטיח את קיום הבקשה – או דוחה אותה.

Algorithm Evaluation

נניח והגענו לעבודה חדשה – ואנחנו קיבלנו משימה: להחליט איזה אלגוריתם scheduling אנחנו נריץ. מה יהיו השלבים בדרך להחלטה?

1. לוקחים דוגמה קיימת – מחשבים מה יהיה המדד המתאים בכל אחד מהאלגוריתמים עבור אותה דוגמה, ולוקחים את זה עם המדד הטוב ביותר עבורנו.
2. תורת התורים – מחשבים בצורה הסתברותית את זמן ההגעה הצפוי (פואסונית וכו'). החיסרון של זה הוא שלרוב המודלים תורת התורים לא יכולה לפתור מתמטית.

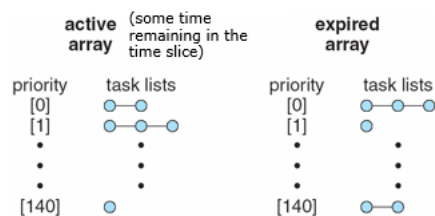
Thread scheduling

כעת עוברים מרמת תהליך לרמת החוט. ישנן כמה שיטות לקביעה מי מהחוסים ירוץ על גבי המעבד.

1. Process contention Scope (PCS): מושג זה רלוונטי עבור Many-to-one and Many-to-Many. המנהל כאן הוא ספריית החוסים, נניח שיש תהליך אחד עם עשרה חוסים. אך מערכת ההפעלה הקצתה לו רק צינור אחד לקרנל - 10 החוסים הללו יריבו ביניהם למי הזכות להשתמש בצינור זה. התחרות פנימית בתוך התהליך!
2. System contention Scope (SCS): רלוונטי עבור חוסים שהקרנל מכיר. המנהל הוא המתזמן של מערכת ההפעלה. במודלים של one-to-one כל חוט שהמתכנת יוצר, מקבל חוט קרנל תואם. כאן החוט מתחרה מול כל שאר החוסים במחשב - התחרות ברמת המערכת כולה.

כעת נעבור לדון על scheduler של מערכות הפעלה ספציפיות.

1. Solaris scheduling: מערכת ההפעלה solaris - משתמשת בtime sharing (כלומר מסוג RR), על בסיס של multi-level feedback queue. המערכת קובעת חוק מעניין - ככל שהעדיפות גבוהה יותר, כך הקוונטום קטן יותר. מדוע? כי תהליכים בעדיפות גבוהה לרוב הם אינטראקטיביים שצריכים תגובה מיידית, אבל מסיימים את עבודתם מהר מאוד. תהליכים בעדיפות נמוכה לרוב הם CPU Bound (חישובים כבדים) שלא צריכים תגובה מהירה.
 2. Windows scheduling: המערכת של ווינדוס תמיד תעדיף להריץ את החוט בעל העדיפות הגבוהה ביותר. אם חוט חשוב יותר מתעורר - הוא קוטע את החוט שכרגע רץ. חוט מפסיק לרוץ באחד מהמקרים הבאים: הוא נעול, נגמר לו הקוונטום, או שחוט חזק יותר הגיע. ווינדוס מחלקת את העדיפויות ל-32 רמות.
 - רמות 1-15: רוב התהליכים הרגילים (כרום, ספוטיפיי וכו'). העדיפות שלהם גמישה - המערכת יכולה להעלות או להוריד אותם ברמה לפי צורך.
 - רמות 16-31: חוסים קריטיים של מערכת ההפעלה שלא עוברים שינויי עדיפויות. תמיד יקבלו עדיפות.
 - רמה 0: שמורה לidle Thread (מה שהמעבד מריץ כשאינו עושה דבר - חוט ה"בטלה" מלשון בטלון).
- לכל רמת עדיפות יש תור משלה, המתזמן סורק תמיד מהרמה הגבוהה ביותר (31) ומטה. רק כשתור גבוה ריק לגמרי, הוא ירד קומה.
3. Linux scheduling: יש עקרונות דומים למה שראינו בווינדוס. כמו כולם - משתמשת בpreemptive: היא לא מחכה שתהליך ירצה להסתיים לבד אלא עוצרת אותו אם מגיע אחד חשוב יותר. מחלקת את המשימות לטווחים של 0-140. אלו בטווח 0-99 הם העדיפויות הגבוהות ביותר, מיועדות למשימות קריטיות. האחרים - 100-140 הם התהליכים הרגילים. לתהליכים בעדיפות גבוהה יש קוונטום גבוה יותר, בשונה מ-solaris.



זו שיטה שמשמשת כחלק ממנגנון התזמון בlinux. המטרה היא להגדיל את ההוגנות בהקצאת המעבד. עבור כל priority נשמרות שתי רשומות - בראשונה תהליכים אשר לא ניצלו את time splicing שלהם במלואו, ובשנייה תהליכים שכן ניצלו אותו. המעבד יוקצה למשימות ע"פ priority כאשר המעבר הוא מרשימה שמאלית לרשימה ימנית וכאשר הרשימה השמאלית ריקה תבוצע החלפה של שתי הרשימות. מטרתו לפתור את ההרעבה ולשמור על הוגנות. במקום רשימה אחת גדולה שבה תהליכים חזקים בעלי עדיפות גבוהה תמיד דוחקים את החלשים משתמשים בשתי מערכי רשומות.

- Active array: מכיל את כל התהליכים שיש להם עדיין זמן מעבד לניצול. המעבד בוחר מתוכה את התהליך עם העדיפות הגבוה ביותר ומריץ אותו.

סיכום מערכות הפעלה | © גיא יער-און

- **Expired array:** מכילה תהליכים שכבר ניצלו את זמן המעבד שלהם. הם מחכים בצד.

כיצד זה עובד? המתזמן מושך תהליכים מ-Active, כל תהליך מקבל זמן ריצה. כשנגמר לתהליך זמן הריצה הוא עובר ל-Expired. בעת שהרשימה הפעילה מתרוקנת לגמרי – כלומר כל התהליכים סיימו את הזמן שלהם ועברו ל-Expired: המערכת מבצעת swap (החלפה). הרשימה שהייתה Expired הופכת להיות Active החדשה. הרשימה שהייתה Active הופכת להיות Expired החדשה. למה זה הוגן? בשיטה זו אי אפשר לדרוס תהליכים אחרים לנצח. גם אם תהליך בעל עדיפות נמוכה לא קיבל זמן מעבד בסיבוב הזה – בעת שכל החזקים יסיימו את הזמן שלהם ויעברו ל-Expires, התהליך החדש יקבל הזדמנות להיות ב-Active (כי החליפו ביניהם) ולהריץ את המשימות שלו בסיבוב הבא.

עיבוד batch: שיטה בה המחשב מריץ קבוצת משימות ארוכות זו אחר זו ללא התערבות המשתמש. המטרה היא למקסם את ניצול המעבד. ממזערים את מס' הקונטקס סוויץ'.

סביבה אינטראקטיבית: מערכת בה המשתמש מתקשר עם המחשב בזמן אמת, והדגש העיקרי שלה הוא על זמן תגובה מהיר. יהיו החלפות בקצב תכוף.

CPU Bound: מצב בו קצב ההתקדמות של תהליך מוגבל ע"י מהירות המעבד בלבד (בחישובים מתמטיים כבדים למשל), כיוון שהוא כמעט ולא עוצר להמתנה של קלט פלט.

הרצאה 6

בהרצאה הקרובה נדבר על סינכרוניזציה.

מהי סינכרוניזציה?

- הבעיה שאנחנו מנסים להימנע ממנה היא data inconsistency: כלומר – אין עקביות בנתונים. מבצעים פעולות על הדטה, ומצפים להגיע לתוצאה מסוימת באופן עקבי אך לא מגיעים לתוצאה הזו. מתי עלולים להתקל בבעיה הזו? נראה כי אם התהליכים לא מופרעים במהלך ריצתם לא אמורה להיות בעיה.
- אם תהליך מאבד את המעבד כתוצאה מinterrupt מבלי שהשלים את burst הנוכחי שלו, אבל אף תהליך אחר לא יכול לגשת למשתנים של התהליך הנ"ל – שוב: לא אמורה להיות בעיה.
- מתי כן יש בעיה? אם התהליך מעדכן משתנים משותפים לתהליכים אחרים עלולה להיות בעיה כי אין תיאום וסינכרוניזציה ביניהם – הם פועלים על אותו זיכרון מבלי לדעת מה השני עושה.

Producer-Consumer Problem: כפי שראינו בעבר, יש לנו במודל זה

producer שאחראי על ייצור וconsumer שצורך ממנו. רצים על ה shared buffer בצורה ציקלית.

מה הבעיה והחיסרון במימוש הזה? אנחנו לא מנצלים את כל היחידות של

הbuffer אלא מנצלים n-1 תמיד בגלל in, out.

בואו ננסה לפתור את זה – ולהשתמש בכל הגודל של הבאפר: כיצד?

נוסיף משתנה בשם count, שיגיד לנו כמה איברים נכתבו ע"י producer ולא נקראו ע"י consumer. נתבונן על קוד שמממש את זה משמאל –

```
while (TRUE)
{
  /* produce an item and put
  in nextProduced */
  while (count == BUFFER_SIZE)
    ; // do nothing
  buffer[in] = nextProduced;
  in = (in + 1) % BUFFER_SIZE;
  count++;
}
producer

while (TRUE)
{
  while (count == 0)
    ; // do nothing
  nextConsumed = buffer[out];
  out = (out + 1) % BUFFER_SIZE;
  count--;
  /* consume the item in
  nextConsumed */
}
consumer
```

כל אחד משני התהליכים (צרכן, יצרן) מריץ קוד תקין. אבל: ההרצה של הקוד אצל היצרן והצרכן מתבצע במקביל – ועלולים להתקל בבעיה של data consistency עבור המשתנה count: מדוע? מכניסים את count הרי לתוך רגיסטר, ולרגיסטר עושים add\sub וכו'. ואז עלולים להתקל במצב הבא:

אם נניח כי count=5, נראה כי ייתכן ויקרה:

S0: producer execute	register1 = count	{ register1 = 5 }
S1: producer execute	register1 = register1 + 1	{ register1 = 6 }
S2: consumer execute	register2 = count	{ register2 = 5 }
S3: consumer execute	register2 = register2 - 1	{ register2 = 4 }
S4: producer execute	count = register1	{ counter = 6 }
S5: consumer execute	count = register2	{ counter = 4 }

מה קרה כאן? נניח שתהליך ראשון בשלב S₂ הפסיק והמעבד עבר לתהליך שני. מה שקרה הוא ששני התהליכים ניגשו בו זמנית למשתנה משותף count, ובשלב השלישי התהליך השני קיבל את הערך counter=5 ובמקום שהתהליך השני יקבל ערך counter=4 הוא קיבל 5. והמסקנה שלנו פשוטה: זה תלוי בסדר ההרצה – כיוון ששני התהליכים חולקים משאב משותף – משתנה.

סיכום מערכות הפעלה | © גיא יער-און

Race condition: "מצב מירוץ" – מצב שבו יש כמה תהליכים שניגשים ומשנים את אותו הדטה במקביל. התוצאה שנקבל תהיה תלויה בסדר הספציפי שבו מתבצעת הגישה של התהליכים השונים – ולכן התוצאה לא דטרמיניסטית. על מנת להימנע ממצב זה נרצה להבטיח שרק תהליך אחד יוכל לשנות את הדטה בו זמנית. נבחין: אם שניים רוצים לקרוא – אין לנו בעיה. אם שניים רוצים לכתוב – כן יש לנו בעיה. מה עם אחד שרוצה לכתוב ואחד לקרוא? לא נרצה לאפשר: שכן יתכן שזה שקורא יקרא ערך לא עדכני.

Critical Section Problem

כעת נגדיר מושג שיעזור לנו לפתור את ה-race condition. מדובר למעשה במקטע (סגמנט) של קוד שבו התהליך משנה משתנים משותפים עם תהליכים אחרים, טבלאות משותפות, כותב לקובץ וכו'. בהינתן מערכת עם n תהליכים $P_0, \dots, P_{n-1}$ שלכל אחד יש critical section משלו, נרצה שאם תהליך נמצא בcritical section שלו אז אף תהליך אחר לא יריץ פקודות באותו critical section. לכן הבעיה היא: לתכנן מעין פרוטוקול שיבטיח זאת. נרצה לפתח מנגנון שיבטיח שרק אחד מהתהליכים יריץ את הcritical section שלו.

```
do {
    entry section
    critical section
    exit section
    remainder section
} while (true);
```

כיצד נפתור את הבעיה? נוסיף קוד לתהליכים שיבטיח שרק תהליך אחד נכנס בו זמנית לcritical section.

נעטוף אותו בשני מקטעים:

- Entry section: צריך לרוץ לפני שנכנסים לקטע הקוד הקריטי.
- Exit section: צריך לרוץ מיד לאחר שסיימנו לבצע את קטע הקוד הקריטי.
- נבחין: זה לא קוד שעושה לנו לפונקציונליות של התהליך, אלא קוד שנועד לעזור לנו לפתור את בעיית הסינכרוניזציה. ולכן זה מעין overhead.

כשאנחנו מציעים פתרון באמצעות הוספת שני המקטעים הנ"ל – **נרצה שהפתרון יענה על שלוש דרישות מרכזיות:**

- mutual exclusion: אם תהליך P_i רץ כרגע בקטע הקוד הקריטי שלו, אז אף תהליך אחר לא יכול לרוץ בקטע הקוד הזה באותו זמן. (ברור).
- Progress: אם אף תהליך לא רץ כרגע בcritical section שלו, ויש תהליכים או תהליך שרוצים להכנס לקטע הקוד הקריטי שלהם – אז הבחירה בתהליך הבא שיכנס לcritical section שלו לא יכולה להידחות ללא הגבלה (כלומר, ישנה התקדמות בדרישותיהם). וכן, רק התהליכים שרוצים להכנס לקטע הקוד הקריטי שלהם – יהיו שותפים לתהליך הבחירה של מי יכנס לקטע הקוד הקריטי כרגע.
- Bounded waiting: חייב להיות חסם על מס' הפעמים שבהם תהליכים אחרים יכולים להיכנס לcritical section שלהם לאחר שתהליך מסוים ביקש להכנס לקטע הקוד הקריטי שלו, ולפני שהבקשה שלו מאושרת. (כלומר, מעין "פייריות" בין התהליכים. ישנו חסם על מס' הפעמים שאחרים יכנסו לקטע הקוד הקריטי שלהם מהרגע שאני ביקשתי להכנס – זה עוזר לנו להימנע ממצב של starvation).

Peterson's Solution

- הפתרון הראשון שנציג לבעיית הcritical section מבוסס קוד – בניגוד לאלו שנראה בהמשך שמבוססים על חומרה. אך: הפתרון מתאים לתרחיש של שני תהליכים
- הפתרון מניח שאי אפשר לעשות interrupt באמצע load או store – וזו הנחה לגיטימית.

סיכום מערכות הפעלה | © גיא יער-און

- הפתרון משתמש בשני משתנים משותפים: משתנה שלם turn שיגיד לנו: "התור של מי מהתהליכים להכנס עכשיו לקטע הקוד הקריטי", משתנה שני הוא מערך בוליאני בגודל 2: flag – מערך זה משמש לומר האם תהליך מסוים מעוניין להכנס לקטע הקוד הקריטי שלו. אם $flag[i]=true$ הוא מעוניין להכנס.

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    critical section
    flag[i] = false;
    remainder section
} while (true);
```

- נתבונן בקטע הקוד עבור תהליך $P[i]$ של האלגוריתם: ראשית הוא מעדכן שהוא מעוניין להכנס לקטע הקוד הקריטי – ובאופן אוטומטי הוא מעביר את התור לצד השני: לתהליך j . בהמשך: כל עוד שזה גם התור של התהליך השני, וגם התהליך השני רוצה להכנס לקטע הקוד הקריטי שלו – אז תישאר במקום. מספיק שאחד מהם לא יתקיים (זה לא התור שלו, או $flag[j]=false$) ואנחנו כן נוכל להכנס. כיצד זה יכול להיות התור של j ? הרי במקביל – j עושה את אותו הדבר, ומעניק לו את התור. כלומר: זה ששיננו את התור עזר לנו. בהמשך – אנחנו משנים את ה- $flag[i]$ ל- $false$ כי אנחנו כבר לא צריכים את קטע הקוד הקריטי.
- שלושת התנאים מתקיימים באלגוריתם של פטרסון:

1. **טענה:** מתקיים באלגוריתם המ- 2 $mutual\ exclusion$ (לא יותר מתהליך אחד יריץ את קטע הקוד הקריטי שלו בו זמנית).

הוכחה: נניח בשלילה שגם P_0 וגם P_1 מריצים את קטע הקוד הקריטי שלהם. זה גורר ששניהם הריצו את $while$ ועברו אותו. אז, P_0 נמצא בקטע הקוד הקריטי שלו וכן $turn=i$ והבדיקה עבור j הייתה שלילית ולכן: או $flag[j]=false$ או $turn=i$. בדומה, P_1 נמצא בקטע הקוד הקריטי שלו ולכן הבדיקה עבור i הייתה שלילית ולכן או $flag[i]=false$ או $turn=j$. אבל אנחנו יודעים כי $flag[i]=flag[j]=true$ שכן הם נמצאים בפנים. לכן בהכרח נקבל כי $turn=i=j$ אבל $i \neq j$! בסתירה.

1. Progress: ישנן 2 אפשרויות:

אפשרות ראשונה – תהליך אחד מבקש להכנס לקטע הקוד הקריטי שלו, והשני לא נמצא בקטע הקוד הקריטי שלו: לכן לפי האלגוריתם התהליך יכנס מיידית לקטע הקוד הקריטי שלו.

אפשרות שנייה – שני התהליכים מבקשים להכנס לקטע הקוד הקריטי שלהם. אך רק אחד יצליח להיות האחרון שיעביר את התור לצד השני – ולכן תהיה בהכרח החלטה מי מבניהם יכנס.

2. Bounded waiting: נבחין כי יש חסם של 1 על מס' הצעדים. לא יתכן ונ יוכל להכנס לקטע הקוד שלו עד שו יסיים אם שניהם ביקשו בו"ז.

```
/* repeat for all partners */
for (count = 0; count < (NUMPROCS-1); count++) {
    flags[j] = count; // I think I'm in position "count" in the queue
    turn[count] = j; // and I'm the most recent process to think I'm in position "count"
    // wait until everyone thinks they're behind me or someone later than me thinks they're in position
    "count"
    "wait until (for all k != i, flags[k]<count) or (turn[count] != i)
    // now I can update my estimated position to "count"+1
}
// now I'm at the head of the queue so I can start my critical section
run critical section
flags[i] = -1; // I'm not in the queue anymore
```

ישנה הרחבה ליותר מ-2 תהליכים, עליה דודי יומר מרמז שיכול לשאול במבחן:

הקוד מנסה להפעיל לוגיקה דומה לזה של הפתרון של פיטרסון. כאשר תהליך רוצה להיכנס לקטע הקריטי הוא מתחיל להתקדם במעין תור תהליכים דמיוני, שבו הוא לא יודע כמה מהתהליכים האחרים באמת

נמצאים בו או מה המיקום שלהם. (בגלל המקביליות בהרצה). הקוד מאפשר לתהליך לצאת מהתור רק כשהוא בטוח שהוא בראש התור. לצורך כך הוא מעדכן את התור שלו בכל שלב (במערך flags) וכן מעדכן שהוא האחרון שעדכן את התור הזה (במערך turn). בצורה זאת במערך turn נשמר לנו מי התהליך האחרון שהכניס את עצמו לרמה הנוכחית. ההתקדמות של התהליך בתור מתאפשרת ככל שהוא רואה שכל התהליכים האחרים חושבים שהם בשלבים מוקדמים יותר בתור ממנו, או שיש לפחות תהליך אחד שחושב שהוא באותה רמה כמו התהליך שלנו, אבל רשם את עצמו ב- $turn$ ברמה הזאת אחרי התהליך שלנו (ובכך דרס את הרישום של התהליך שלנו). הדבר מאפשר

סיכום מערכות הפעלה | © גיא יער-און

שמספר התהליכים המקסימלי בכל רמה בתור הולך ויורד (באחד) כך שבראש התור יכול להיות רק תהליך אחד. השחרור לאחר ה-critical section הוא באמצעות סימון רמה של מינוס אחד, מה שלא מהווה איום על אף תהליך אחר בתור.

פתרונות מבוססי חומרה:

רוב הפתרונות מבוססי החומרה מבוססים על רעיון של locking: הגנה על הcritical section באמצעות מנעול.

כיצד יראה פתרון מבוסס lock? כשננסה להכנס לקטע הקוד הקריטי – ננסה להשתמש בlock וברגע שנסיים את קטע הקוד הקריטי: נבצע release לlock – ותהליכים אחרים יוכלו להשתמש בו.

ברוב המערכות המודרניות – כבר יש הרבה instructions אטומים: שנעולים באמצעות מנעול. ישנן 2 מרכזיות שנדון בהן ונראה כיצד ניתן לממש lock באמצעותן:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}

do {
    while (test_and_set(&lock))
        ; /* do nothing */
        /* critical section */
    lock = false;
        /* remainder section */
} while (true);
```

Test and set: פעולה שמבצעים על משתנה בוליאני ומחזירים משתנה בוליאני. מה קורה כאן? מעבירים לפונקציה ערך בוליאני. הפונקציה משנה את ערך המשתנה לTrue ומחזירה את הערך הקודם שהיה למשתנה הבוליאני. מה שמובטח – בעת שהפעלנו את הפעולה: כל מי שיבצע Test_and_set לאחריו יראה ערך של True בוודאות. מדובר בפעולה אטומית (רק אחד יכול לבצע).

בפונקציה זו אנחנו נשתמש על מנת לכתוב פונקציית lock: נראה כי מובטח לנו באמצעות הפונקציה הזו שאנחנו הראשונים שנכנסנו וראינו כי ערך המשתנה הוא true, שכן מבצעים test_and_set וזו מתבצעת בצורה אטומית. לכן, אחד התהליכים (הראשון) יריץ אותה ויקבל true ואכן יכנס. בסיום – הוא ישנה את lock לfalse, ולכן הוא משחרר את המנעול עבור תהליכים אחרים: וזה מבטיח שמיד לאחריו תהליך אחר יוכל להשתמש בו.

Swap: קוד אטומי נוסף הוא swap: ללא הפרעה.

```
do {
    key = TRUE;
    while (key == TRUE)
        Swap (&lock, &key );
    // critical section
    lock = FALSE;
    // remainder section
} while ( TRUE);
```

גם בקטע קוד זה ניתן להשתמש על מנת לממש lock(): כל תהליך שמנסה להכנס לקטע הקוד הקריטי יוצר לעצמו משתנה מקומי key=True. כעת, כל עוד ערך זה באמת "אמת" הפונקציה Swap מחליפה בצורה אטומית בין הערך של lock לזה של key. ישנם שני תרחישים:

1. אם המנעול היה false (פנוי): אזי key=False והלולאה תעצר, התהליך יכנס לקטע הקוד הקריטי והמנעול יינעל (יהפוך לTrue, כי key=True וביצענו החלפה). כך ששאר התהליכים יישארו תקועים בwhile ולא יכנסו למנעול.

2. אם lock היה כבר אמת – כלומר נעול, ההחלפה תשאיר אותו true והתהליך ימשיך להסתובב בלולאה עד שהמנעול יתפנה.

בסיום, בעת שהתהליך מסיים את קטע הקוד הקריטי הוא משחרר את המנעול (הופך אותו לfalse) מה שמאפשר לתהליך הבא בתור להכנס.

- נבחין כי שני הפתרונות שהצענו לא מקיימים את עקרון bounded waiting: שכן אם יש כמה תהליכים, ויש תהליך שכל הזמן מאבד זמן מעבד בעת הבדיקה של המנעול – ולכן יתכן מצב של הרעבה של אחד התהליכים. דימוי טוב של דודי: ילד שמבצע בצורה אינסופית מתקן בלונה פארק, כי בכל שלב הילד שלפניו הולך לקנות גלידה, ואז חוזר לתור.

סיכום מערכות הפעלה | © גיא יער-און

לכן – נרצה ללמוד פתרון שתומך גם בדרישה של bounded waiting: נגדיר משתנים משותפים הבאים: מערך בגודל n בשם waiting – בוליאני. ששומר לכל תהליך אם הוא רוצה להכנס לקטע הקוד הקריטי שלו. וכן משתנה בוליאני משותף נוסף בשם lock.

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```

הפתרון הנ"ל מונע מצב של הרעבה ומחולק לכמה חלקים:

1. **בעת הכניסה** – תהליך i מסמן שהוא מחכה $waiting[i]=true$ ומנסה לתפוס את המנעול באמצעות `test_and_set`. הלולאה `while` תמשיך לרוץ כל עוד הוא מחכה ומישהו אחר מחזיק במנעול – בעת שהוא מצליח, הוא משנה את $waiting[i]=false$ ונכנס לקטע הקוד הקריטי שלו.
2. **בעת יציאה:** לאחר שביצע את קטע הקוד הקריטי שלו, התהליך לא משחרר את המנעול ישירות. התהליך מחפש בצורה ציקלית מעגלית את התהליך הבא בתור שבאמת מחכה – $waiting[j]=true$. אם נמצא תהליך שכזה – "נעביר לו את המקל" ע"י שינוי ה $waiting[j]$ שלו ל `false`. כעת התהליך הבא יפסיק את ה `while` שלו ויכנס. אך, אם לא נמצא אף תהליך שמחכה ($j=i$) שזהו מצב שעשינו סיבוב ציקלי שלם, נשחרר את המנעול.

כעת נשים לב כי כל התנאים אכן מתקיימים, שכן:

- **Mutual exclusion:** כל תהליך שיוצא מקטע הקוד הקריטי יכול להעביר רק אחד $waiting[i]=true$ ל `false`. וכן רק תהליך אחד יכול לתפוס את המנעול אם הוא משוחרר – כלומר מבין מי שממתין כולם למעט אחד יהיו ללא המנעול ועם $waiting=true$. לכן רק אחד יוכל להשתמש.
- **Progress:** תהליך שיוצא מקטע הקוד הקריטי יעביר את המנעול ל `false` או שיקבע $waiting[i]=false$ לאחד התהליכים הממתנים – ולכן אחד התהליכים הממתנים יוכל להתקדם.
- **Bounded waiting:** המעבר הציקלי על רשימת התהליכים בעת היציאה מקטע הקוד הקריטי מבטיח שבמקסימום n-1 תהליכים אחרים יריצו את קטע הקוד הקריטי שלהם מהרגע שאנחנו ביקשנו לרוץ עד שרצנו. (במקרה ה"גרוע" ביותר בכל שלב הוא נותן לאחד אחריו בתור ולכן במקסימום תהליך יחכה n-1 סבבים).

Mutex Locks

- `Mutex()` זה פתרון תוכנה לבעיית הקוד הקריטי. הוא ממומש ע"י מערכת ההפעלה, אך דורש תמיכה של חומרה שכן המימוש דורש פקודות אטומיות – כמו `test and set`.
- ה `mutex` הוא משתנה בוליאני שמשמש כמנעול. אופן השימוש מתבצע כך: תפיסת מנעול באמצעות `acquire()` ו `release()` משחרר את המנעול.

Semaphore

```
□ acquire() {
    while (!available)
        ; /* busy wait */
    available = false;;
}
□ release() {
    available = true;
}
□ do {
    acquire lock
    critical section
    release lock
    remainder section
} while (true);
```

סיכום מערכות הפעלה | © גיא יער-און

- Semaphore הוא משתנה integer שעוזר לסינכרוניזציה. ניתן לבצע עליו שתי פעולות אטומיות –

1. Wait: בעת שערך הסמפור קטן או שווה לאפס – נשארים במקום. כאשר הערך שלו חיובי ממש: ניתן להתקדם ולהוריד את הערך שלו באחד.

2. Signal: מעלה באחד את הערך של הסמפור.

- נעיר, למרות שטענו כי הפעולות אטומיות: רק החלקים של $S++$, $S--$ הם אטומיים.

- לסמפור גם נקרא counting semaphore: כיוון שמדובר בערך שלם ולא בוליאני, ניתן

להשתמש בו על מנת להגביל מספרית את מס' התהליכים שיכולים לבצע דבר מסוים. למשל – אם יש לנו פיזית 5

מדפסות, נוכל להגדיר סמפור שמאוחל לערך 5 שידאג שלא יתכן כי יותר מ-5 תהליכים ינסו להדפיס (שהרי אין מדפסות).

- אם הסמפור בינארי, כלומר מקבל ערכים מ- $\{0,1\}$ נקרא לו mutex – בדיוק אותו דבר.

- Busy waiting: מצב שיש לולאה וכל הזמן אנחנו מבצעים בה בדיקות. מדובר בבזבוז של משאבים! מה יכולה להיות חלופה לזה?

ניתן לבצע לעצמנו Blocking ומישהו מבחוץ יחזיר אותי לזמן מעבד כשנוכל. כלומר נרצה לשנות את הסמפור כפי שהגדרנו

שיוכל לנהל מעין מחסנית של תהליכים – שימתינו עד שהערך של הסמפור יחזור להיות חיובי, והם יכולים להשתמש בו. כך לא

יהיה busy waiting. לשם כך נצטרך 2 פעולות:

1. Block: לוקחים את התהליך שמחכה לתוך תור waiting queue.

2. Wakeup: מעירים את התהליך ומבטלים את הBLOCK.

כעת בהתאמה, נשנה את $wait()$, $signal()$:

- ב $signal()$ (שחרור משאב): מגדילים את ערך הסמפור כמו קודם. אם הערך

של value הפך לשלילי או אפס, זה גורר כי יש עוד תהליכים שמחכים

בתור למשאב שהרגע שוחרר. לכן, מוציאים תהליכים מהתור ומעירים

אותו – מבטלים את הBLOCK

- Wait (לקיחת משאב): מקטינים את הערך. אם הערך שלו הופך לשלילי

ממש, זה אומר שלא נותרו משאבים פנויים ולכן הוא נכנס לתור ועובר

למצב חסום – Block. אם הערך היה חיובי הוא היה ממשיך בדרכו.

- ניתן לממש את הסמפור בצורה של FIFO (ואז מונעים הרעבה – מי

שמחכה ונכנס הוא מי שמחכה הכי הרבה זמן), וגם בצורה של LIFO

(מחסנית) – כאן יש הרעבה.

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}

signal(S) {
    S++;
}
```

indivisible

```
typedef struct{
    int value;

    struct process *list;
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block(); // מעבר למצב waiting
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value >= 0) {
        remove a process P from S->list;
        wakeup(P); // חזרה למצב ready
    }
}
```

בעיות קלאסיות של סינכרוניזציה

הערה: נפתור את כל הבעיות באמצעות סמפור. **דודי יותר מרמז שבמבחן הוא יבקש לפתור בדרכים אחרות.**

The Bounded-Buffer Problem

בעיית היצרן-צרכן. נתון: buffer בגודל n מקומות ושני תהליכים: אחד כותב לתוך buffer ואחד קורא ממנו.

נשתמש בסמפור מסוג mutex על מנת להגן על buffer מגישה בו זמנית (נאחל את הערך שלו לאחד).

סיכום מערכות הפעלה | © גיא יער-און

וכן, נשתמש בcounting semaphores על מנת לייצג את מס' המקומות הריקים והמלאים שישנם בבאפר:

```
do {
    ...
    // produce an item in nextp
    ...

    wait(empty);
    wait(mutex);

    // add nextp to buffer

    signal(mutex);
    signal(full);

} while (TRUE);

do {
    wait(full);
    wait(mutex);

    // remove an item from buffer
    // to nextc

    signal(mutex);
    signal(empty);

    // consume the item in nextc

} while (TRUE);
```

1. אחד יהיה empty – ייצג את מס' המקומות הריקים ויאוחלל לח.
 2. Full – ייצג את מס' המקומות שמולאו ויאוחלל ל0.
- היצרן, משמאל: בתחילה הוא תופס מקום ריק, ונועל את הבאפר wait(mutex) על מנת לעבוד לבד ושהצרכן לא יגע בו. לאחר מכן, מתבצעת הכתיבה לבאפר. בסיום: משחרר את הנעילה של הבאפר, וכן מגדיל את הערך של full (נוסף מקום אחד).

הצרכן, מימין: ראשית הוא בודק אם יש פריט - אם הבאפר ריק (full=0) הוא נחסם ומחכה. שהרי הוא צרכן ולא יכול לקחת פריטים אם אין כאלו. בהמשך, הוא נכנס ונועל את הבאפר על מנת לקרוא ממנו לבד ללא הצרכן. הוא קורא מהבאפר. לאחר מכן - משחרר את נעילת הבאפר, ומעדכן שעכשיו יש מקום אחד פנוי נוסף (מעדכן את empty): מקטין אותו באחד. נבהיר – הוא קודם כל בודק אם יש מקום / אין מקום ורק אז נועלים את הבאפר.

The Readers-Writers Problem

נניח שיש לנו דטה בייס ותהליכים שיש להם גישה אליו. חלק מהתהליכים רק קוראים מהדטה בייס – יקראו כreaders, וכן אלו שכותבים וגם קוראים מהדטה בייס יקראו כwriters. אין כל בעיה במצב שבו יותר readers אחד ניגש לDB.

מה הבעיה? אם writer ותהליך נוסף (כלשהו, קורא או כותב) ניגשים במקביל לDB עלול להיווצר כאוס. נבחין כי עלינו שלא לגרום לעיכוב תהליכים שלא לצורך.

נחזיר במשתנים משותפים: סמפור בשם mutex: משותף לreaders וכן נחזיר מנעול wrt מסוג סמפור שיהיה משותף לכולם – גם לreaders and writers. שניהם יאוחללו ל1. וכן, נחזיר משתנה שלם בשם readcount שמשותף לכל readers ויאוחלל ל0. (יספור כמה readers

קוראים כעת).

הקורא, מימין: הקורא ראשית מקדם את readcount – אם זה הקורא הראשון הוא נועל את בסיס הנתונים בפני הכותבים שלא יוכלו לכתוב. שאר הקוראים שיבואו אחריו לא יבצעו נעילה, שכן הוא כבר בוצע וערך המשתנה גדול ממש מאחד. הקוראים מבצעים קריאה, ובסיום: הקורא מעדכן את readcount שירד באחד ובדומה. אם זה הקורא האחרון – הוא משחרר את בסיס הנתונים מה שיאפשר לכותבים לכתוב בו.

הכותב, משמאל: מנסה להשתמש במשאב הכתיבה – אם כבר יש קוראים או כותבים בפנים הוא יחסם. ביציאה – משחרר את הנעילה ומאפשר לאחרים להכנס.

```
Writers' Code
do {
    wait(wrt);

    ...
    // writing is performed
    ...

    signal(wrt);

} while(TRUE);

Readers' Code
do {
    wait(mutex);
    readcount++;
    if (readcount == 1)
        wait(wrt);
    signal(mutex);

    ...
    // reading is performed
    ...

    wait(mutex);
    readcount--;
    if (readcount == 0)
        signal(wrt);
    signal(mutex);

} while(TRUE);
```

```

do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);

    ...
    // eat
    ...

    signal(chopstick[i]);
    signal(chopstick[(i+1)%5]);

    ...
    // think
    ...

}while (TRUE);

```

הבעיה מוגדרת כך: חמישה פילוסופים מעבירים את חייהם בישיבה סביב שולחן כשהם אחד בשני מצבים: הוגים או אוכלים. לא ישנים! השולחן מסודר כך שבמרכזו קערת אוכל וסביבו 5 מקלות אכילה (בשביל לאכול צריך 2 מקלות). כאשר פילוסוף נהיה רעב, הוא מנסה להרים מהשולחן את שני המקלות הקרובים ביותר אליו. לא ניתן להרים שני מקלות בו זמנית – רק אחד בכל פעם. וכן: לא ניתן לקחת מקל אם הוא נמצא בידי פילוסוף סמוך. בעת שפילוסוף שבע, הוא מניח את המקלות על השולחן וחוזר להגות. כיצד נפתור זאת?

ניצור מערך chopstick מסוג סמפור בגודל 5 – שמשותף לכל התהליכים (פילוסופים). בהתחלה הוא יאותחל ל 1 כל איבריו. בעת שצ'ופסטיק נתפס – הוא תופס (wait) את הצ'ופסטיק שמצא ואת האחד אחריו (במעגל). הוא אוכל – ובעת שמסיים: הוא מבצע signal על שני הצ'ופסטיקים.

הרצאה 7

מהו Deadlock? מילונית – מדובר בשתי פעולות מתחרות שמחכות כל אחת לסיומה של האחרת, ומכיוון שכך אף אחת לא מסיימת. למשל – שני אנשים שעומדים לפתח הדלת ונותנים לשני את הזכות להיות ראשון להכנס. בפועל: אף לא אחד מהם יכנס לעולם.

הגדרה: deadlock הוא מצב בו קיים סט של תהליכים שכל אחד מהם מחזיק משאב וממתין לקבל משאב המוחזק על-ידי תהליך אחר בסט על-מנת להמשיך בפעולתו, כל אחד מהתהליכים נמצא במצב blocked.

- מה הקשר לstarvation? אין קשר בכלל. מדובר בשני אירועים נפרדים. הרעבה לבסוף נפתרת – בעוד דדלוק ייפתר רק באמצעות עזרה חיצונית.

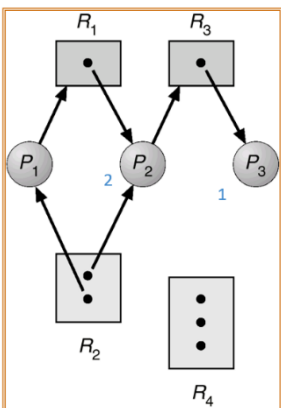
מודל מערכת

המערכת מורכבת מתהליכים ומשאבים – משאבים יכולים להיות ניתנים לשימוש חוזר, וכאלו שניתן להשתמש בהם פעם אחת בלבד. כל המשאבים מסוג מסוים מאוחדים תחת Resource type. כעת ישנם משאבים $R_1, \dots, R_m$ כך שלכל סוג משאב R_i ישנם W_i מופעים (יחידות מאותו משאב)

המודל יניח כי השימוש במשאבים הוא לפי Request, Use, Release: כלומר מבקשים משאב – משתמשים בו – ומשחררים אותו. מדוע ההנחה הזו חשובה? כי כשימשהו ישתמש במשאב ויסיים – נדע שהוא יחזיר בוודאות ולא "ישמור אצלו".

נגדיר כעת את התנאים הבאים שצריכים להתקיים בו זמנית, ומהווים תנאי הכרחי לקיום deadlock (אך גם אם הם יקרו, זה לא גורר שקרה דדלוק – אלא תנאי מקדים לדדלוק):

- Mutual Exclusion: רק תהליך אחד יכול להשתמש בו זמנית במופע אחד של משאב.
- Hold and wait: קיים לפחות תהליך אחד שמחזיק משאב וממתין למשאב (אחרת אין דדלוק).
- No Preemption: תהליך שתפס משאב כלשהו ישחרר אותו רק באופן התנדבותי – לא יכפו עליו את זה.
- Circular wait: ניתן לזהות תת קבוצה של תהליכים מתוך כלל התהליכים במערכת – שכל אחד מהם הוא Hold and wait וכן P_0 מחזיק משאב וממתין למשאב שמוחזק ע"י P_1 וכו', עד ש P_m מחזיק משאב וממתין למשאב שמחזיק P_0 . כלומר: ישנו מעגל שכל אחד תלוי במשאב שמחזיק מישהו אחר.



Resource-Allocation Graph

נגדיר את מודל המערכת כגרף תהליכים ו $Resource\ types$. הגרף יהיה גרף מכוון.

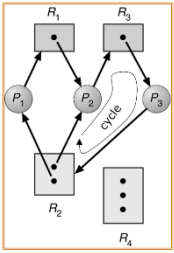
נגדיר את קבוצת הקודקודים V להיות: אוסף התהליכים $P = \{P_1, \dots, P_n\}$ ו $r = \{r_1, \dots, r_m\}$ להיות קבוצת כל $resource\ types$.

הקשתות E יוגדרו כך: קבוצה ראשונה בין תהליך למשאב (P_i, R_j) וקבוצה שנייה ממשאב לתהליך (R_i, P_j) .

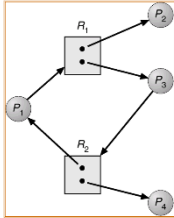
משמאל: דוגמה לגרף כנ"ל. מס' העיגולים בתוך כל R_i הוא מס' המופעים של אותו משאב. קל לראות שבדוגמה משמאל אין דדלוק כי אין מעגל!

הגרף למעשה מתאר איזה משאב דורש כל תהליך, ואיזה תהליך שולט על כל משאב.

סיכום מערכות הפעלה | © גיא יער-און



נתבונן בדוגמה נוספת. כאן קיבלנו מעגל. האם זה גורר קיום דדלוק? במקרה שלנו כן! P1 לא יכול להתקדם כי ממתין לP2 שנמצא במעגל ותלוי בשאר התהליכים ולכן כאן נקבל דדלוק.



לעומת זאת: כאן ישנו מעגל אך מתי שהוא תהליך P4 ישחרר את המשאבים שלו (כי זה המודל שלנו) ולכן נמנע ממצב של דדלוק.

טענה:

1. אם בגרף אין מעגלים $\Rightarrow$ אין דדלוק.
 2. אם בגרף כן יש מעגל:
- אם ישנו משאב אחד לכל Resource type, ישנו דדלוק בהכרח.
 - אחרת: אם יש מס' מופעים לכל Resource type: ייתכן שיש דדלוק – עלינו לבדוק זאת.

התמודדות עם Deadlock

ישנן כמה גישות:

- מניעה: נבטיח שהמערכת לעולם לא תכנס למצב של דדלוק. נבחין שבמצב זה אנחנו נוריד את הנצילות שכן לפעמים יהיה לנו את המשאבים שצריך תהליך על מנת להתקדם, אך לא ניתן לו אותם מהחשש שזה יוביל לדדלוק.
- התאוששות - לא נמנע מהמערכת להכנס למצב דדלוק, אך כשנזהה מצב כזה נתאושש ממנו.
- התעלמות - נתעלם מהאפשרות שהמערכת עלולה להכנס למצב של דדלוק.

רוב המערכות משתמשות בשיטת ההתעלמות. מדוע? הכי קל לממש, וכן בשיטות האחרות יש לפתח מנגנון זה ידרוש משאבים וכן בגישת המניעה למשל – הנצילות יורדת משמעותית. אנחנו נרחיב על המניעה:

גישה ראשונה (מניעה) – לוותר על חלק מהדרישות

- ניתן לוותר על Mutual Exclusion – אם כי רוב המשאבים לא מאפשרים לוותר על זה.
- ניתן לוותר על Hold and wait – למשל: לפני שתהליך מתחיל לרוץ הוא צריך לוודא שיש לו את כל המשאבים שהוא צריך, ורק אז הוא יקצה אותם. בנוסף, אפשר לאפשר לתהליך לבקש משאבים רק כאשר הם לא מחזיקים שום משאב ביד.

מה הבעיה בגישות אלו? זה מביא לניצולת נמוכה של המשאבים שלנו, ואף עלולה לגרום לstarvation.

- ניתן לוותר על No preemption: אם תהליך שמחזיק חלק מהמשאבים אך אין לו את כולם על מנת לבצע מה שהוא צריך לעשות, נבצע preemption ונקח לו בכוח את המשאבים. כתוצאה מכך – הוא יצטרך בהמשך לבקש את המשאבים שוב.
- גם על המעגל ניתן לוותר – ניתן לתת מס' סידורי לכל Resource type. ואז: כל תהליך יכול לקבל משאבים רק בסדר עולה. למשל אם תהליך מחזיק משאב מס' 2 ומס' 5 הוא יכול לבקש משאבים רק ממס' 6 ומעלה. מדוע זה מבטיח שלא יהיה מעגל? נניח בשלילה שישנו מעגל. אזי בהכרח יצרנו רצף כך שהאיבר האחרון בו הוא מסדר x. הוא לא יכול להתחבר לאף אחד מהאיברים במסלולו כי הוא בהכרח גדול מכולם. לכן לא יהיה מעגל.

גישה שנייה (מניעה)

סיכום מערכות הפעלה | © גיא יער-און

היא לא לנסות לוותר על אחת הדרישות שלנו, אלא לשמור על המערכת מפני כניסה למצב של דדלוק באמצעות אלגוריתמים ייעודיים לכך. זה מחייב כי למערכת ישנו מידע נוסף על התהליכים שרצים. כל תהליך יודע לומר מה גודל הזיכרון המקסימלי שיצטרך ברגע נתון.

ניתן לפתח אלגוריתם שיכול להימנע באופן רציף ממצבים מסוכנים של דדלוק. הוא יבדוק באופן דינמי את resource-allocation state (מיד נרחיב) וידאג שלעולם לא יהיה מעגל.

resource-allocation state: מוגדר באמצעות 3 פרמטרים –

1. המספר של המשאבים הזמינים.
2. המספר של המשאבים המוקצים.
3. המספר המקסימלי של דרישות של תהליך מסוים.

כאשר תהליך מבקש משהו זמין, המערכת (אלגוריתם) תצטרך להחליט אם הקצאה מיידית של אותם משאבים לתהליך ישמרו את המערכת עדיין במצב safe state. מה הכוונה? **מערכת נמצאת במצב safe state** אם קיים רצף של כל התהליכים במערכת $\langle P_1, \dots, P_n \rangle$, כך שלכל P_i כמות המשאבים שהוא יכול לבקש על מנת להתקדם – ניתנים לסיפוק ע"י כל המשאבים הנוכחיים יחד עם כל המשאבים שיש ל- P_j כך ש- $j < i$. מה בעצם דרשנו? קיים רצף כנ"ל כך שתמיד ניתן לספק כל איבר ברצף ע"י המשאבים הקיימים יחד עם כמות המשאבים של קודמיו ברצף. נבחין כי במצב זה נוכל להריץ את התהליכים לפי הרצף הנ"ל ואז בוודאות כל איבר ברצף יוכל להסתיים – ולא יהיה דדלוק בוודאות.

- Assume 12 units available:

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

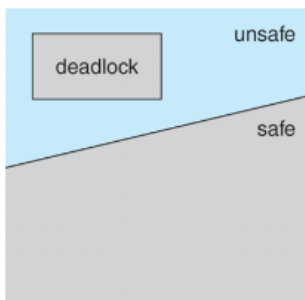
לדוגמה – נניח שישנן 12 יחידות זמינות. לכל תהליך יש מס' יחידות שהוא דורש. נבחין כי סכום הcurrent needs צריך להיות לכל היותר 12 (וכך גם אצלנו).

בתחילה מס' המשאבים הזמינים הוא 3 ($3=12-5-2-2$). נבחין כי ל- P_1 ישנם סה"כ עוד 2 יחידות שהוא צריך. לכן אם ניתן לו את היחידות שהוא צריך – כאשר הוא יסיים יישאר בידיו סה"כ 5 יחידות (כולל היחידה שקודם לא השתמשנו בה). ואז נוכל לספק את P_0 – שדורש עוד 5. כשהוא יסיים – יהיה לנו 10 יחידות לפחות, ונוכל לספק את P_2 .

- Assume P_2 was allocated an additional unit:

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	3

לעומת זאת – נתבונן בדוגמה השנייה. נניח כעת כי P_2 מראש קיבל 3 יחידות. כעת, נותרו 2 יחידות במערכת (יש סה"כ 12). כעת P_1 ירוץ ראשון – ויסיים, ואז יישארו לנו 4 יחידות. נבחין כי P_0 דורש 5 ו- P_2 דורש 6: ולנו ביד יש 4. לכן אנחנו לא יכולים כעת להקצות לאף אחד יחידות. זה עדיין לא גורר דדלוק – אך אכן מסוכן לכיוון דדלוק.

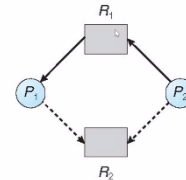


טענה:

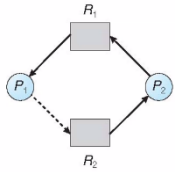
- אם מערכת נמצאת במצב safe – בהכרח אין דדלוק.
- אם מערכת נמצאת במצב unsafe – ייתכן ויהיה דדלוק.
- אנחנו למעשה נותנים הבטחה שהמערכת לעולם לא תכנס לunsafe state.

כעת נרצה לפתח אלגוריתם שיבטיח שאנחנו תמיד במצב safe. נחזור לגרף מקודם – ונשדרג אותו. נוסיף Claim edge: תמיד ילך מ- P_i (תהליך) ל- R_j (משאב) – זה לא אומר שהתהליך רוצה את המשאב. אלא: התהליך אולי בהמשך ירצה את המשאב. נייצג את הקשת הזו בחץ מקוקו. בעת שתהליך ירצה את המשאב (וזה לא יהיה רק אפשרות לבקשה עתידית) – הקשת הזו תהפוך לקשת רגילה. בעת שהמשאב יינתן, כיוון החץ ישתנה (ממשאב לתהליך, המשאב בידי התהליך). בעת שהמשאב יילקח – הקשת תחזור להיות קשת מקווקו ותחזור חלילה.

לדוגמה נביט על הגרף הנ"ל.



נניח כי כעת הדרישה של P_2 ל R_2 אכן התגשמה (הפכה מאפשרות לבקשה אמיתית לבקשה בפועל). אזי במצב זה נקבל את הגרף הבא – המשאב R_2 תפוס כעת ע"י התהליך P_2 .



נבחין כי למעשה בעת המעבר יצרנו מעגל. לכן לפני שנענה לכל דרישה נרצה לבדוק אם מקבלים מעגל בגרף: ואם כן – לא נענה על הדרישה.

לסיכום – מדובר באלגוריתם פשוט להימנעות: בגישה הזו המערכת לא פוסלת מראש שום התנהגות ולא קובעת חוקים נוקשים על סדר הבקשות. במקום זה היא נותנת לתהליכים לבקש מה שבא להם – אך בזמן אמת האלגוריתם מריץ בדיקה דינמית על הגרף: הבדיקה הופכת את הקשת המקווקות לקשת בקשה רגילה ובודקת – זה יוצר מעגל בגרף? אם כן: היא לא מאשרת את הבקשה ואם לא נוצר מעגל היא כן מאשרת.

Banker's Algorithm (גישה ההימנעות – יותר ממופץ אחד לכל Resource type)

מה קורה כאשר ישנם כמה instances מכל Resource type? אם היה מופץ אחד מכל משאב, היינו מריצים את הגרף שתיארנו קודם ומחפשים מעגל. אחרת, אנחנו נשתמש באלגוריתם של בנקר. הוא מניח מס' הנחות:

1. כל תהליך צריך להודיע מראש על מס' היחידות המקסימלי (max demand) שישתמש בהם.
2. יתכן שתהליך יצטרך לחכות לאחר שביקש.
3. לאחר שתהליך קיבל את המשאבים שהוקצו לו הוא חייב להחזיר אותם.

נסמן בח את מס' התהליכים ובח את מס' Resource typen. נגדיר:

1. Available – מערך מגודל m כך שאם $Available[i]=k$ זה גורר שיש k מופעים ממשאב i.
2. Max – מטריצה מגודל $m \times n$ כך שאם $Max[i, j]=k$ אזי תהליך P_j ידרוש לכל היותר k יחידות ממשאב R_j .
3. Allocation – מטריצה מגודל $m \times n$ כך שאם $Allocation[i, j]=k$ אזי תהליך P_j כרגע משתמש באיחידות ממשאב R_j .
4. Need – מטריצה מגודל $m \times n$ כך שאם $Allocation[i, j]=k$ אזי תהליך P_j צריך עוד k יחידות ממשאב R_j על מנת לסיים את משימתו.

סיכום מערכות הפעלה | © גיא יער-און

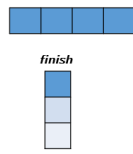
נגדיר את ה-Safety Algorithm כך שיבדוק אם המערכת נמצאת במצב safe:

1. Let $Work$ and $Finish$ be vectors of length m and n , respectively.

Initialize:

$Work = Available$

$Finish[i] = false$ for $i=1,2,3, \dots, n$.



2. Find i such that both:

(a) $Finish[i] = false$

(b) $Need_i \leq Work$

If no such i exists, go to step 4.

3. $Work = Work + Allocation_i$

$Finish[i] = true$

go to step 2.

4. If $Finish[i] == true$ for all i , then the system is in a safe state.

בתחילה המערכת מייצגת שני מבני נתונים – $Work$ הוא וקטור באורך m של מס' סוגי המשאבים הפנויים בכל רגע. הוא מתחיל מהמבצב הנתון, $Work=Available$. וכן: וקטור בוליאני בשם $Finish$ שמאותחל לערכי שקר – המשמעות: התהליך עדיין לא סיים את ריצתו בסימולציה.

בשלב השני המערכת סורקת את התהליכים ומחפשת תהליך שיקיים כי הוא עדיין לא סיים לרוץ וכן מס' המשאבים שהוא עוד צריך על מנת לסיים את משימתו קטן שווה מכמות המשאבים שישנם בסימולציה (מאותו משאב). אם לא מצא כזה הוא עובר לשלב 4.

בשלב השלישי אם מצאנו בשלב הקודם תהליך כנ"ל, המערכת מעדכנת שהוא סיים את משימתו והוא מחזיר למערכת (זה ה+) את המשאבים שהיו לו. מכאן המערכת חוזרת לשלב 2.

בשלב האחרון אם כל התהליכים סיימו, מצאנו סדר כרונולוגי בו כל התהליכים סיימו בבטחה ולכן המערכת נמצאת ב-safe state.

דוגמה:

- 5 processes P_0 through P_4 ; 3 resource types A (10 instances), B (5 instances), and C (7 instances).

- Snapshot at time T_0 :

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

10,5,7

נניח מערכת שמכילה 5 תהליכים, ו3 סוגי תהליכים שלכל אחד מס' מופעים שונה (10 5 7) בהתאמה). ב-ALLOCATION נוכל לראות למעשה את המטריצה מגודל 5×3 – שמתארת כמה משאבים מוקצים כרגע לכל תהליך. בנוסף, ישנה מטריצת MAX שהיא הצהרת הכוונות המקורית של התהליכים – מה הכמות המקסימלית שיצטרך תהליך בנק' זמן כלשהי בשביל לסיים.

נבחין כי אם נסכם את כל המשאבים שתפוסים כרגע ע"י התהליכים (סכום עמודות Allocation) ונוסיף להם את מה שיש ב-Available אנחנו חייבים לקבל את סך המשאבים הכולל במערכת. אכן אם נסכום נקבל (10,5,7). נעיר כי למערך Available אין קשר לתהליך P_0 – מדובר במערך יחיד למערכת.

- The content of the matrix Need is defined to be $Max - Allocation$.

	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

	Allocation			Max		
	A	B	C	A	B	C
P_0	0	1	0	7	5	3
P_1	2	0	0	3	2	2
P_2	3	0	2	9	0	2
P_3	2	1	1	2	2	2
P_4	0	0	2	4	3	3

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria.

3,3,2 5,3,2 7,4,3 7,4,5 10,4,7 10,5,7

כעת נבחין כי need זה כמה נדרש לסיים את המשימה. לכן הנוסחה לכך היא למעשה:

$$Need = Max - allocation$$

שכן Max זה כמות היחידות המקסימלית שישתמש התהליך, וallocation זה כמה הוא משתמש כרגע. כעת נריץ את האלגוריתם –

בתחילה $Work$ מאותחל למערך (3,3,2) (המערך הראשוני של כמה פנויים יש). כעת נתבונן ברצף הבא –

1. הרצת P_1 : האם $Need(P_1) = (1,2,2) \leq (3,3,2)$? כן! לכן P_1 רץ ומסיים. הוא השתמש במשאבים (2,0,0) ולכן כעת מס' המשאבים הפנויים (5,3,2).

2. הרצת P_3 : האם $Need(P_3) = (0,1,1) \leq (5,3,2)$? כן. לכן P_3 רץ ומסיים (יכול להשתמש בכל המשאבים כעת). לכן הוא השתמש במשאבים (2,1,1) ולכן כעת $Work = (5,3,2) + (2,1,1) = (7,4,3)$

3. הרצת P_4 : האם $Need(P_4) = (4,3,1) \leq (7,4,3)$? כן! לכן P_4 רץ ומסיים. השתמש במשאבים (0,0,2) ולכן כעת $Work = (7,4,5)$

4. הרצת P_2 : האם $Need(P_2) = (6,0,0) \leq (7,4,5)$? כן! לכן P_2 רץ ומסיים. השתמש במשאבים (3,0,2) ולכן כעת $Work = (10,4,7)$

5. הרצת P_0 : האם $Need(P_0) = (7,4,3) \leq (10,4,7)$? כן! לכן P_0 רץ ומסיים. השתמש במשאבים (0,1,0) ולכן כעת $Work = (10,5,7)$

סיכום מערכות הפעלה | © גיא יער-און

הגענו למצב שבו כל התהליכים עם true, ולכן המערכת נמצאת כעת במצב safe.

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true.

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Allocation	Max
A B C	A B C
P_0	0 1 0
P_1	2 0 0
P_2	3 0 2
P_3	2 1 1
P_4	0 0 2

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.
- Can request for $(3,3,0)$ by P_4 be granted (next page)?
- Can request for $(0,2,0)$ by P_0 be granted?

כעת נראה דוגמה דינמית. נניח והגיע תהליך P_1 שמחליט לבקש משאבים באופן מידי. הוא שולח בקשה למשאבים $(1,0,2)$. נראה כי בקשתו מקיימת $Request \leq Need_i$ ולכן הוא לא ביקש יותר ממה שהצהיר מראש שחסר לו. בשלב השני, נבדוק אם $Request \leq Available$. זה קורה אצלנו ולכן שני התנאים עברו ולכן מערכת ההפעלה הולכת ומבצעת סימולציה זמנית. היא מעדכנת כי כעת מס' המשאבים של התהליך הנ"ל היא:

$$Allocation(P_1) = (2,0,0) + (1,0,2) = (3,0,2)$$

כמו כן מס' הנדרשים לסיום שלו:

$$Need(P_1) = (1,2,2) + (1,0,2) = (0,2,0)$$

וכן כעת מס' המשאבים שנותרו למערכת:

$$Available = (3,3,2) - (1,0,2) = (2,3,0)$$

כעת היא תריץ את safety algorithm על המצב החדש של הטבלה הנ"ל. אם, ורק אם נקבל כי המערכת במצב safe נאשר את הבקשה סופית ונעביר את המשאבים ל- P_1 .

שיטת ההתאוששות:

בניגוד לשיטות הקודמות – בנקר והגרף הקודם, בו מערכת ההפעלה התנהגה כמעין שוטר ועצרה תהליכים בזמן אמת כדי שלא נכנס למצב מסוכן, כאן מאפשרים לקבל דדלוק. מערכת ההפעלה מאפשרת דדלוק אך מפעילה שני מנגנונים ברקע – אחד הוא אלגוריתם זיהוי, אלגוריתם שרץ מדי פעם ברקע (למשל כל כמה דקות) ובדוק אם ישנו דדלוק ברגע זה במערכת. והשני – תוכנית התאוששות: אם אלגוריתם הזיהוי מצא דדלוק, המערכת תתערב חיצונית בכוח.

כעת נניח שוב כי ישנו רק מופע יחיד מכל משאב: הדרך הקלה ביותר לזיהוי דדלוק היא באמצעות תורת הגרפים.

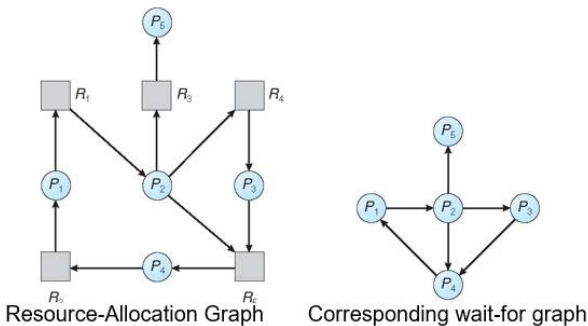
נגדיר את ה Wait for Graph להיות גרף שקודקודיו הם התהליכים והקשתות מוגדרות באופן הבא: קשת (P_i, P_j) אם "מ תהליך P_i ממתינ כרגע למשאב שמחזיק P_j .

במערכת ההפעלה רץ באופן מחזורי אלגוריתם פשוט שמחפש מעגלים על הגרף. נבחין: אם יש מעגל בגרף בהכרח ישנו דדלוק (כי שוב, זה מופע יחיד לכל משאב).

כאשר יש מס' מופעים מכל משאב:

נרצה תחילה להגדיר את אלגוריתם הזיהוי. נגדיר את מבני הנתונים הבאים -

1. Available – מערך (וקטור) באורך m. מצוין כמה מופעים פנויים יש כרגע ממשאב מסוים.
2. Allocation – מטריצה בגודל $m \times n$. מטריצה שמגדירה כמה מופעים מכל משאב תפוסים פיזית ע"י תהליך מסוים.
3. Request – מטריצה בגודל $m \times n$. Request[i][j]=k משמעותו כי תהליך P_j מבקש עוד k מופעים ממשאב R_i .



- Let **Work** and **Finish** be vectors of length m and n , respectively Initialize:
 - Work = Available**
 - For $i = 1, 2, \dots, n$, if **Allocation_i ≠ 0**, then **Finish[i] = false**; otherwise, **Finish[i] = true**
- Find an index i such that both:
 - Finish[i] == false**
 - Request_i ≤ Work**
 If no such i exists, go to step 4
- Work = Work + Allocation_i**
Finish[i] = true
 go to step 2
- If **Finish[i] == false**, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if **Finish[i] == false**, then P_i is deadlocked

כעת נציג את האלגוריתם.

בשלב הראשון מאתחלים את Work להיות מס' הפנויים כרגע מכל משאב. וכן, מאתחלים את Finish באופן הבא: אם לתהליך בכלל אין משאבים הוא יוגדר כtrue (זה אם Allocation=0) ואחרת יוגדר כfalse. מדוע? כי תהליך שלא מחזיק משאבים בכיס לא יכול לחסום אף אחד אחר.

בשלב השני מחפשים כמו קודם, אינדקס שעוד לא סיים ושכמה שהוא מבקש לא עולה על מס' הפנויים כרגע. אם לא מצאנו כזה נעבור לשלב 4.

בשלב השלישי בדיוק כמו קודם – מוסיפים כמה ששוחרר, ומעדכנים שהוא בוצע.

בשלב האחרון – אם ישנו תהליך שנותר עם Finish[i]=false אזי אנחנו נמצאים בדדלוק. מי חלק מהדדלוק? אותו תהליך.

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			
- Sequence $\langle P_0, P_2, P_3, P_4 \rangle$ will result in **Finish[i] = true** for all i

Example (Cont.)

- P_2 requests an additional instance of type C

	Request		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes' requests
 - Deadlock exists, consisting of processes P_1, P_2, P_3 , and P_4

דוגמה:

נניח מערכת עם 5 תהליכים שסכומם (7,2,6). בקופה המרכזית כעת אין כלום – (0,0,0). נרצה לבדוק אם יש דדלוק. מאתחלים בהתחלה את כל התהליכים לשקר, שכן יש להם משאבים. נריץ:

- הרצת P_0 : האם $\text{Request}(P_0)=(0,0,0) \leq \text{Work}=(0,0,0)$? כן! לכן הוא לא מבקש כלום כרגע ו $\text{Work}=(0,1,0)$.
- הרצת P_2 : האם $\text{Request}(P_2)=(0,0,0) \leq (0,1,0)$? כן לכן $\text{Work}=(3,1,3)$.
- הרצת P_3 : האם $\text{Request}(P_3)=(1,0,0) \leq (3,1,3)$? כן. לכן $\text{Work}=(5,2,4)$.

בדומה כך נמשיך. כולם הגיעו ל $\text{Finish}[i]=\text{true}$ ולכן המערכת נקייה מדדלוק.

כעת נראה בקשה דינמית. נניח שתהליך 2 מבקש יחידה נוספת ממשאב C. **מאשרים לו**. לכן מעדכנים את Request שלו. מריצים את האלגוריתם. מקבלים כי המערכת נמצאת בדדלוק.

Detection-Algorithm Usage

האלגוריתם לוקח זמן להרצה. מדובר במעבר על מטריצות וחיפוש מעגלים – זו פעולה שמבזבזת זמן מעבד וגורמת ל overhead . ישנם שני שוקלים שקובעים מתי נריץ את האלגוריתם:

- באיזה תדירות דדלוק צפוי להתרחש במערכת? אם המערכת יציבה מאוד ודדלוק הוא אירוע נדיר – אין הגיון להריץ את המערכת כל 5 שניות. אם המערכת עמוסה בתהליכים שמתחרים על אותם משאבים נפעיל יותר את האלגוריתם.
- כמה תהליכים נצטרך לגלג אחורה או להרוג?

אם נריץ את האלגוריתם לעיתים קרובות: נזהה את הדדלוק מיד כשהוא נוצר. היתרון הגדול הוא שנדע בדיוק מי התהליך הספציפי שביקש משאב ו"הצית" את הדדלוק, ויהיה קל מאוד להתאושש (Recovery) על ידי פגיעה מינימלית בתהליך אחד או שניים.

סיכום מערכות הפעלה | © גיא יער-און

אם נריץ את האלגוריתם לעיתים רחוקות (למשל, פעם בשעה): במהלך השעה הזו, תהליכים נוספים יבקשו משאבים וייחסמו, וטבעת הדדלוק תלך ותגדל. כשהאלגוריתם סוף סוף ירוץ, אנחנו נגלה טבעת דדלוק ענקית ומסובכת שמורכבת מהמון מעגלים זרים/נפרדים). במצב כזה, נצטרך לבצע הפלה או גלגול לאחור של המון תהליכים בו-זמנית, והמחיר של ההתאוששות יהיה כבד ויקר מאוד.

Recovery from Deadlock: Process Termination

נניח והרצנו את האלגוריתם לזיהוי ונמצא דדלוק. כיצד מתאוששים מכך?

האסטרטגיה האגרסיבית ביותר היא חיסול תהליכים. המטרה פשוטה: להרוג תהליך אחד או יותר על מנת לשבור את המעגל החסום ולשחרר את המשאבים שלו חזרה למערכת. מערכת ההפעלה יכולה לבחור באחת משתי דרכים כדי לנקות את הדדלוק.

1. להרוג את כל התהליכים התקועים – מנקים את השולחן לחלוטין, כל תהליך שהאלגוריתם הקודם סימן עליו false מקבל kill מיידית. החיסרון – זה יקר מאוד, אם היו שם תהליכים שרצו שעות וחישבו חישובים מורכבים כל העבודה שלהם נזרקה לפח.
2. להרוג תהליך אחד כל פעם – בוחרים תהליך קורבן אחד, הורגים אותו, ואז מריצים מחדש את האלגוריתם, אם המערכת עדיין בדדלוק – בוחרים קורבן נוסף הורגים אותו וחוזר חלילה. החיסרון הגדול: יוצר overhead מטורף. המערכת צריכה להריץ את אלגוריתם הזיהוי שוב ושוב.

אם נלך על הגישה השנייה – ישנם שיקולים בבחירת הקורבן. ישנם 6 קריטריונים שמערכת ההפעלה עובדת לפיהם:

1. עדיפות התהליך – נרצה להרוג תהליך עם עדיפות נמוכה (של משתמש) ולא תהליך מערכת.
 2. זמן ריצה נוכחי וזמן לסיום: נעדיף להרוג תהליך שהתחיל לרוץ לא מזמן ונשאר לו קצת זמן לסיום.
 3. משאבים שהתהליך כבר ניצל: אם תהליך מחזיק המון משאבים ביד, שווה להרוג אותו כי החיסול שלו ישחרר שלל גדול לקופה ויפתור כנראה את הבעיה להרבה תהליכים.
 4. כמה תהליכים יושפעו מהחיסול
 5. סוג התהליך – אם זה תהליך אינטראקטיבי (ממשק משתמש) או תהליך Batch (ריצה ארוכה ברקע) - נעדיף לחסל Batch.
 6. משאבים שהתהליך עוד צריך בכדי לסיים – אם תהליך דורש עוד הרבה משאבים עד לסיומו, הוא מועמד פוטנציאלי לחיסול.
- נשים לב – ישנה סכנה של starvation: אם מערכת ההפעלה תסתכל רק על העלות החישובית היבשה (המדדים שלמעלה), עלול להיווצר מצב שבו **אותו תהליך ייבחר תמיד כקורבן!** (למשל, כי הוא תמיד בעל העדיפות הנמוכה ביותר). הוא יתחיל לרוץ, ייכנס לדדלוק, המערכת תהרוג אותו, הוא יתחיל שוב, ושוב ייהרג – הוא לא יסיים לעולם. **הפתרון:** להכניס את מספר הפעמים שהתהליך כבר חטף גלגול לאחור/חיסול כפקטור בתוך נוסחת המחי. (ככל שתהליך "סבל" יותר פעמים בעבר, ככה המחיר הווירטואלי שלו יעלה, והמערכת תימנע מלבחור בו שוב כקורבן.

הגישה הכללית	הרעיון בקצרה	מה עושים במופע יחיד?	מה עושים במספר מופעים?
מניעה והימנעות <i>(הבטחה שלא ניכנס לדדלוק)</i>	לא מאפשרים לדדלוק לקרות לעולם. עוצרים או מקפיאים תהליכים ליתר ביטחון (מוריד נצילות).	מניעה: ביטול תנאים (סדר נומרי). הימנעות: גרף RAG (בדיקת מעגלים עם קשתות דרישה).	מניעה: חוקים גורפים על המערכת. הימנעות: אלגוריתם הבנקר
זיהוי והתאוששות <i>(טיפול בדיעבד)</i>	משחררים רסן ומאשרים בקשות חופשי. אם קורה דדלוק – מזהים אותו ומפעילים מנגנון ריפוי בכוח.	זיהוי: גרף Wait-for מצומצם (חיפוש מעגלים ישירים בין תהליכים).	זיהוי: אלגוריתם מטריציוני לפי בקשות בפועל
התעלמות	מתעלמים לחלוטין מהבעיה. מניחים שדדלוק קורה לעיתים נדירות מדי מכדי להצדיק פגיעה בביצועים.	כלום. אם המערכת קופאת, המשתמש או מנהל המערכת יצטרכו לעשות Restart ידני.	כלום. הגישה הכי נפוצה במערכות הפעלה מסחריות (כמו Windows ו-Linux) בגלל אפס Overhead.

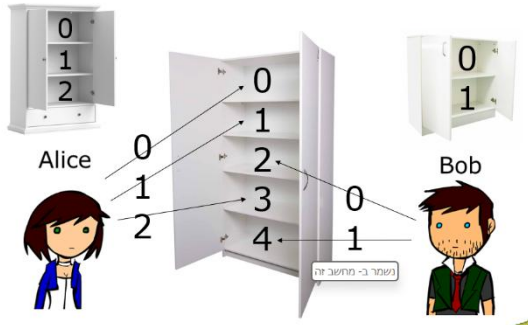
הרצאה 9

הערה. הרצאה 8 הייתה הרצאת חזרה – ולכן לא מסוכמת.

מה החשיבות של הזיכרון הראשי? הזיכרון הראשי, הרגיסטרים והקאש הם אמצעי האחסון היחידים אליהם יכול לגשת המעבד.

בשביל שנוכל להריץ תוכנית שמאוחסנת בדיסק – עלינו ליצור ממנה תהליך בזיכרון.

נסביר את ההבדל בין מרחב כתובות לוגי למרחב כתובות פיזי באמצעות מטאפורה על ארונות.



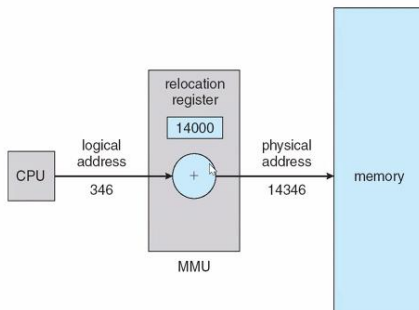
- מרחב כתובות לוגי: מיוצג ע"י הארונות הקטנים של אליס ובוב. לכל אחד מהם יש תפיסה עצמאית של כתובות (מ0 ועד למספר כלשהו), כאילו הם הבעלים של הזיכרון. כתובת לוגית זו כתובת שנוצרת ע"י הCPU בזמן שהתוכנית רצה. התוכנית עצמה חושבת שהיא ניגשת לזיכרון בזמן מסוים, אך היא לא מודעת למיקום האמיתי בזיכרון הפיזי.
- מרחב כתובות פיזי: מיוצג ע"י הארון המרכזי הגדול. הכתובות הלוגיות של אליס ובוב ממופות למיקומים שונים בארון הפיזי המשותף. זו הכתובת המרכזית שנשלחת ליחידת הזיכרון (RAM). זהו המקום הפיזי בו המידע מאוחסן בשבבי הזיכרון.

משמעות הדבר – המערכת מאפשרת לכל תהליך לעבוד עם מרחב כתובות לוגי משלו, בעוד מערכת ההפעלה דואגת לתרגם אותם למקומות פיזיים שונים בזיכרון. מדוע זה חשוב? היכולת להפריד בין מה שהCPU מייצר (לוגי) לבין מה שהזיכרון מקבל (פיזי) הוא לב ליבו של ניהול הזיכרון במערכות הפעלה מודרניות. זה מאפשר הגנה (אף אחד לא יכול לגשת לזיכרון של תהליך אחר בטעות, כי הכתובות הלוגיות שלהם ממופות לאזורים פיזיים שונים), גמישות (מערכת ההפעלה יכולה להעביר תוכניות בזיכרון הפיזי מבלי שהתוכנית תצטרך לדעת על כך).

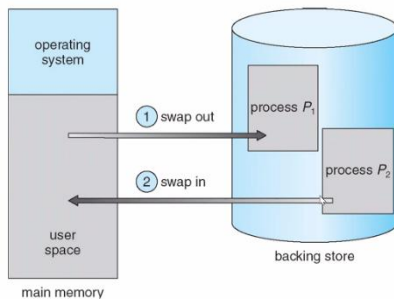
- בעת שאנחנו כותבים תוכנית, אנחנו כותבים אותה לפי כתובות לוגיות שכן לא נדע היכן התהליך ישב בזמן שנריץ את התהליך
- שיצא מהProgram! צימוד ההוראות והדטה לכתובות זיכרון הוא תהליך שנקרא Binding ויכול לקרות בכל אחד משלושת השלבים הבאים –
- בזמן קימפול: הקומפיילר יודע מראש היכן התוכנית תרוץ בזיכרון. נוצר קוד אבסולוטי בזיכרון (קוד עם כתובות קבועות). החיסרון – אם נרצה להזיז את התוכנית לכתובת אחרת בזיכרון, נהיה חייבים לקמפל אותה מחדש.
- בזמן Loading (טעינה) – הקומפיילר לא יודע היכן התוכנית תרוץ. לכן הוא מייצר Relocatable code (קוד שניתן להזיז). הקישור לכתובות הפיזיות מתבצע ברגע שבו המערכת טוענת את התוכנית לזיכרון (כאשר מריצים אותה). אם נשנה את המיקום – נצטרך להטעין את התוכנית מחדש. משמעות הדבר שכל פעם שנעשה LOADING לPROGRAM שלנו ונהפוך אותה לPROCESS נסתכל בזיכרון מה פנוי – ונקבע בהתאם. זה שיפור ביחס לBINDING בזמן קימפול – שכן יש יותר גמישות ביחס לקומפילציה (לא קובעים לכל החיים איפה תשב התוכנית, אלא רק בהרצה הקרובה).
- בזמן הריצה – המצב הנפוץ ביותר במערכות הפעלה מודרניות. הקישור לכתובות קורה תוך כדי שהתוכנית רצה. זה מאפשר לתוכנית לעבור ממקום פיזי אחד למשנהו בזיכרון במהלך חייה, מבלי שהיא תרגיש בכך. זה מאפשר גמישות מקסימלית לנצילות הזיכרון.

כאשר עובדים בקומפילציה או בזמן טעינה – הכתובות שנוצרות בזמן הריצה הן כבר כתובות פיזיות, לעומת זאת בזמן ריצה אלו כתובות לוגיות. כיצד זה מתבצע?

מדובר ברכיב חומרה שממפה בין כתובת לוגית לכתובת פיזית. המימוש הפשוט ביותר (מתאים למצבים בהם התהליך שמור בסגמנט רציף ב-RAM) הוא באמצעות רגיסטר בשם – Relocation register: הערך שנמצא בתוכו הוא הערך של תחילת המיקום הפיזי בזיכרון. בכל שלב שבו ה-CPU מנסה לגשת לכתובת מסוימת, ה-MMU לוקח את הכתובת הלוגית ומוסיף לה את הכתובת שברגיסטר.



למשל: סיפרנו ל-MMU שהתהליך שלנו יושב בזיכרון מכתובת פיזית 14000. מה הוא יעשה? בכל פעם שייקח כתובת לוגית – למשל 346, הוא יוסיף את הערך ששמרנו ברגיסטר – ויקבל את הכתובת הפיזית – 14346.

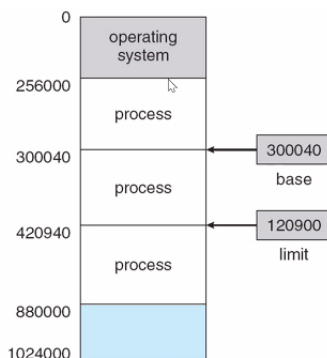
Swapping

כאשר המערכת צריכה להריץ תהליכים שתופסים יותר מקום ממה שיש פיזית ב-RAM היא משתמשת בטכניקה של העברת תהליכים מהזיכרון המהיר לאחסון משני כמו דיסק.

- **Backing store:** דיסק שבו נשמרים התהליכים שהוצאו מהזיכרון. הוא חייב להיות מהיר מספיק על מנת לא להאט את המערכת יותר מדי, וגדול מספיק על מנת להכיל תמונות זיכרון של כל התהליכים.
- **Roll out/Roll in:** וריאציה של החלפה שמשמשת אלגוריתמים של תזמון מבוסס עדיפויות. אם תהליך בעל עדיפות גבוהה צריך לרוץ, המערכת תוציא Roll out (תהליך בעל עדיפות נמוכה מה-RAM לדיסק) ותכניס את התהליך החשוב ביותר – Roll in שיקבל זמן מעבד.

שיטות לניהול זיכרון**שיטה ראשונה - Contiguous Allocation**

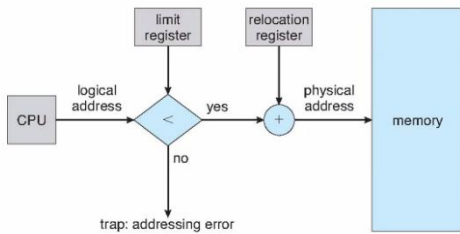
לוקחים את כל הזיכרון הפיזי ומחלקים אותו לשני רצפים. רצף אחד – מערכת ההפעלה והיא יושבת בתחילת הזיכרון. הרצף השני – כל התהליכים שהריץ המשתמש על בסיס PROGRAMS שהיו שמורות בדיסק. הרצף השני מגיע לאחר הרצף הראשון. התהליכים שלנו נשמרים בצורה **רציפה** בזיכרון. בשביל לשלוט בכל הניהול של זה – נשתמש ברגיסטר מסוג Allocation register (שיעזור לומר מה



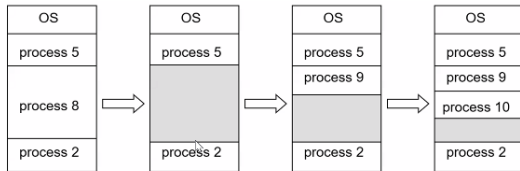
הכתובת האמיתית שנרצה להגיע אליה). נרצה מנגנון שיאפשר להגן על גישה לתהליך שלנו מגישה לא מורשית של תהליכים אחרים. נשמור **Base register** שישמור את המיקום הפיזי של התהליך ב-RAM ונשמור רגיסטר נוסף בשם Limit register שמציין את גודל המקום (בבייטים) של התהליך הנוכחי (אם נוסיף ערך זה ל-BASE זה הערך המקסימלי שהתהליך שלנו יכול להגיע אליו). חיבור ערכי הרגיסטרים מאפשר להגיע למיקום האחרון שהתהליך שלנו יכול להגיע אליו.

למשל, אם מערכת ההפעלה יושבת בתחילה עד כתובת 256000, לאחריה נוכל לשבץ תהליך חדש.

נסכם:



- המעבד מוציא כתובת לוגית. היא יכולה להיות לכל היותר בגודל התהליך.
- בודקים מול limit register: אם הכתובת שקיבלנו היא לכל היותר מגודלו (לא עברנו את הגבול) – נוכל להתקדם. אחרת: נוציא Trap שאומר שיש שגיאה בכתובות.
- אם הגענו לכאן: נקח את Base register ונוסיף אותה לכתובת הלוגית – ונגיע לכתובת הפיזית בזיכרון האמיתית.



נתבונן בדוגמה. נזכיר שהתהליכים שלנו יושבים בצורה רציפה בזיכרון. נניח והיה לנו מס' תהליכים. תהליך 8 פתאום סיים – המקום שלו בזיכרון התפנה. קיבלנו חור. מה נעשה? במקום שקיבלנו בו חור אנחנו לכאורה יכולים להכניס תהליכים אחרים בחור (המקום שהתפנה לנו בזיכרון). עם זאת – ככל שנתקדם – גודל החור ילך ויקטן, וכיוון שאנחנו שומרים בצורה רציפה בזיכרון לא נוכל לגשת למקומות זיכרון שפשוט יישארו לנו ריקים כי הם קטנים מדי ולא יכולים להכיל תהליכים.

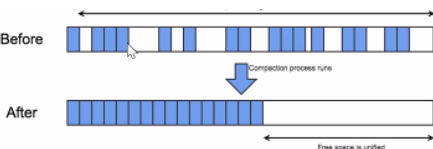
כשתהליך מגיע מערכת ההפעלה צריכה להחליט באיזה חור (איזור זיכרון פנוי) להכניס אותו. ישנם 3 גישות:

1. **First Fit**: סורקים את רשימת החורים מההתחלה ובחרים את החור הראשון שנמצא שהוא מספיק גדול עבור התהליך (הגודל של החור הוא לפחות כגודל התהליך). היתרון שלו – הוא הכי מהיר. לא צריך לעבור על כל החורים, אלא רק הראשון שמתאים.
2. **Best-Fit**: עוברים על כל החורים, ויוצרים תת קבוצה של כל אלו שמתאימים (גודלם הוא לפחות כגודל התהליך שלנו) לנו. מבין החורים בתת הקבוצה, לוקחים את זה בגודל הקטן ביותר. מה היתרון של זה? המטרה היא להשאיר חורים פנויים גדולים ככל האפשר במקומות אחרים – ולאפשר גמישות בעתיד למען תהליכים גדולים יותר. החיסרון הוא שזה איטי יותר (חייב לסרוק את כל הרשימה).
3. **Worst-Fit**: בוחרים את החור הגדול ביותר שנמצא. הרעיון – ע"י שימוש בחור הכי גדול אנחנו משאירים שארית גדולה ככל האפשר, בתקווה שהיא תהיה מספיק גדולה על מנת להכיל תהליך נוסף בהמשך.

במדדים של מהירות וניצול זיכרון, שני האלגוריתמים הראשונים נוטים להיות יותר טובים מהשלישי. עם זאת: יתכנו מצבים שבהם רק באמצעות השיטה השלישית נצליח לשבץ את כל התהליכים שיגיעו.

Fragmentation: פרגמנטציה היא מצב שבו הזיכרון הופך למפוזר ולא יעיל. ישנם שני סוגים עיקריים:

- **External Fragmentation**: יש לנו מספיק זיכרון פנוי על מנת לספק בקשה של תהליך, אבל הוא לא נמצא בבלוק אחד רציף. הזיכרון הפנוי מפוזר בין תהליכים קיימים כחורים קטנים. אנחנו יושבים ומבאסים – יש לנו מלא זיכרון פנוי, לא נשתמש בו? לכן אנחנו נאחד את הזיכרון. נזיז את כל התהליכים בזיכרון כך שכל הזיכרון התפוס יהיה בצד אחד וכל הזיכרון הפנוי יתאים לבלוק גדול ורציף אחד בצד השני.



על מנת שנוכל לבצע את הדחיסה הנ"ל (**Compaction**) חובה שה **Binding** יהיה דינמי ויתבצע בזמן ריצה – אם הקוד היה אבסולוטי לא היינו יכולים להזיז את הכתובות.

חיסרון של הדחיסה הוא כמובן זמן ריצה – על מנת לבצע את התהליך יש (באופן תאורטי) לקרוא את כל הזיכרון, ולכתוב את כל הזיכרון מחדש. זה מעבר על כל הזיכרון שיש כרגע במערכת!

סיכום מערכות הפעלה | © גיא יער-און

חיסרון נוסף – בעיית IO. למדנו בעבר על DMA - בה נותנים לו חור בזיכרון למשל, ואומרים לו להעביר את כל המידע לשם, ובמקביל אנחנו מבצעים דחיסה – הוא יעביר לאותו חור את המידע, אבל אנחנו ביצענו דחיסה והתהליך המתאים כבר זז למקום אחר בזיכרון. זה יוצר בעיות של סינכרוניזציה. לכן, כל עוד התהליך נמצא ב IO יש לנעול אותו שלא נוכל להזיזו.

- **Internal Fragmentation**: כאשר מערכת ההפעלה מקצה לתהליך בלוק זיכרון גדול יותר ממה שהוא באמת ביקש (למשל: מטעמי נוחות שלה). החלק העודף בתוך הבלוק יהיה פשוט מבזבז – אף תהליך אחר לא יכול להשתמש בו.

שיטה שנייה – Paging

נדבר על שיטה שנייה לניהול הזיכרון. כאן – אנחנו לא דורשים שתהליך אחד יהיה חייב להישמר בצורה רציפה בזיכרון (מה שיגרור לפרגמנטציה חיצונית). אנחנו נחלק את הזיכרון ליחידות קטנות וקבועות בגודלן. אנחנו לוקחים את ה RAM שלנו (הזיכרון הפיזי) לבלוקים בגודל קבוע.

- **Frames**: חלוקת ה RAM (הזיכרון הפיזי) ל"חלקים". לכל FRAME נותנים מספר.
- **Pages**: את הזיכרון הלוגי מחלקים לבלוקים בגדלים שווים (לגודל של FRAME) וקוראים לכל אחד PAGE.

הכלל: גודל כל PAGE תמיד זהה לגודל כל FRAME. גודל כל FAGE (וכך גם FRAME) יהיה לרוב חזקה של 2.

כעת, אנחנו יודעים לעקוב אחרי הפריימים שפנויים. כעת, אם מגיע תהליך שצריך במהלך הריצה שלו n PAGE צריך למצוא n פריימים בזיכרון הפיזי. פריימים אלו בזיכרון הפיזי – לא חייבים להיות רציפים! המידע רציף רק בתוך כל PAGE.

Page Table: כיוון שהדפים מפוזרים בזיכרון הפיזי, מערכת ההפעלה מחזירה עבור כל תהליך Page Table. טבלה זו היא מפת דרכים שאומרת – "הדף הלוגי ה X שלך נמצא במסגרת הפיזית X ב RAM". היא מתרגמת לכל תהליך את הכתובות הלוגיות לכתובות הפיזיות. מערכת ההפעלה עוקבת אחרי כל הפריימים הפנויים בזיכרון ומקצה אותם לפי צורכי תהליכים.

- **הבהרה**: לא ניתן להקצות לתהליך חצי FRAME. אלא רק FRAME שלם. מדוע אי אפשר? אם הקצנו שני תהליכים ל FRAME אחד אין לנו איך לנהל ודרך לדעת מי שייך לאן.

כיצד ה CPU מפרק את הכתובות הלוגיות על מנת להגיע לזיכרון הפיזי? הכתובת הלוגית שמייצר ה CPU איננה מס' אקראי. היא בנויה משני חלקים:

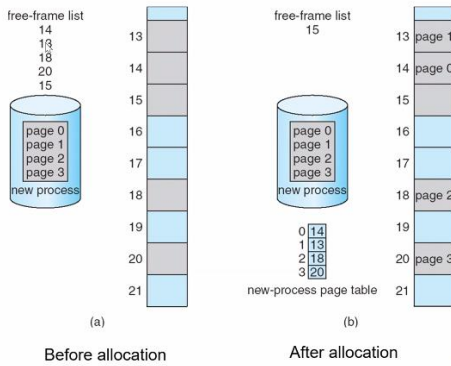
- **Page Number** (מס' דף - p): החלק העליון של הכתובת. משמש כאינדקס ל Page Table (מספר לנו באיזה דף אנחנו מחפשים).
- **Page offset** (היסט בתוך דף - d): זהו החלק התחתון של הכתובת. אומר לנו באיזה מקום ספציפי בתוך הדף נמצא המידע שאנחנו רוצים.

page number	page offset
p	d
m - n	n

כאשר ה CPU מבקש לגשת לכתובת לוגית – המערכת לוקחת את הק וניגשת איתו לטבלת הדפים, טבלת הדפים מחזירה את הכתובת הבסיסית של אותו FRAME בזיכרון הפיזי שבה הדף מאוחסן. המערכת מוסיפה לכתובת הבסיסית הזו את ההיסט d וכך מקבלת את הכתובת הפיזית המדויקת ב RAM.

נעיר. אם מרחב הכתובות הלוגי הוא 2^m וגודל כל דף הוא 2^n . אזי: n ביטים מוקצים ל offset ו m ביטים מוקצים עבור מס' page.

סיכום מערכות הפעלה | © גיא יער-און



מערכת ההפעלה צריכה לשמור את כל ה-Free Frames (הפריימים שלא מוקצים כעת לאף תהליך) ואז אם יגיע תהליך חדש שיבקש מס' מסוימת של דפים – נתן לנו מהפריימים הפנויים כרגע.

כך למשל – אם לפני הגעתו של תהליך P free frame list היה {14,13,18,20,15} והוא דרש 4 דפים – מערכת ההפעלה הקצתה לו 4, יצרה לו טבלת דפים ועדכנה את free frame list.

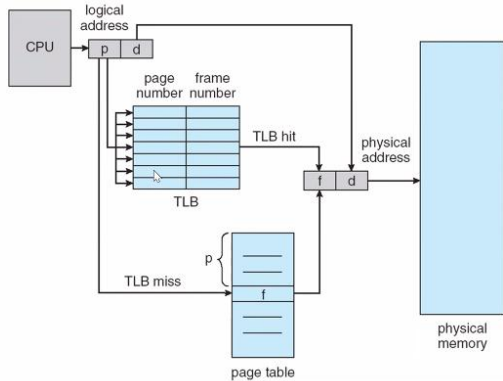
על מנת לגשת לנתון בזיכרון, עלינו לגשת קודם לטבלת הדפים על מנת לדעת היכן הוא נמצא ורק אז לגשת לכתובת הפיזית ב-RAM. זה אומר שכל גישה לזיכרון הופכת לשתי גישות פיזיות ל-RAM. זה מאט את המערכת משמעותית. כיצד המערכת מנהלת את טבלת הדפים? הטבלה נשמרת בזיכרון הראשי.

- Page table base register – PTBR: רגיסטר שמצביע על תחילת טבלת הדפים בזיכרון עבור התהליך הנוכחי.
- Page table length register – PLTR: רגיסטר שמציין מהו גודל הטבלה (על מנת למנוע גישה מחוץ לגבולות הטבלה)

על מנת למנוע מהצורך לגשת ל-RAM פעמיים בכל פעם, המערכת משתמשת בחומרה מיוחדת ומהירה מאוד שנקראת TLB – מדובר בזיכרון מטמון (קאש), קטן ומהיר מאוד בתוך המעבד ששומר את המיפויים האחרונים שביצענו (למשל: דף לוגי מס' 5 נמצא במסגרת פיזית מס' 12). מדובר במעין טבלה בקאש שמתרגמת את מס' ה-page למס' הפריים המתאים לו.

כיצד זה משפר את הביצועים? כאשר ה-CPU צריך לגשת לכתובת, הוא קודם בודק ב-TLB. אם המיפוי קיים שם (TLB hit) אנחנו ניגשים ל-RAM רק פעם אחת. אם המיפוי לא קיים (TLB miss) אנחנו ניגשים לטבלת הדפים ב-RAM, מביאים את המיפוי ומעדכנים את ה-TLB לפעם הבאה. בזכות ה-TLB מערכות הפעלה יכולות להשתמש ב-Paging מבלי לשלם מחיר ביצועים כבד.

כך נראה התהליך הכולל:



- קבלת כתובות מה-CPU: המעבד מייצר כתובת לוגית שמחולקת ל-p, d.
- בדיקת ה-TLB: אם מס' הדף נמצא ב-TLB, המערכת שולפת מיד את מס' הפריים המתאים. אחרת, אם הוא לא נמצא המערכת ניגשת לטבלת הדפים בזיכרון הראשי, מחשבת את הפריים המתאים ומעדכנת את ה-TLB לשימוש עתידי.
- הרכבת הכתובת הפיזית: המערכת משלבת את מס' הפריים שהתקבל עם האופסט המקורי.
- המערכת כרגע ניגשת לכתובת הפיזית.

Effective Access Time: כיוון שכל גישה לזיכרון דורשת גישה ל-Page Table, נרצה לחשב את זמן הגישה האמיתי. נרצה לדעת כמה המערכת מהירה, לכן נחשב את ה-EAT – זמן הגישה הממוצע לזיכרון. המטרה היא להבין כמה זמן המעבד באמת מחכה על מנת לקבל נתון מהזיכרון כשיש לנו TLB.

נגדיר את החיפוש ב-TLB כ-ε יחידות זמן. תמיד זה קורה – כי צריך לבדוק ב-TLB לפני הכל.

נגדיר α כאחוז ההצלחות שלנו (Hit ratio) – הסיכוי שמה שאנחנו צריכים נמצא ב-TLB. בדומה, $1 - \alpha$ הוא אחוז הכישלונות. זהו פרמטר תאורטי – המערכת לא יודעת על כך.

מקרה ראשון (נניח שזמן הגישה לזיכרון הוא 1 מיקרו שנייה): אזי –

1. אם זה HIT: מצאנו את המיפוי בTLB באפסילון יחידות זמן. כעת ניגשים לזיכרון הפיזי פעם אחת על מנת להביא את הנתון. סך הכל הזמן הוא $1 + \epsilon$. אחוז ההצלחה הוא α
 2. אם זה MISS: חיפשנו בTLB ולא מצאנו. כאן אנחנו חייבים גישה לטבלת הדפים בזיכרון (1) כדי למצוא את המיפוי, ואז גישה נוספת לזיכרון הפיזי על מנת להביא את הנתון. לכן סה"כ הזמן הוא $(1 + 1 + \epsilon)$.
- לכן – סה"כ הזמן הוא –

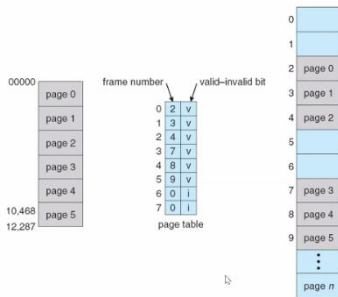
$$EAT = (1 + \epsilon)\alpha + (2 + \epsilon)(1 - \alpha) = 2 + \epsilon - \alpha$$

אם נניח כי זמן הגישה לזיכרון הוא 2 מיקרו שנייה:

$$EAT = (2 + \epsilon)\alpha + (4 + \epsilon)(1 - \alpha) = 4 + \epsilon - 2\alpha$$

Memory Protection

בטבלת Page Table נוסף עמודה נוספת – של ביט נוסף. ביט הגנה. המערכת מבצעת הגנה על הזיכרון ע"י קישור של ביט הגנה לכל פריים בזיכרון הפיזי. לכל כניסה בטבלת הבתים נכנסים ביט שנקרא valid-invalid bit.



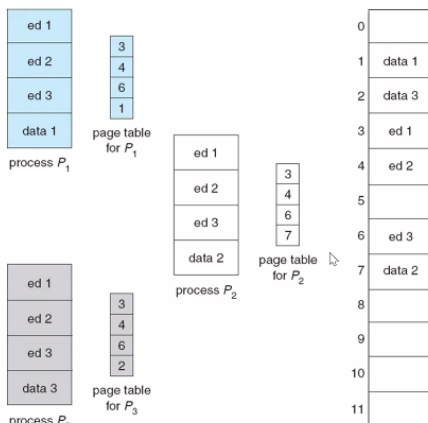
- Valid: תקין. מציין שהדף המשויך נמצא בתוך מרחב הכתובות הלוגי של התהליך, לכן מדובר בדף חוקי.
- Invalid: לא תקין. מציין שהדף אינו נמצא במרחב הכתובות הלוגי של התהליך.

למשל – מצב שבו אנחנו נמנע ממצב של גישה לזיכרון שלא הוקצה ולא מורשה לתהליך. מדוע הוקצו 8 אם הוא משתמש רק ב6? נזכר שאנחנו מקצים לרוב בחזקות של 2.

Shared Page

Shared code: אם יש לנו למשל עורך טקסט (למשל WORD) שרץ בשני חלונות שונים. אין סיבה לשמור פעמיים את קוד התוכנה ב-RAM. הקוד חייב להיות Read-Only, אם הוא לא משתנה הוא בטוח לשיתוף ברמת הדפים. כיצד זה עובד? כל התהליכים יצביעו בטבלת הדפים שלהם לאותו פריים בזיכרון הפיזי שבו יושב הקוד. הקוד המשותף חייב להופיע באותה כתובת לוגית אצל כל התהליכים.

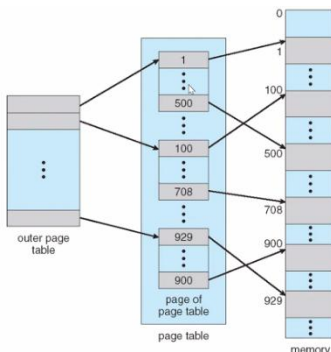
Private code: לעומת זאת, דברים שמשתנים (כמו הטקסט בתוך WORD) הוא "פרטי". לכל תהליך חייב להיות עותק נפרד ב-RAM כי כל אחד כותב דברים אחרים.



סה"כ זו אופטימיזציה של יעילות – לחסוך מקום בזיכרון. כך זה נראה למשל עם קוד משותף ולכל תהליך גם קוד פרטי (data 1, data 3):

סיכום מערכות הפעלה | © גיא יער-און

בעת שעברנו ל-Page Table נוצרה בעיה. נניח שהיה לנו מכונה שעובדת על 32 ביט. כל תהליך בגודל לכל היותר 2^{32} . והשתמשנו ב-PAGE בגודל של KB1. יהיו המון דפים – אנחנו נצטרך לשמור Page Table את המידע כפי שאמרנו (הקישור לפריימים) בצורה רציפה. אבל אנחנו צריכים עכשיו לשמור טבלאות מאוד גדולות – רציפות. אנחנו לא רוצים לשמור משהו גדול ורציף בזיכרון. נראה שלוש שיטות להתמודד עם הבעיה הזו:



1. **Hierarchical Paging:** שמירת רמה היררכית. פיצול טבלת הדפים לרמות (כמו עץ). במקום טבלה אחת ענקית, יש טבלה ראשית שמצביעה לטבלאות קטנות יותר. זה חוסך המון מקום בזיכרון כי לא צריך להחזיק טבלאות שלמות עבור חלקים בזיכרון שאינם בשימוש. לכן יוצרים – Outer Page Table: טבלה של טבלאות. היא לא מצביעה ישירות לזיכרון הפיזי, אלא מצביעה לטבלאות דפים אחרות. Page Of page Table: טבלאות פנימיות, שמכילות את המיפויים האמיתיים לכתובות הפיזיות שב-RAM. עכשיו רק הטבלה של הטבלאות והטבלאות הקטנות צריכים להיות רציפים, אבל כל המידע כולו ממש לא חייב להיות רציף.

נניח ויש לנו כתובת לוגית של 32 סיביות שמתחלקת כך: 10 ביטים ל-Page offset (גודל כל דף, קבוע). Page Number (החלק שצריך למפות) – בגודל 22 סיביות. כאן נכנס הפיצול: מחלקים את 22 הביטים לשתי קבוצות –

page number		page offset
p_1	p_2	d
12	10	10

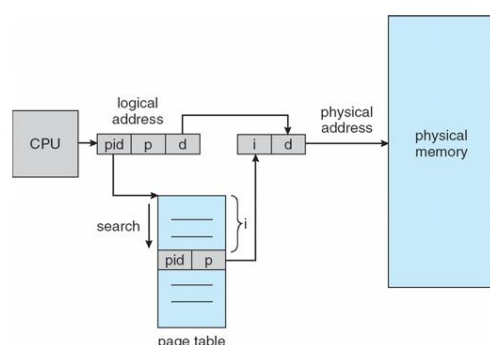
אחת, בגודל 12 ביטים משמשת כאינדקס לטבלה החיצונית.

שנייה, בגודל 10 ביטים משמשת כאינדקס לטבלה פנימית.

לכן, הכתובת הלוגית תראה כך:

- באופן דומה, ניתן לחלק גם ל-3 שכבות וכו'.
2. **Hashed Page Tables:** נשתמש בטבלת האש על מנת למצוא את המיפוי של דף לוגי למסגרת פיזית במהירות. כשיש לנו מרחב כתובות עצום (מעל 32 ביט), טבלת דפים רגילה הופכת להיות גדולה מדי לזיכרון. במקום להחזיק טבלה ענקית אנחנו משתמשים בהאש - לוקחים את מס' הדף הלוגי k ומעבירים אותו בפונקציית האש כדי לקבל מיקום בטבלה. בגלל שפונקציית האש יכולה להוביל לאותה תוצאה (התנגשות), כל תא בטבלה מצביע לשרשרת (רשימה מקושרת) של אלמנטים. אנחנו עוברים על השרשרת ומשווים את מס' הדף הלוגי שחיפשונו לזה שנמצא בכל חוליה. התוצאה: בעת שמצאנו התאמה – אנחנו שולפים את מס' הפריים שצמוד אליו ומחברים אליו את offset על מנת להגיע לזיכרון הפיזי.

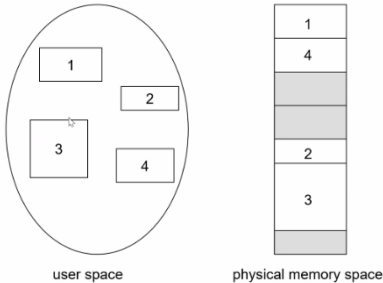
זו דרך טובה לחסוך מקום, אך מבצעים הרבה גישות לזיכרון.



3. **Inverted Page Table:** במקום ליצור לכל תהליך טבלת דפים, מחזיקים טבלה אחת בלבד לכל המחשב. הטבלה הזו ממופת לפי המסגרות הפיזיות ולא לפי הדפים הלוגיים. לכל פריים יש כניסה בטבלה ב-RAM. בכל כניסה – במקום לשמור לאיזה פריים הולך דף, אנחנו שומרים איזה דף לוגי (של איזה תהליך PID) יושב בפריים הזה. החיסרון הוא בחיפוש – כשצריך למצוא כתובת המערכת צריכה לסרוק את כל הטבלה הזו כדי למצוא את הדף המתאים וזה איטי.

שיטה שלישית – Segmentation

סגמנטציה היא גישה שמשקפת את הדרך שבה המתכנת רואה את התוכנית. כל תוכנית היא אוסף של סגמנטים, במקום רצף אחד ארוך של זיכרון התוכנית מחולקת ליחידות לוגיות בעלי משמעות. קוד, נתונים, מבני נתונים וכו'. המתכנת יודע שהוא כתב פונקציה או מערך והסגמנטציה שומרת על ההפרדה הזו בזיכרון הפיזי. מדוע זה טוב? זה מאפשר הגנה טובה יותר (ניתן לקבוע הרשאות לכל סגמנט בנפרד). למשל סגמנט הקוד יהיה לקריאה בלבד בעוד שסגמנט הנתונים יהיה לקריאה וכתיבה.



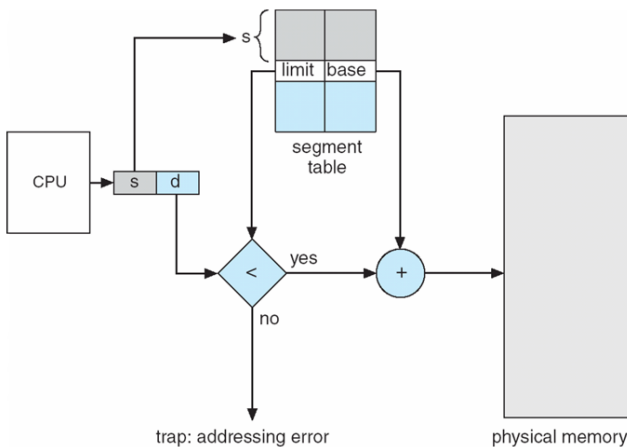
כלומר אם User space מכיל סגמנטים 1 עד 4, בזיכרון הפיזי הם ישבו באזורים שונים – כל אחד מהם ברציפות, אך לא כל המידע ברציפות.

כיצד מערכת ההפעלה מנהלת זיכרון באמצעות סגמנטציה?

1. במקום כתובת רציפה אחת, הכתובת הלוגית מורכבת מטאפל:

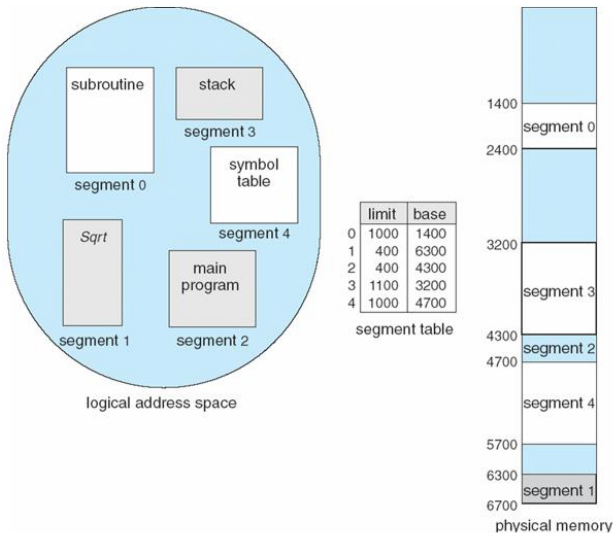
<segment-number, offset>. כאשר:

- Segment-number: מס' הסגמנט (למשל: סגמנט הקוד, המחסנית או סגמנט הנתונים)
- Offset – המיקום המדויק בתוך הסגמנט הזה (המרחק מתחילת הסגמנט).
- 2. בונים טבלת סגמנטים: זו טבלה שמתרגמת את הכתובת הלוגית לכתובת פיזית ב-RAM. כל שורה בטבלה מכילה:
 - Base: הכתובת הפיזית ההתחלתית שבה הסגמנט הנ"ל מתחיל ב-RAM.
 - LIMIT: האורך של הסגמנט, שקובע כמה זיכרון הוא תופס.
- 3. מנגנון הבקרה: המערכת משתמשת בשני רגיסטרים כדי לנהל את הטבלה הנ"ל ביעילות.
 - STBR: מכיל את כתובת הזיכרון שבה טבלת הסגמנטים נמצאת ב-RAM.
 - STLR: מצוין כמה סגמנטים יש לתוכנית זו.
- 4. הגנה: לכל כניסה מצורפים ביטי הגנה (read/write/execute) וביט ולידציה. קוד ניתן לשיתוף בין תהליכים אם הוא מוגדר Read-Only.

אם נסכם –

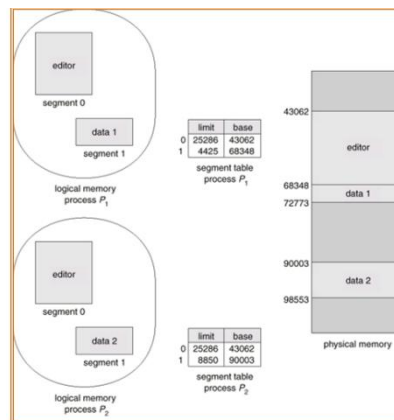
1. CPU מייצר כתובת לוגית שמורכבת משני חלקים: s מזהה את הסגמנט הלוגי (קוד, דטה, מחסנית וכו'). d את המיקום המדויק באותו הסגמנט (אופסט)
2. גישה לטבלת הסגמנטים – כל שורה בטבלה מכילה שני נתונים קריטיים: Base נכתובת הפיזית ההתחלתית שבה הסגמנט מתחיל בזיכרון ה-RAM וכן ה-LIMIT (האורך המקסימלי שיש לסגמנט זה).
3. בדיקת תקינות: המערכת משווה את ה- d (offset) שביקש המעבד מול ה-LIMIT של הסגמנט בטבלה.
4. אם $d < \text{limit}$ הבקשה חוקית – ממשיכים הלאה.
5. אם $d \geq \text{limit}$ זו חריגה מגבולות הזיכרון של הסגמנט. המערכת מקפיצה Trap (שגיאת כתובת) ועוצרת את הפעולה.
5. חישוב הכתובת הפיזית: אם הבדיקה עברה בהצלחה – המערכת מחברת את הכתובת הבסיסית של הסגמנט לאופסט המבוקש, וניגשת לזיכרון.

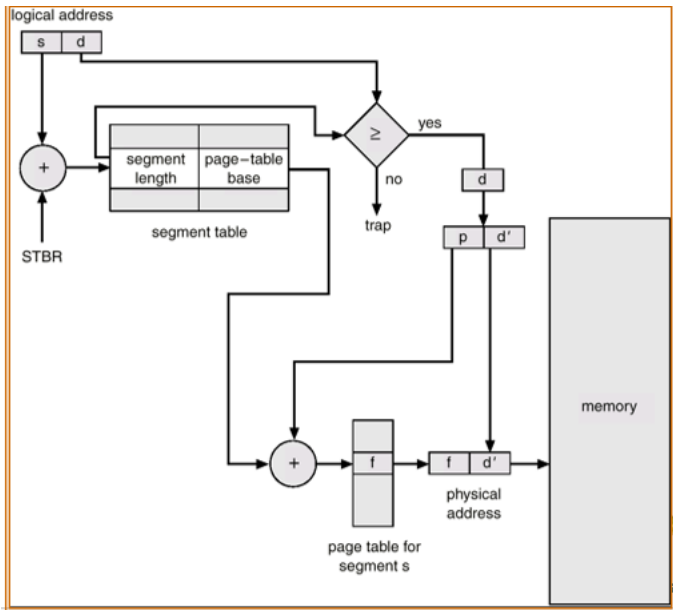
דוגמה:



- סגמנטים לא חייבים לשבת תמיד באותו מיקום ב-RAM, מערכת ההפעלה יכולה להזיז אותם ממקום למקום בזיכרון הפיזי לפי הצורך מבלי שהתהליך ירגיש בשינוי. הכל קורה באמצעות טבלת הסגמנטים – המעבד תמיד ניגש לכתובת הלוגית, טבלת הסגמנטים מעדכנת את base ב-RAM. ברגע שמעדכנים את base בטבלה המידע הועבר למיקום פיזי חדש מבלי לשנות שורת קוד אחת בתוכנה.
- אם יש לנו סגמנט שמכיל קוד משותף (כמו ספריות מערכת) אין צורך לשמור עותק שלו לכל תהליך, מספיק לשמור אותו פעם אחת בזיכרון הפיזי וכל תהליך יפנה אליו באמצעות טבלת הסגמנטים שלו. כדי שהשיתוף יהיה יעיל, מקובל שהסגמנט המשותף יקבל את אותו מס' סגמנט בכל התהליכים שמשתפים אותו.
- כיוון שהסגמנטים בגדלים שונים, הזיכרון הפיזי מתמלא בשיטות כמו First Fit (הראשון שנכנס) או Best Fit (המתאים ביותר). החיסרון הוא שלאורך זמן נוצרים ב-RAM חורים קטנים בין הסגמנטים השונים. לפעמים יש מספיק זיכרון פנוי בסך הכל אך הוא לא רציף ולכן אי אפשר להכניס סגמנט חדש (זו הבעיה שדפים פותרים).

דוגמה לשיתוף תהליכים:





Segmentation with Paging (MULTICS)

למה שנרצה לשלב את שתי השיטות? ישנן 2 בעיות.

1. סגמנטציה מעולה ללוגיקה (הגנה ושיתוף) אך סובלת מפרגמנטציה חיצונית (חורים בזיכרון).
2. פייג'ינג מונע פרגמנטציה חיצונית מעולה – אך אין לו הבנה לוגית של התוכנית.

לכן: למה שלא נשתמש בשניהם בו זמנית?

נסביר כיצד זה עובד:

1. הכתובת הלוגית – מורכבת עדיין מ- $\langle \text{segment}, \text{offset} \rangle$.
 2. טבלת הסגמנטים: במקום לשמור base פיזי, כל כניסה בטבלה מצביעה על טבלת דפים של אותו סגמנט ספציפי.
 3. האופסט מתפצל לשני חלקים –
- P: מס' הדף בתוך הסגמנט.
 - D': המיקום המדויק בתוך הדף.

היתרון: כל סגמנט לוגי מחולק לדפים – זה אומר שאנחנו לא צריכים למצוא גוש זיכרון רציף וענק עבור סגמנט שלם, אלא לפזר את הדפים של הסגמנט הזה בכל מקום בזיכרון הפיזי. התוצאה: קיבלנו נוחות של סגמנטים (הגנה ושיתוף ברמת הפונקציה – כל מה שבדף הוא מאותו סגמנט) ואת היעילות הפיזית של הדפים (אין חורים).

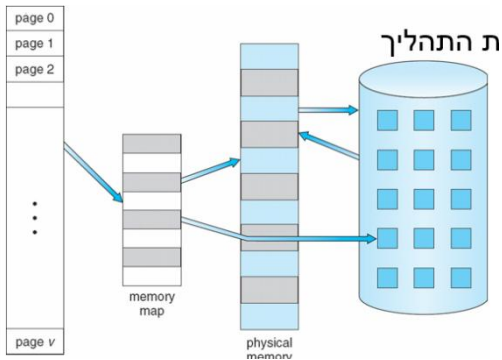
חסרונות: ניהול יותר מורכב, וכן כל בקשה גוררת 3 בקשות לזיכרון (גישה ראשונה לטבלת הסגמנטים – לדעת היכן נמצאת טבלת הדפין של הסגמנט הספציפי אליו ביקשנו לגשת, גישה שנייה לטבלת הדפים של הסגמנט, וגישה שלישית לזיכרון הפיזי – יש לנו את ה- $\text{offset} + \text{frame}$ וניגשים ל-RAM).

סיכום שיטות ניהול הזיכרון:

אם נרצה להשוות –

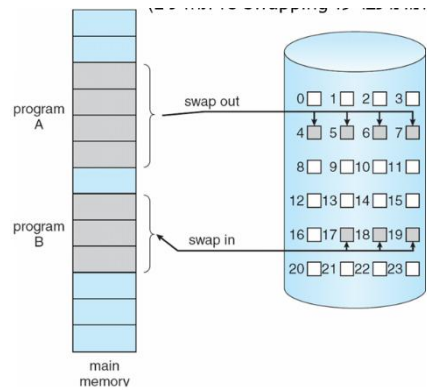
- Contiguous Allocation: הכל חייב להיות גוש רציף ב-RAM.
- Paging: הכל מחולק לבלוקים קטנים וקבועים (דפים) שמפוזרים בזיכרון (אין בזבוז כלל, אך אין משמעות לתוכן)
- Segmentation: הכל מחולק ליחידות לוגיות (סטאק, דטה וכו') שיושבות בבלוקים רציפים בזיכרון. הבלוקים עצמם לא צריכים להיות רציפים.

הרצאה 10



מה זה **virtual memory**? אנו יודעים שישנה הפרדה בין הכתובות הלוגיות לפיזיות. ראינו כי ניתן לחלק תהליך לדפים, וליצור לכל תהליך **page table** (בזיכרון) שמספר לנו באיזה פריים ב-RAM נמצא את תוכן ה-PAGE הרלוונטי. עד היום – הגבלנו את כמות הזיכרון הפיזי שהייתה לנו ובפרט חסמנו את גודל מרחב הזיכרון הלוגי שלנו. אם היו לנו כמה תהליכים, היינו צריכים להוריד לכל אחד מהם את גודל מרחב הזיכרון הלוגי – כי יותר מתחרים על הזיכרון. עד כה: כל התהליך ישב ב-RAM מבחינתנו בקורס. האמנם? אנחנו לא בהכרח נחזיק את כל התהליך שלנו ב-RAM – אלא חלקים מתוכו. זה המנגנון של **Virtual memory** – כלומר ישנו זיכרון וירטואלי: אינו יושב באמת ב-RAM, חלק מהפריימים של התהליך יושבים בזיכרון משני. בדיסק שהוא הזיכרון המשני – נאחסן חלק מהפריימים.

מדוע מותר לנו לעשות דבר שכזה? כי באופן רגעי – בעת הרצת instruction אין לנו צורך שכל התהליך ישב ב-RAM, אלא רק הפקודה הנוכחית (והנתונים שנצטרך במהלך הפקודה) ישבו בזיכרון. אם לא נטען את כל התוכנית לזיכרון – ונטען בתחילה רק את ה-PAGES הראשונים של הקובץ, זה יהיה הרבה יותר מהיר. לכן: הצלחנו להגביר באופן משמעותי את מהירות **loading** של התוכנית.



כל מערכות ההפעלה המודרניות עובדות עם זיכרון וירטואלי. כעת נרצה לשאול את עצמנו כיצד עושים דבר שכזה?

Demand paging: הרעיון הוא להביא PAGE לזיכרון רק כשזקוקים לתוכן של אותו PAGE – או שיש בו פקודה, או דטה שאנחנו זקוקים לו עבור פקודה שאנו רוצים להריץ כרגע. המנגנון הזה עובד על **swapping**. עם זאת, במקום להעביר תהליך שלם נרצה להעביר חלקים מתוך התהליך לזיכרון – **PAGES**. המנגנון שמאפשר לעשות זאת נקרא **lazy swapper** (מדוע עצלן? הוא אף פעם לא יביא משהו אם לא באמת צריך אותו). בעת שאנו צריכים PAGE – נבקש **reference** אליו ונבדוק אם הוא חוקי או לא (אם לא – נוציא הודעה בהתאם), ואם הוא חוקי: אם הוא בזיכרון, נשתמש בו. אם לא: אנחנו נצטרך להביא אותו לזיכרון מהדיסק. יתרונות:

- בעת ייצור תהליך חדש נצטרך פחות IO (העברת מידע בין ה-RAM לבין הדיסק), אך עם זאת בזמן הריצה עצמה יהיה יותר IO (נצטרך להביא PAGE שלא נמצאים כרגע בזיכרון).
- נצטרך פחות זיכרון על מנת להריץ את התהליך, מה שאומר שאפשר לשרת יותר משתמשים ותהליכים על אותו בסיס של זיכרון.
- מהירות תגובה טובה יותר (ביצירת התהליך!) לא בהרצתו.

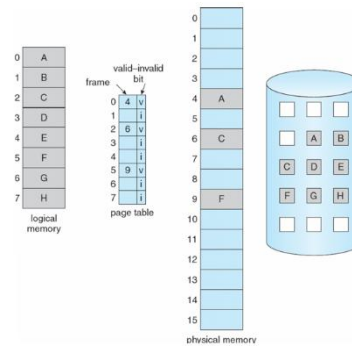
ניתן לראות בתמונה מעלה על **swapping** שאנחנו שומרים בדיסק את כל התהליך – לא רק את החלקים שלא נמצאים ב-RAM. מדוע? בשם הרציפות. כלומר, אם היינו שומרים בדיסק רק חלק היינו צריכים לעבור בלוק בלוק ולבדוק האם הבלוק הזה מכיל את הפייג' שאנו מחפשים. אך אם נשמור את הכל – ובפרט את נקודת ההתחלה, לא נצטרך לעבור בלוק בלוק. נשים לב שבמילא יש כאן הרבה IO בזמן הריצה – ואנחנו לא רוצים לבזבז זמן בחיפוש בדיסק, לכן נשמור את כל התהליך בדיסק.

Valid-Invalid bit: לכל כניסה ב-page table נשמור ביט שמעיד האם ה-PAGE ואליד.

- Valid: תוכן ה-page נמצא בזיכרון.
- Invalid: תוכן ה-page לא נמצא בזיכרון.

ושוב, נשים לב שאנחנו שומרים ב-page table את כל ה-PAGES ולא רק את אלו שבזיכרון: בשביל הרציפות.

לדוגמה – משמאל:



כיצד התהליך עובד? נחלק בשלבים ונעזר בתמונה משמאל. נניח שהתוכנית רצה ופתאום יש בקשה לגשת לזיכרון (למען פקודה/ נתון שאנו צריכים) והמעבד מוציא כתובת (p,d):

1. אנו ניגשים לpage table. אם גילינו שזה invalid – ייצרנו trap למערכת ההפעלה (page fault). מערכת ההפעלה תקבע האם מדובר ב invalid reference או שפשוט לא נמצא בזיכרון.
2. נניח וגילינו שהוא לא נמצא בזיכרון – היא הולכת לדיסק, למיקום המתאים, והיא קוראת את תוכן הimage של pagen ומביאה אותו לframe פנוי בזיכרון. אם לא קיים פנוי – נצטרך לפנות אחד לדיסק עם swapping.
3. בסיום, מעדכנים את הpage table (מעדכנים את הביט valid, וכותבים את הframe שמתאים לPAGE הנ"ל).
4. מבצעים מחדש את הפקודה שבגללה היה page fault.

נזכר בEAT – תוחלת זמן הגישה לזיכרון. יהי PAGE Fault Rate המקיים:

$$0 \leq p \leq 1$$

אם $p=0$ אזי לא היו page faults ואם $p=1$ אז כל פניה לזיכרון הייתה fault. הוא למעשה אחוז הפעמים שניסינו לגשת וגילינו שהנתון לא בזיכרון. מכאן ש:

$$EAT = (1 - p) * 2 * Memory Access + p * (page fault overhead + swap page out + swap page in + restart overhead)$$

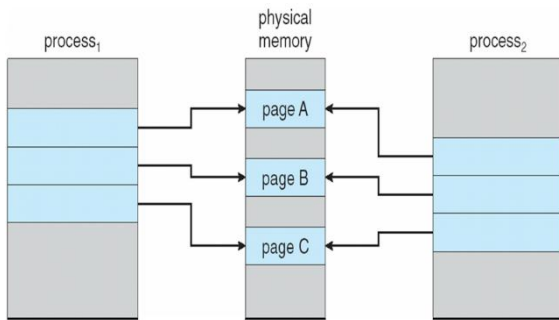
שכן אם הצלחנו אז מדובר פשוט בשתי גישות לזיכרון, ואחרת: יש לנו הרבה גישות נוספות כפי שתוארו קודם.

הערה. לעולם לא יתקיים כי $p=0$ שכן swapper עצלן: והוא מביא מהדיסק לזיכרון רק כשממש חייב. יתכן כי $p=1$.

נשים לב כי EAT מאוד גדול כאן. האם באמת יתכן שזה המצב? הרי החו, swap page out, לוקחים המון זמן! מה שנגלה בהמשך הוא שק מאוד מאוד נמוך (1 למאה אלף), ולכן EAT קטן יחסית.

עקרון הלוואה: בזמן נתון, אנחנו כנראה רצים על אותן PAGES בזיכרון.

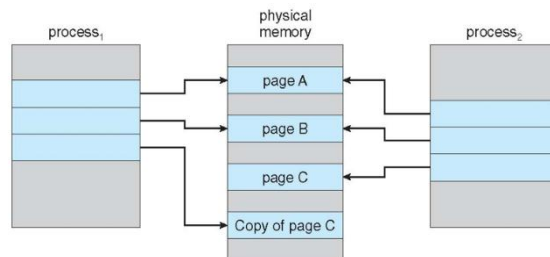
- Copy on write
- Memory mapped files (נדון בהמשך)



מה זה Copy on write? מאפשר לאבא ובן מלכתחילה לחלוק את כל הpages ששומרים בזיכרון. בעת ביצוע הfork() למעשה הpage table של תהליך הבן מצביע על אותם הפריימים שמצביע האב.

רק בעת שינוי בתוכן כתוצאה מפקודה שמריץ אחד התהליכים מיוצר העתק נוסף של הpage שהיה משותף להם. מאפשר תהליך יצירה מהיר יותר של תהליך הבן משום שרק pages שיש בהם שינוי מועתקים.

כך למשל אם תהליך 1 ישנה את PAGEC זה יראה כך:



אם כן, אמרנו שאם אנחנו מביאים מהדיסק לזיכרון – זה לתוך פריים קיים. ומה אם אין פריים פנוי בזיכרון? נרצה למצוא page שכרגע בזיכרון אך לא בשימוש, ולעשות לו swap out. נשתמש באלגוריתמים לבחירה בPAGE שייצא. נרצה שהאלגוריתמים יביאו למינימום page fault. יתכן שיוביל למצב שאותו PAGE נכנס ויוצא מהזיכרון כמה פעמים לאורך ההרצה.

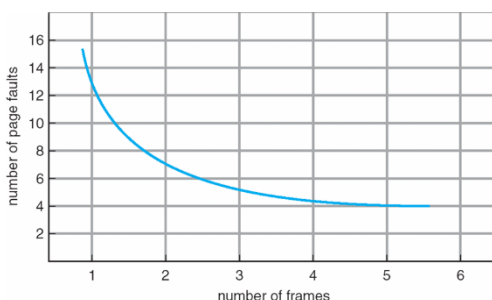
להלן המודל למצב שבו התוכן שנדרש איננו בזיכרון (שתמיד קבוע, רק האלגוריתם לבחירת הקורבן משתנה):

1. מצא בדיסק את תוכן הPAGE הדרוש.
2. מצא free frame בזיכרון הראשי:
- במידה וקיים פנוי, השתמש בו.
- אם לא, השתמש באלגוריתם להחלפת פריימים כלשהו על מנת לבחור victim frame.
3. הבא את הpage הנדרש לזיכרון וכתוב אותו בפריים הפנוי.
4. עדכן את הpage table.
5. בצע ריסטארט לinstruction של התהליך.

אלגוריתמי Page Replacement

המטרה של האלגוריתמים הללו הוא מזעור p. אנחנו נשווה את האלגו' השונים ע"י כך שנשתמש כל פעם באותו קלט. למען הדוגמה נשתמש ב: 1,2,3,4,1,2,5,1,2,3,4,5. אלו מס' הPAGES אליהם אנחנו רוצים לגשת בציר זמן משמאל לימין.

מהי ההתנהגות שאנו מצפים לראות בקשר בין מס' הפריימים שמוקצים לתהליך לבין מס' הpage fault? ככל שמס' הפריימים יגדל, כך מס' הpage fault יקטן:



סיכום מערכות הפעלה | © גיא יער-און

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

1. FIFO Algorithm

מה הרעיון? משבצים את ה-PAGES בזיכרון, בעת שיש Page fault אנחנו מוציאים את ה-PAGE שנכנס ראשון לזיכרון מבין אלו שכרגע בזיכרון.

נתבונן בדוגמה משמאל. נניח שיש לנו 3 פריימים בזיכרון עבור התהליך שלנו. בתחילה, יש לנו בקשה ל page מס' 1, הוא לא נמצא בזיכרון ולכן מכניסים אותו. כך שוב, עם 2 ו-3. לאחר מכן נכנס 4: נשאל מבין אלו שבזיכרון עכשיו (1 ו-2) מי נכנס ראשון? 1 – לכן הוא יוצא. כך זה מתקדם הלאה. נשים לב שתהליך יכול לצאת מהזיכרון ומיד אח"כ להכנס כאן בחזרה. (מבאס).

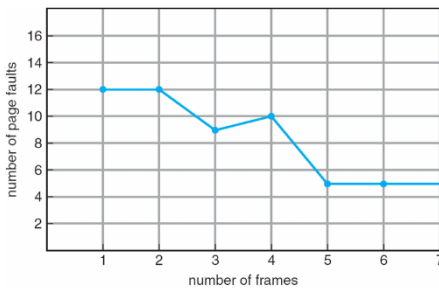
נשים לב כי אם היינו מגדילים את מס' הפריימים – זה לא היה מוריד את מס' page faults. זה אפילו היה מגדיל באחד.

- 4 frames

1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

- Belady's Anomaly: more frames $\Rightarrow$ more page faults



האנומליה של בלדי: בתרחישים מסוימים, אם נגדיל את מס' הפריימים יגדלו מס' ה page faults.

להלן נראה את הגרף התואם – גם הוא לא מראה ירידה ברורה. יש לפעמים עליה. מדוע ישנו קו ישר ב-5? כי ישנם 5 PAGES סה"כ.

OPTIMAL Algorithm

- Replace page that will not be used for longest period of time
- 4 frames example

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	4
2	
3	
4	5

6 page faults

- How do you know this?
- Used for measuring how well your algorithm performs

שיטה שבה האנומליה של בלדי לעולם לא תקרה. זו גם שיטה שנותנת את המינימום האפשרי של page faults עבור כל מחרוזת שהיא. כיצד עושים זאת? בוחרים את הקורבן בתור זה שמבין אלו שנמצאים בזיכרון, נצטרך אותו הכי רחוק ביחס לאחרים.

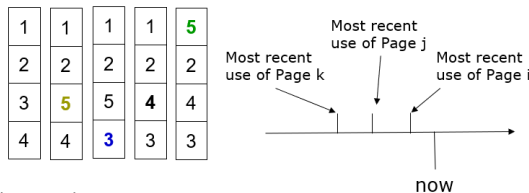
נשמע מושלם – מה הבעיה בזה? אין לנו דרך לחזות את העתיד. לכן, האלגו' הזה לא באמת עובד (אלא אם כן נתבסס על חיזוי מקרי עבר).

Least recently used (LRU)

אנחנו מחפשים באלגוריתם הנ"ל את השימוש האחרון שהיה לנו בכל אחד מהפייגים שכרגע נמצאים בזיכרון – ומבין הסט של השימושים האחרונים של כל אחד מהתהליכים, אנחנו רוצים להוציא את זה שהשתמשנו בו הכי רחוק אחורה בעבר.

מה הבעיה בשיטה הזו? מבחינת מימוש – קשה מאוד לממש LRU. דרך למימוש היא להחזיק את כל הדפים שלנו באיזו רשימה, ובכל פעם שאנחנו נוגעים בדף כלשהו להעביר אותו לתחילת הרשימה (ראשון). כשנצטרך לבחור את הקורבן – נקח את זה שנמצא בתחתית הרשימה. זה OVERHEAD משמעותי.

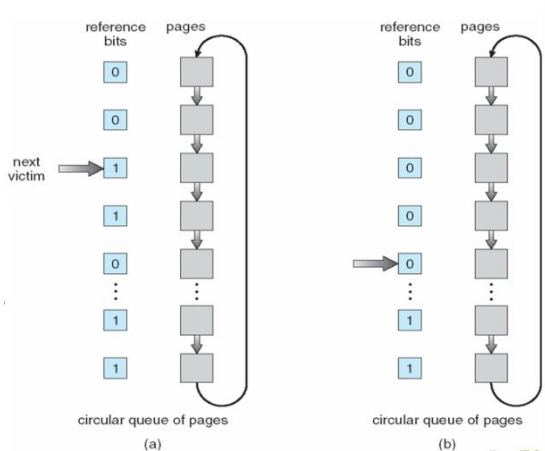
- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



- Counter implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to determine which are to change

סיכום מערכות הפעלה | © גיא יער-און

ניתן לאפסטיז שיטה זו: נרצה להחזיק מערך של ביטים, כל איבר במערך מייצג PAGE. נתחיל באיפוס המערך. בכל פעם שנגע ב-PAGE מסוים – נדרוס את התא המתאים במערך עם 1. בעת פקודה – כשנרצה למצוא קורבן: נצטרך למצוא איבר אחד שהוא אפס. מתי נהפוך את כולם במערך לאפס? כשכולם 1. **עם זאת – זו לא שיטה שתמיד תעזור לנו בהכרח.** מתי השיטה הזו כן טובה? כשכולם אחד פרט לאחד שהוא אפס.



Second chance

מה עושים בשיטה זו? נותנים סיכוי נוסף ל-PAGE. כיצד? מסתכלים על כל ה-PAGES שיש בזיכרון ומחזיקים ביט לכל PAGE. בכל פעם כשאנחנו נגע באיזהו PAGE אנחנו נכתוב בביט שלו 1. כמו כן, נוסף מצביע למי שאמור להיות המועמד הבא להיות הקורבן. וכך ממשיכים. ואז: אם יש Page Fault ניגשים למצביע, ועוברים עליו בצורה ציקלית וכל זמן שבו אנחנו מוצאים PAGE שהביט שלו הוא 1, נותנים לו סיכוי מחדש, משנים את הביט שלו לאפס וממשיכים בדרכנו.

Counting algorithms: אלגוריתמים שסופרים – סופרים את מס' הפעמים שנגענו ב-PAGE מאז שהוא נכנס לזיכרון.

LFU – נבחר את הקורבן להיות הדף עם counter הקטן ביותר (אם יש לנו משהו שיושב בזיכרון והcounter שלו מאוד נמוך כנראה שלא ניגשו למידע שלו בזיכרון ואין לנו צורך בו). זה בעייתי עם משהו שכרגע הבאתי לזיכרון.

MFU – הפוך. נבחר את הקורבן להיות הדף עם counter הגדול ביותר – כנראה ש"עשית את שלך", ולא יצטרך אותו שוב. גם זה לא תמיד נכון.

Allocation of frames

הקצאה של פריימים. אם יש לנו המון זיכרון – לא נצטרך להתעסק בזה. אך אם, יש לנו יותר תהליכים (עם דרישות זיכרון) גדולות מכמות הזיכרון שיש לנו, עלינו להחליט כמה פריימים יקבל כל תהליך. דבר ראשון שנרצה לשאול – האם יש מינימום פריימים שעלינו לתת לתהליך על מנת שיוכל לרוץ?

נבחין שאם ניתן לתהליך פריימים בודד – יתכן שלא נוכל להריצו. שכן יתכן שהפקודות שלה נמצאות בפייג' אחד, והמשתנים עליהם עובדת בפייג' אחר.

כמו כן – instruction יכולה להיות על 6 ביטים, 8 ביטים וכו'. אם ככה – יתכן שהיא יכולה ליפול בתפר שבין PAGES. לכן הגענו לתובנה: פריימים בודד לא מספיק.

Fixed allocation: נותנים לכל אחד מהתהליכים כמות שווה של הקצאה.

- **Equal allocation:** לוקחים את כל הפריימים שיש לנו בזיכרון, ומחלקים אותם במס' התהליכים. אם ישנם 100 פריימים ו-5 תהליכים כל אחד יקבל 20.

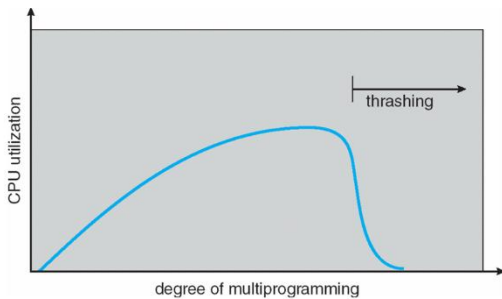
- הקצאה פרופורציונלית: מקצים בהתאם לגודל התהליך, באופן יחסי לגודל התהליך.

Priority allocation: כאן מקצים לפי העדיפות של תהליך. עדיפות גבוהה יותר: הקצאה גדולה יותר.

- **הקצאה גלובלית:** ברגע שמביאים לתהליך FRAME מסוים, יתכן שזה יגיע מתוך הפריימים שמוקצים לתהליך נמוך יותר, ולא בהכרח מהפריימים שהוקצו לו.

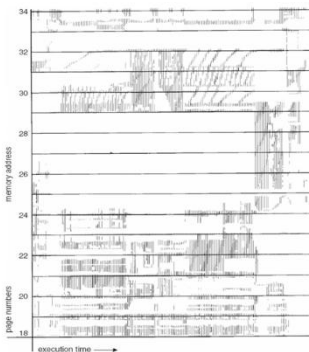
Thrashing

מצב במערכת. כיצד מגיעים למצב הזה?



נחשוב על מערכת, ובה רצים תהליכים. אחד מהם, אין לו מספיק פריימים. יש לו את המינימום שלו, אך אין לו מספיק במובן שיש לו המון page faults. הוא כל הזמן מוציא, ומכניס. מצב זה מוביל אותו לניצולת נמוכה של המעבד. הוא כל הזמן בIO. בעת שמערכת ההפעלה רואה מעבד בניצולת נמוכה – מה הדרך של המעבד לשנות זאת? הוא דואג שיהיו עוד תהליכים במערכת – ושהם יתפסו את המעבד. כלומר מגדילים את ה degree of multiprogramming. ואז – ניצולת המעבד תגדל. בגרף משמאל ניתן לראות שככל שיש קצת תהליכים, ניצולת המעבד נמוכה. ככל שמוסיפים היא עולה – עד לשלב מסוים שבה פשוט יש התרסקות. מדוע זה קרה? כי אם מערכת ההפעלה מרגישה בשלב זה שיש להוסיף עוד תהליך – הוא צריך עוד זיכרון, ומאיפה יבוא הזיכרון? מתהליכים קיימים. לכן התהליכים שקודם ישבו בIO בלי סוף – רק יעשו יותר כעת כי המקום שלהם קטן עוד יותר.

מה הקשר של demand paging ל thrashing? נתבונן בגרף משמאל – על ציר הy ישנם הכתובות הלוגיות. על ציר האיקס מצוין ציר הזמן. כל הנקודות השחורות בגרף זו נגיעה בהמון אזורים בזיכרון באותה ננו שניה.



- מודל locality: תוכניות נוטות לגשת לאזורים מסוימים בזיכרון בזמן מסוים, אזור זה נקרא Locality.
- התהליכים עוברים בין לוקאליות שונות. יתכן שיהיו בניהם חפיפות. thrashing קורה כאשר גודל הלוקאליטי גדול מסך הזיכרון הפיזי.

$$\sum \text{size of locality} > \text{total memory size}$$

- כיצד המערכת יכולה לזהות מצב זה? ע"י ניטור ניצולת הCPU והשוואתו למס' התהליכים שרצים במקביל. במידה ויש מס' גדול של תהליכים שרצים ועדיין ניצולת הCPU נמוכה, הרי שאנו במצב של thrashing. ניתן לצאת ממצב זה ע"י הקטנת מס' התהליכים (עצירתם), או לקחת זיכרון מתהליך שאין לו הרבה page faults למען תהליך שיש לו הרבה. ניתן גם להגדיל זיכרון אם כי זה פתרון חומרתי ולא פתרון של מערכת ההפעלה.

Working-Set Model

אנחנו כעת רוצים למצוא דרך למדוד את locality שלנו בכל יחידת זמן. אם נדע אותו (כלומר נדע את סט הדפים שתהליך ניגש אליו בזמן מסוים), נוכל לדעת את המינימום של פריימים שצריך לתת לכל תהליך. שכן, אם מערכת ההפעלה יודעת בדיוק איזה דפים צריך התהליך עכשיו היא יודעת כמה פריימים בRAM הוא חייב על מנת שלא להכנס ל thrashing.

נקח אנלוגיה יפה שתסביר זאת יותר טוב. נניח ואנחנו סטודנט שלומד למבחן, סטודנט מסתכל בו זמנית ב3 ספרים. locality הם 3 הספרים – אם הם היו יושבים כולם על השולחן, הוא יעבוד מהר. אם השולחן קטן מדי ומכיל מקום רק ל2 ספרים, הסטודנט בבעיה – הוא יפתח ספר א' וב', ואז יצטרך ג', ואז א' שוב. יבלה הרבה בהחלפות ספרים מהשולחן (RAM). זה בדיוק thrashing. אבל – אם מערכת ההפעלה הייתה יודעת שהסטודנט צריך 3 ספרים בשביל ללמוד, היא הייתה יודעת שאסור לתת לו שולחן (הקצאת פריימים בRAM) בגודל 2 ספרים, אלא 3 ספרים. לכן: אם מערכת ההפעלה תדע לחשב בכל רגע כמה דפים (locality) כל תהליך צריך בשביל לעבוד, היא תוכל לדעת מהו המס' הקטן ביותר של פריימים שהיא צריכה לתת לו.

סיכום מערכות הפעלה | © גיא יער-און

נסמן ב Δ את working-set-window – מוגדר כחלונית זמן קבועה שמורכבת ממס' קבוע של הפניות לדפים. לדוגמה, חלונית זו יכולה להיות בגודל 10,000 הוראות.

נסמן ב WSS_i את המס' הכולל של דפים אליהם התהליך P_i ניגש במהלך Δ האחרונים. למעשה זה סופר "כמה דפים ייחודיים התהליך הנ"ל נגע בהם במהלך חלון זמן הפקודות דלתא". זה working set שלו.

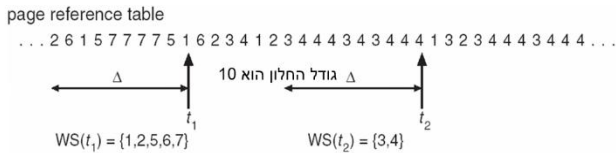
- אם דלתא קטן מדי, הוא לא יכיל את כל locality.
- אם דלתא גדול מדי, הוא יכיל מספר localities.
- אם דלתא = אינסוף, הוא יכיל את התוכנית כולה.

נגדיר את D להיות

$$D = \sum WSS_i$$

והוא מייצג את מס' הפריימים שהמערכת דורשת.

- אם $D > m$ (m הוא סך הזיכרון הפיזי הזמין) ייווצר מצב של thrashing. כאשר זה קורה משהים את אחת התהליכים על מנת להימנע ממצב זה.



נתבונן בדוגמה. נניח שהקריאות הם המחרוזת הארוכה (מס' הפייגים). וכן דלתא הוא 10. החלון משמאל שנפתח מקיים כי $WSS(p_1) = \{1, 2, 5, 6, 7\}$ כלומר 5 פייגים.

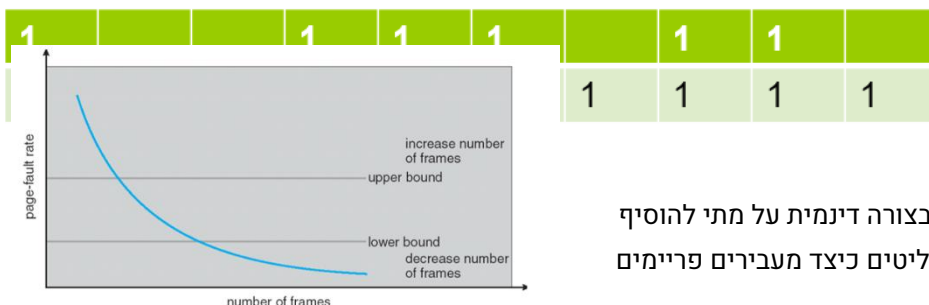
עם זאת, יש לנו בעיה. קשה לנו לחשב את working set: החלון שאנחנו מזיזים כל פעם משתנה – אי אפשר באמת לעקוב אחרי כל גישה לכל דף בכל רגע נתון כי זה יכביד על המעבד, לכן משתמשים בקירוב. במקום לעקוב אחרי כל פעולה המערכת משתמשת בשני כלים פשוטים:

- Interval timer: שעון שקוטע את פעולת המעבד בזמנים קבועים.
- Reference bit: ביט זיכרון שאומר לנו – "האם הדף הזה היה בשימוש?"

לדוגמה, אם דלתא הוא 10,000 והטיימר קופץ כל 5,000 פעולות. לכל דף שומרים 2 ביטים. בכל פעם שהטיימר נקטע המערכת מעתיקה את ערכי reference bits הקיימים ומאפסת אותם לאפס. אם באחד משני הביטים הללו בזיכרון מופיע הערך 1, המשמעות שהדף היה חלק מ working set במהלך חלון הזמן האחרון. עם זאת, המנגנון הנ"ל לא מדויק לחלוטין שכן מדובר בדגימה תקופתית שאינה מתעדת את הרגע המדויק של גישה לדף בתוך מרווח הזמן שבין הקטיעות. שיפור הדיוק נעשה ע"י הגדלת מס' הביטים וקיצור מרווח הזמן של הטיימר (למשל ל 1000 יחידות). מדוע זה טוב יותר? אם נגדיל את מס' הביטים ל 10 ואת מרווח הזמן ל 1000, נקבל מערך כבר עם 10 שורות. אם למשל פייג מסוים היה בשימוש רק ב 1000 הפקודות הראשונות, ולא הופיע יותר ב 9000 האחרות נקבל שכל הביטים שלו פרט לראשון יהיו אפס. המערכת תדע זאת.

כך זה ישמר במערך – מספיק לעשות OR בין השורות על מנת לדעת אם הדף חלק מה working set. המערך בגודל של מס' הדפים של אותו תהליך.

נתבונן בגרף משמאל – ניתן לראות שיטה כיצד להחליט בצורה דינמית על מתי להוסיף פריימים לתהליך ומתי לקחת פריימים מתהליך. כיצד מחליטים כיצד מעבירים פריימים



סיכום מערכות הפעלה | © גיא יער-און

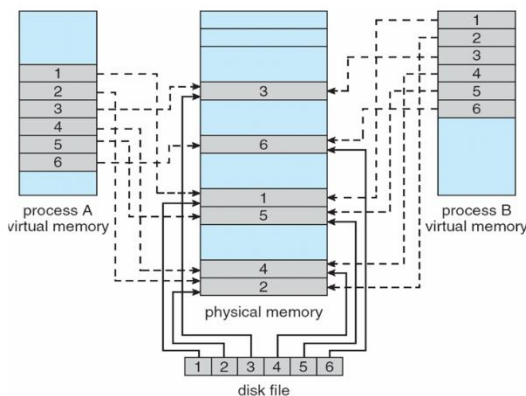
בין תהליכים? מסתכלים על אחוז ה page fault שלו ב10 שניות האחרונות למשל. כעת נראה בגרף שככל שמס' הפריימים גדל, כך קטן אחוז ה page fault. אבל השינוי הוא בשלבים. ניתן לראות שהירידה מתחלקת ל3 סוגים. בתחילה, אם נוסף עוד קצת פריימים לתהליך שהיה לו מעט פריימים, אם נוסף לו פריימים זה ישנה לו משמעותית ויקטין את אחוז ה page fault שלו. מגדירים שתי רמות:

- Upper bound: אם ה page fault rate נעדיף לתת לו עוד פריימים.
- Lower bound: נעדיף לקחת ממנו פריימים.

כך אפשר לנהל את כל הסיפור גם ללא working set, רק באמצעות מעקב אחרי ה page fault rate.

Memory mapped files

הרעיון הוא, למה שנגיש לקובץ דרך file system כשיש לנו וירטואל ממורי? הרי כל גישה לקובץ היא עם open, read, write וכו'. מדובר



במנגנון שמערכת ההפעלה יודעת לקחת את הקובץ שנמצא לנו בדיסק, לחלק אותו בצורה לוגית לבלוקים בגודל של PAGE ומהרגע הזה היא יודעת להתייחס לבלוקים האלו כאילו הם היו חלק מPAGES של תהליך. כלומר, בpage table של התהליך, נוספו גם הPAGES של הקובץ. וכעת, אם נרצה לגשת לקובץ ולשנות משהו, רק נצטרך לדעת כיצד למפות את המיקום שבו נרצה לשנות/לכתוב משהו בקובץ בoffset שבתוך הקובץ. בעת שנרצה לכתוב/לקרוא – נוציא בקשה לקריאה/כתיבה לזיכרון: ואם זה לא נמצא בזיכרון נקבל page fault. מה הרווחנו כאן למעשה? מהירות מטורפת. במקום לעבוד על הקבצים בדיסק ולשלוח כל פעם לdevice controller מה אנחנו רוצים לעשות, אנחנו יוצרים את הקבצים הללו בתור חלק מהזיכרון. במידה ואנחנו ניגשים למשהו מהקובץ, זה ממש עולה לזיכרון, ועל הפעולות שאנחנו עושים על הקובץ נעשות ממש על הזיכרון.

Prepaging: מה הכוונה? במקום לחכות שתהליך יבקש דף ואז יהיה page fault, אנחנו מראש ננחש איזה דפים התהליך יצטרך ונביא אותם לRAM לפני שהם נדרשים. המטרה היא להפחית את כמות ה page faults לאורך התהליך. עם זאת – זה יכול להיות מזיק: יתכן שהבאנו דפים לRAM שהתהליך בכלל לא יצטרך. כמו כן, המערכת צריכה להשוות בין שני דברים: הרווח – כמה page faults חסכנו, וההפסד: כמה זיכרון וזמן IO בזבזנו על דפים מיותרים.

Page Size: תמיד גודל דף הוא חזקה של 2. מה יקרה אם נגדיל את גודל הדף?

- Fragmentation: פנימית – זה יגדיל את הפרגמנטציה. שכן, אם נארוז את הדברים שלנו בקופסה גדולה יותר, הסיכוי שישאר משהו יותר גדול. חיצונית לא תשתנה.
- גודל טבלת page table תקטן, כי יהיו פחות דפים.
- IO Overhead: הזמן שלוקח בעת page fault, ככל שגודל הדף יהיה גדול יותר כך נדרש יותר להשקיע בIO כי יש לקרוא מהדיסק יותר נתונים לכל פריימים.
- Locality: ככל שגודל הדף יותר גדול הלוקליטי שלנו (שמורכבת מכל הפריימים בהם מצויים נתונים הנדרשים לנו) תופסת יותר נפח.

TLB Reach: סך הזיכרון שהמעבד יכול לראות בבת אחת ישירות דרך הTLB (זיכרון מטמון מהיר שמתרגם כתובות לוגיות לפיזיות). הנוסחה היא $TLB\ REACH = (TLB\ SIZE) * (Page\ Size)$. היינו רוצים באופן אידיאלי שיתפוס את כל הlocality.

סיכום מערכות הפעלה | © גיא יער-און

```
Program 1
for (j = 0; j < 128; j++)
  for (i = 0; i < 128; i++)
    data[i][j] = 0;

128 x 128 = 16,384 page faults

Program 2
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    data[i][j] = 0;

128 page faults
```

ולסיום, כמו שראינו במבנה מחשב. מעבר על מטריצה כשעוברים לפי השורות יותר מהיר. מדוע? בגלל הקאשינג. ואיך זה מתקשר אלינו? הרי מערכת ההפעלה מביאה לRAM פייג' שלם. כשניגשים לdata[i][j] MMU בודק אם הדף שמכיל את הכתובת הזו נמצא בRAM. אם לא, יש page fault. כשרצים לפי השורה, הרי המטריצה מאוחסנת שורה שורה, ניגשים לתאים סמוכים בזיכרון, שנמצאים באותו דף. לכן אחרי שטוענים את הדף הראשון, כל 128 הגישות הבאות הן באותו דף.

הרצאה 11

הערה. בהרצאה זו נלמד את החומר שנלמד בשנים רגילות בשתי הרצאות, ולכן חלק מהחומר שמופיע במצגות לא למבחן השנה לפי דודי. כלומר: רק מה שבסיכום הנ"ל יופיע במבחן.

נדון בFile System interfaces – כיצד אנחנו ניגשים לסייר הקבצים.

שאלה. הפעלנו את סייר הקבצים ויצרנו קובץ חדש. לא כתבנו לתוך הקובץ כלום! מה נוצר?

פתרון. נוצר אובייקט שמייצג קובץ, אך אין בו דטה עדיין.

שאלה. מחקנו מקובץ מסוים את כל הנתונים. האם הוא עדיין קיים?

פתרון. כן. כיוון שמחקנו רק את הדטה ולא את הקובץ עצמו.

שאלה. הפעלנו מחדש את סייר הקבצים וביקשנו למחוק קובץ מסוים. האם הנתונים שהיו שמורים בו כבר לא קיימים?

פתרון. לא – הם עדיין קיימים.

הגדרה: קובץ הוא אוסף של מידע שיש בין המידע הנ"ל קשר, המידע מאוגד תחת מבנה לוגי של קובץ. מאוחסן בזיכרון משני (דיסק). הקובץ מייצג מרחב זיכרון לוגי רציף (הדטה יישמר באופן רציף).

- מה שומרים בקבצים? דטה (נומרי, בינארי וכו') programsi.
- אין מבנה קבוע של קובץ. אין הפרדה לתווים, שורות או שדות. הפענוח של הדטה בקובץ תלוי בתוכנית שקוראת את הקובץ, ובכמה אנחנו יודעים על המידע של הקובץ. מתוכן הקובץ אין דרך להסיק על המבנה שלו.
- מאפיינים של קובץ:
 1. שם – לכל קובץ יש שם. זו האינפורמציה היחידה על הקובץ שבני האדם מבינים. לכן, השם ייתן רמז לרוב על מה שמור בתוך הקובץ. בעת שנתנו לקובץ שם, יצרנו ישות שלא תלויה באף תהליך. אם התהליך יסיים את הריצה שלו – הקובץ עדיין נשמר. התהליך "הותיר זכר" אחריו.
 2. Identifier – מזהה ייחודי חד חד ערכי של הקובץ. באמצעות המזהה אפשר לבדל את הקובץ מקבצים שונים.
 3. Type – רומז על המבנה של הקובץ. למשל אם הטייפ הוא docs. אזי נצפה לפתוח אותו ולראות קובץ בפורמט word. אך – זה "רומז", זה לא מחייב שהקובץ יהיה מסוג זה. זה יכול גם לבלבל אותנו שכן ניתן לשמור כל קובץ עם כל extension שנרצה.
 4. Location – מזהים שמצביעים על היכן נמצא הקובץ ("היכן" מתחלק לשניים: איפה הוא נמצא בFile system, וכן איפה הבלוקים עצמם של הדטה שמורים בדיסק – זו הדרך שלנו להגיע לדטה של הקובץ בדיסק).

5. Size – גודל הקובץ.

6. Protection – מידע לגבי מי יכול לקרוא, לכתוב ולהריץ את הקובץ.

7. Time, data and user identification – מידע שקשור באבטחה ובקרה, מי פתח פעם אחרונה את הקובץ, וכו'.

כל המאפיינים של הקובץ נמצאים ב**File control block – FCB**: מבנה נתונים שמערכת ההפעלה משתמשת בו בכדי לשמור מידע על הקובץ. הוא ה"תעודת זהות" של הקובץ.

directory הוא מבנה נתונים שממפה שמות קבצים אל המידע שמאפשר למצוא אותם. אפשר לחשוב על כך כמעין טבלה, עם שם הקובץ ומצביע ל**FCB** שלו. הוא יושב בדיסק, כמו כל קובץ אחר.

ישנם מס' פעולות שניתן לבצע על קובץ (כל פעולה היא כמובן system call):

- Create: יצירת קובץ. צריך למצוא מקום לקובץ החדש שיווצר, ולייצר entry בתוך directory.
- Write: מחפשת את הקובץ בdirectory (צריך לראות שמותר לנו בכלל לכתוב לקובץ), וכן אנחנו רוצים לראות ברשומה של הקובץ איפה נמצאים פיזית הבלוקים של הדטה. מתחזקת פוינטר – write pointer שמצביע על המקום הבא שיש לבצע אליו כתיבה.
- Read: יש למצוא את מיקום הקובץ בdirectory, וגם כאן ישנו read pointer שמצביע על המקום הבא שיש לקרוא ממנו.
- Delete: מחפשת את הרשומה בdirectory, משחררת את כל המקום בדיסק של הקובץ ומוחקת את הרשומה הרלוונטית מהdirectory.
- Truncate: כמו מחיקה, אך משאירה את הקובץ הקיים (מאפסת את התוכן שלו).
- Open(F): מחפשת בdirectory את הרשומה המתאימה, ומעבירה את התוכן של הקובץ לזיכרון. מדוע לזיכרון? כי לא נרצה לפנות לדיסק אח"כ כל פעם.
- Close(F): מחזירה את התוכן של הרשומה מהזיכרון חזרה לdirectory שנמצא בדיסק.

Open Files: בעת שביצענו open לאיזה קובץ, הוא נכנס לרשומה open Files. כאשר תהליך במחשב מבקש לפתוח קובץ, מערכת ההפעלה לא פשוט נותנת לו את הקובץ וזהו. היא צריכה לנהל רישום מסודר על מנת לדעת מה קורה עם הקוב הזה, במיוחד אם כמה תהליכים משתמשים בו במקביל. עבור כל קובץ פתוח מערכת ההפעלה שומרת בטבלה מיוחדת ארבעה סוגי מידע:

- File pointer: מחזיקים לכל תהליך שפתח קובץ פוינטר לקריאה של read/written האחרונה שהייתה. ניתן גם לשנות את המצביע באמצעות seek (זה לדעת מהיכן נקרא/נכתוב פעם הבאה). זה ברמת התהליך – כלומר פר תהליך שפתח את הקובץ.
- File-open count: מס' שסופר כמה תהליכים שונים מחזיקים את הקובץ פתוח ברגע זה. למה צריך את זה? המערכת צריכה לדעת מתי היא יכולה למחוק את המידע על הקובץ מטבלת הקבצים הפתוחים בזיכרון. כל עוד המונה גדול מאפס, המערכת משאירה את המידע זמין. כשהמונה מגיע לאפס – המערכת יכולה לנקות את הטבלה מקובץ זה.
- Disk location: cache של המידע המבני לגבי היכן הקובץ נמצא פיזית על הדיסק (למשל: בבלוקים של הדיסק). ההגיון – חיפוש של המיקום הפיזי של קובץ בדיסק הוא פעולה איטית שדורשת הרבה משאבים. ברגע שהמערכת מצאה את הקובץ בפעם הראשונה, היא תשמור את המיקום הזה בזיכרון המהיר כדי שבפעם הבאה שהתהליך יבקש לקרוא מהקובץ המערכת תדע מיד לאן לגשת מבלי לחפש מחדש.
- Access rights: לכל תהליך מחזיקים את מה שמותר לו לעשות (הרשאות כתיבה/קריאה וכו').

מערכת ההפעלה שומרת חלק מנתוני הקבצים הפתוחים בזיכרון ה**RAM** על מנת לשפר ביצועים ויעילות. מנגנון זה (המכניזם הזה) נקרא **File system caching**. כאשר ניגשים לקובץ, המערכת מעתיקה חלקים ממנו (דפים) לתוך ה**RAM**. העתק זה מאפשר קריאות עתידיות מהירות יותר, מה שמפחית את הצורך לגשת פיזית לדיסק. תהליך זה קורה באופן אוטומטי, גם אם האפליקציה לא ביקשה זאת במפורש.

Open file Locking: מנגנון שנועד למנוע תאונות שקורות כששני תהליכים מנסים לשנות את אותו הקובץ באותו זמן. ישנן 2 סוגי נעילות:

סיכום מערכות הפעלה | © גיא יער-און

- Shared lock: נעילה משותפת. מאפשרת לכמה תהליכים לקרוא את הקובץ בו זמנית, אך מונעת מהם לשנות אותו.
- Exclusive block: נעילה בלעדית. תהליך אחד מקבל גישה מלאה (קריאה וכתיבה) ואף אחד אחר לא יכול אפילו לקרוא את הקובץ עד שהנעילה תשתחרר.

ישנן 2 שיטות אכיפה –

1. Mandatory: מערכת ההפעלה היא השוטר. אם תהליך מנסה לגשת לקובץ נעול – המערכת חוסמת אותו בכוח ולא מאפשרת לו לבצע את הפעולה. זה מה שקורה בwindows.
2. Advisory: המערכת היא רק ה"יועץ". היא לא חוסמת תהליכים באופן אוטומטי, אלא רק מציעה מידע. התהליך צריך להחליט אם לציית לנעילה או להתעלם ממנה. זה מה שיש בלינוקס.

כיצד ניגשים לקובץ? כלומר, כיצד מערכת ההפעלה יכולה לגשת למידע ששמור בתוך קובץ?

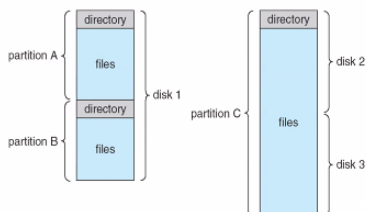
- Sequential access – זו הגישה הסדרתית. אם רוצים להגיע למידע שנמצא באמצע הקובץ יש לקרוא את כל מה שמגיע לפניו. אפשר לחשוב על זה כמו קלטת וידאו ישנה – אם רוצים לראות סצנה באמצע הסרט, חייבים להריץ את הקלטת מההתחלה עד שנגיע לסצנה הרלוונטית.
- Direct Access: כאן יש למערכת יכולת לקפוץ ישר למיקום ספציפי בתוך קובץ, בלי לעבור על מה שהיה לפניו. המערכת מבקשת בלוק מסוים, לפי מס' סידורי ח.

המבנה של Directory

הdirectory הוא אוסף של כל הFCB שמחזיקים את המידע על הקבצים. גם הdirectory עצמו וגם התוכן של הקבצים בFCB יושבים בדיסק. לכן, כשמגבים מידע על קובץ, צריך גם לעדכן את הdirectory בו זמנית. אחרת, עדכנו את המידע אבל אין לנו איך להגיע אליו.

ישנם מס' פעולות שנרצה לאפשר לבצע על directory:

- חיפוש קובץ
- יצירה של קובץ
- מחיקה של קובץ
- לראות את התוכן שיש כרגע בdirectory
- לשנות שם לקובץ מסוים בו.
- לסייר על הקבצים שיש לנו כרגע.



Partitions: מחיצה היא מבנה low level שעליו יושבים הקבצים והתיקיות של המערכת. היא יכולה להיות חלק מאזור פיזי מוגדר בתוך דיסק יחיד, ויכולה לאגד מס' דיסקים פיזיים לכדי יחידה לוגית אחת. במילים פשוטות יותר – ללא מחיצות הדיסק הקשיח הוא שטח אחד ענק – עם מחיצות: מציבים "גדרות" ומחלקים את המגרש לאזורים נפרדים. למה עושים את זה? כשמחלקים את הדיסק למחיצות מערכת ההפעלה מתייחסת לכל מחיצה כאילו היא דיסק עצמאי לגמרי – זה עוזר עבור סדר: מחיצה אחת למערכת ההפעלה, מחיצה שנייה לקבצים האישיים שלנו.

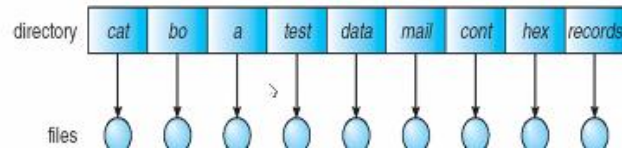
מבנה הדיסק:

- הדיסק יכול להיות raw – ללא file system, כלומר ללא מערכת קבצים. מערכת ההפעלה לא יודעת שיש שם קבצים בדיסק, כי אין מערכת קבצים שמנהלת את הסדר. למה שנעשה זאת? לפעמים תוכנות כמו מסדי נתונים רוצות לנהל את הגישה למידע בעצמן, הן אומרות למערכת ההפעלה: זוהי הצידה, אני אנהל את הקריאה והכתיבה מהדיסק בדרך הכי מהירה שאני יודעת. לעומת

סיכום מערכות הפעלה | © גיא יער-און

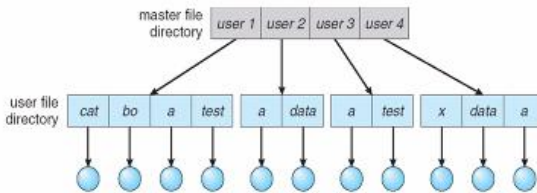
זאת, בגישה הרגילה – כן משתמשים במערכת הקבצים: מערכת הקבצים היא ה"ספרנית" שיוצרת קבצים ועושה סדר. היא מאפשרת למחשב לדעת איפה מתחיל כל קובץ והיכן הוא נגמר.

כיצד מתחזקים את הdirectory? ישנן כמה דרישות. קודם כל, נרצה שהיעילות תהיה טובה: שיהיה לנו קל למצוא קובץ. כמו כן, נרצה נוחות עבור המשתמשים כך שיותר ממשתמש אחד יוכל להשתמש באותו שם לקבצים שונים. למשל – על אותו שרת יהיה לנו ex1 של משתמש א' וגם ex1 של משתמש ב'. וכן, נרצה שלאותו קובץ נוכל לגשת בשמות שונים. דבר אחרון: נרצה דרך לוגית לבצע grouping לפי קשר בין קבצים.



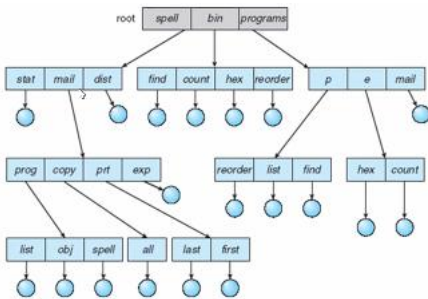
Single-Level directory: directory אחד לכל היוזרים. זה הכי קל למימוש.

עם זאת, יש בכך בעיות למשל עם naming – אי אפשר כפילויות שם.



Two-Level directory: נותנים לכל יוזר רק את הקבצים שלו. זה מעין יצירת directory

לכל יוזר בנפרד. כעת, אפשר שיהיו לשני יוזרים שונים אותו שם לקובץ. עם זאת, אין לנו כאן grouping (דרך לאחד קבצים). וכן – אין לנו דרך לגשת לאותו קובץ משני שמות שונים.



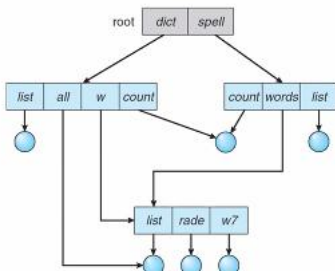
Tree-structured directories: כאן נשתמש כבר בעץ. זהו המבנה ההיררכי המקובל ביותר כיום

לניהול קבצים במחשב – מבנה עץ. במקום רשימה ארוכה ושטוחה של אלפי קבצים, המערכת מארגנת אותם בתוך תיקיות (ספריות) שיכולות להכיל תתי ספריות וכן הלאה. ישנו שורש – ממנו יוצאים ענפים (תיקיות ראשיות), כל ענף יכול להתפצל לענפים קטנים יותר ובקצוות נמצאים הקבצים (העגולים). יש לכך כמה יתרונות:

- חיפוש יעיל – במקום לסרוק את כל הדיסק, עוברים רק דרך הנתיב הרלוונטי.
- Grouping: ניתן לשמור יחד קבצים שקשורים זה לזה (למשל: כל קבצי התמונות בתיקייה אחת, וכו').

כיצד מתפעלים את זה? המערכת מאפשרת לנו לנווט ולנהל את הקבצים באמצעות פקודות.

- נתיבים: Absolute (מוחלט) הוא נתיב מלא מהשורש. למשל /spell/mail/prog/ - זה תמיד יוביל לאותו מקום – לא משנה היכן אנו נמצאים במחשב. Relative (יחסי) הוא נתיב ביחס למיקום הנוכחי שלך. אם אתה בתוך spell/ אתה יכול פשוט לכתוב mail/prog.
- יצירה ומחיקה: mkdir יוצר תיקייה חדשה בתוך התיקייה הנוכחית, וrmdir מוחק קובץ.

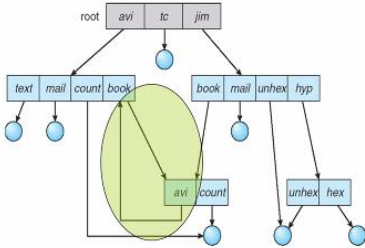


Acyclic Graph directories

במבנה ההיררכי רגיל של עץ, לכל קובץ יש רק אבא אחד. זה מגביל – אם יש שני אנשים שעובדים על פרויקטים שונים וצריכים להשתמש באותו קובץ נתונים, בעץ רגיל נהיה חייבים להעתיק את הקובץ פעמיים (אחד לכל תיקייה). זה מבזבז מקום ויוצר בעיית סנכרון. הגרף האציקלי (ללא מעגלים) מציע פתרון – שיתוף: במקום להעתיק, אנו יוצרים קישור מאותו קובץ לעוד מקומות. כעת, לא משנה מאיזו תיקייה נכנס נגיע לאותו קובץ מקורי בדיוק. הקשרים הם רק למטה, והמערכת אוסרת על מצב שבו חוזרים לתיקיית האב בתוך המסלול (תיקייה שמכילה את עצמה). אם היו מעגלים – המחשב היה נכנס ללולאה אינסופית בחיפוש אחר קבצים ולא היה מצליח לסיים את הפעולה. מה הבעיה שנוצרת כאן? כשקובץ משותף לכמה תיקיות, נוצרת בעיה

סיכום מערכות הפעלה | © גיא יער-און

של בעלות: אם משתמש אחד מוחק את הקובץ, המשתמש השני שנשאר יקבל שגיאה כי הלינק שלו כרגע מוביל לחלל ריק. הפתרון לבעיה זו הוא מונה – כל פעם שמקשרים קובץ לתיקיה חדשה המערכת מוסיפה +1 למונה של הקובץ, כל פעם שמוחקים קישור המערכת מבצעת 1- המחיצה האמיתית מהדיסק תקרה רק כשמישהו מבקש למחוק והמונה ירד לאפס. כל עוד המונה גדול מאפס – הקובץ חי כי מישהו אחר עדיין צריך אותו. כעת הוספנו דרך לגשת לאותו קובץ משני מקומות.



General Graph Directory

כאן אנחנו כבר לא מגבילים לגרף ללא מעגלים, מאפשרים לכל דבר להצביע על כל דבר. מה זה אומר בפועל? תיקייה אחת יכולה להכיל הפנייה לתיקיית אבא שלה או אפילו להצביע על עצמה. הבעיה הגדולה היא כשיש מעגלים – פעולות פשוטות הופכות לסיוט.

File system mounting: נדמיין שיש לנו מחשב בו כל הקבצים מסודרים בעץ אחד גדול. שורש העץ הזה הוא המקום בו נמצאת כרגע מערכת ההפעלה שלנו. עכשיו – נדמיין שאנחנו מחברים דיסק און קי למחשב, הדיסק הזה הוא בעולם משלו. יש לו מערכת קבצים משלו, תיקיות משלו וקבצים משלו. המחשב לא יכול פשוט לראות אותם כי הם לא חלק מהעץ המרכזי שלו. הוא מתייחס אליהם כמו אי בודד שמנותק מיבשת. כדי שמחשב יוכל לגשת לקבצים בדיסק און קי (או לכל קבצים שנמצאים בהתקן אחסון שהוא), הוא חייב לבצע פעולה של "עיגון" (Mounting). איך זה עובד? בוחרים תיקייה ריקה בעץ המרכזי של המחשב, מערכת ההפעלה מדביקה את הדיסק החיצוני לתוך התיקייה הריקה הזו. התוצאה – מעכשיו: כל מה שנראה באותה תיקייה הוא בעצם התוכן של הדיסק החיצוני. הדיסק הפך להיות "ענף" בעץ של המחשב שלנו.

File sharing: על מנת שנוכל לשתף קבצים במערכות מרובות משתמשים, חשוב לאפשר לכמה אנשים לגשת לאותם קבצים.

- User Ids: המערכת מזהה כל משתמש בנפרד, מה שמאפשר לקבוע הרשאות הגנה מותאמות אישית.
- Group Ids: המערכת מאפשרת לשייך משתמשים לקבוצות, כך שניתן לתת הרשאות גישה לכל הקבוצה בבת אחת.

בעלי הקבצים (מי שיצר אותם) צריכים לשלוט על מה ניתן לעשות עם הקובץ ומי רשאי לעשות זאת – מי יכול לקרוא, לכתוב, להוסיף, למחוק וכו'.

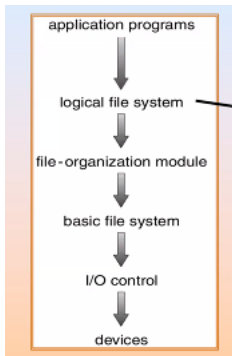
כדי ליישם את ההגנה, המערכת מחלקת את המשתמשים ל3 סוגים לכל קובץ:

1. Owner – בעלים, מי יצר את הקובץ.
2. Group – קבוצה, קבוצת משתמשים שהוגדרה כבעלת הרשאות מסוימות לקובץ זה.
3. Public – כל שאר המשתמשים במערכת.

המערכת משתמשת לעיתים קרובות בשיטת ה-RWX (קריאה, כתיבה, ביצוע). דוגמה: 761 – הבעלים מקבל 7 (1+2+4 = כל ההרשאות), הקבוצה מקבלת 6 (2+4: קריאה וכתיבה בלבד), והציבור מקביל 1 (הרצה בלבד).

File System Implementation

סיכום מערכות הפעלה | © גיא יער-און



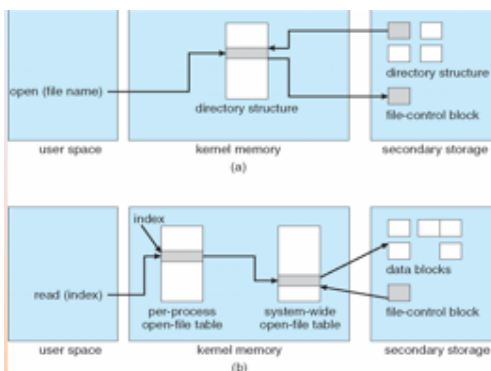
מערכת ההפעלה לא ניגשת לקבצים בבת אחת – אלא מחלקת את העבודה ל"שכבות" שונות, כאשר כל שכבה מטפלת ברמת פירוט שונה. החל משם הקובץ ועד לתנועה הפיזית של המידע על הדיסק.

- **Logical file system:** השכבה הזו אחראית על ניהול המבנה הלוגי של הקבצים. היא מתרגמת שמות קבצים למספרים ומזהים, ומהלך את המטה דטה (המידע על הקובץ), כמו איפה הוא נמצא. היא גם אחראית על ניהול התיקיות והרשאות ההגנה של המשתמשים.
- **File organization module:** שכבה זו יודעת את מיקום הקבצים בבלוקים לוגיים ופיזיים. תפקידה העיקרי הוא לתרגם את הפעולה הלוגית למיקום פיזי על גבי הדיסק.
- **Basic file system:** השכבה הזו שולחת פקודות גנריות למנהל ההתקן כדי לקרוא או לכתוב בלוקים פיזיים, היא גם מנהלת את הקאש של נתונים שנמצאים בשימוש תדיר ב-RAM כדי לשפר מהירות.
- **IO CONTROL:** השכבה הקרובה ביותר לחומרה, כוללת את הdevice drivers (מנהלי ההתקנים) שמעבירים את המידע בין הזיכרון הראשי למערכת הדיסקים. מתרגמת פקודות ספציפיות לחומרה.

כך המידע יורד בשכבות מהתוכנה ועד לחומרה.

- מערכת ההפעלה לא מוגבלת לסוג אחד של מערכת קבצים, בפועל היא יודעת לדבר עם סוגים רבים ושונים של אמצעי אחסון ומבני נתונים, כדי להתאים את עצמה לצרכים שונים של משתמשים ושל חומרה.
- כאשר אנו מבצעים פעולה שנראית פשוטה למשתמש, כמו פתיחת קובץ – בפועל זה system call. יש לנו קריאות מערכת עבור קבצים ברמת API, אך כיצד מממשים את הפונקציות שלהן בפועל? מערכת ההפעלה צריכה לנהל מבני נתונים מורכבים על הדיסק כדי לדעת איפה הקובץ נמצא וכיצד לגשת אליו.

- **Boot control block:** בלוק מיוחד בדיסק (בתוך partition) שמכיל את המידע הדרוש כדי לבצע את תהליך האתחול של המערכת (boot), הוא נדרש רק אם המחיצה בדיסק מכילה מערכת הפעלה, ולרוב הוא מאוחסן בבלוק הראשון של אותה מחיצה.
- **Volume control block:** מבנה שמכיל מידע כללי על המחיצה עצמה. זה כולל נתונים כמו מס' הבלוקים הכולל במחיצה, מס' הבלוקים הפנויים, גודל הבלוק, ומצביעים לבלוקים פנויים.
- **Directory structure:** המבנה שאחראי על ארגון הקבצים בתוך המחיצה עצמה. הוא כולל את שמות הקבצים ואת מספרי inodes שלהם, זהו החלק שמשמר את המיפוי בין שם הקובץ לבין המיקום הפיזי שלו על הדיסק.



כיצד מערכת ההפעלה מנהלת קבצים פתוחים בזיכרון? כדי שלא נצטרך לגשת לדיסק האיטי בכל פעם שנרצה לקרוא או לכתוב לקובץ, מערכת ההפעלה משתמשת בטבלאות בתוך זיכרון הקרנל כדי לשמור מידע על קבצים שנמצאים כרגע בשימוש.

- שלב א': פתיחת קובץ. הקרנל מחפש את הFCB הרלוונטי על הדיסק. אם המידע הזה עדיין לא קיים בזיכרון, הוא מועלה מהדיסק לזיכרון הקרנל כדי להאיץ את הגישה אליו בעתיד.
- שלב ב': קריאה מקובץ. לאחר שהקובץ פתוח, המערכת משתמשת בשתי טבלאות מרכזיות כדי לנהל את הפעולות:

1. **Per process open-file table:** טבלה אישית לכל תהליך שבה מופיעים הקבצים שהתהליך הספציפי הזה פתח.
2. **System wide open file table:** טבלה כלל מערכתית שמכילה מידע על כל הקבצים שפתוחים כרגע בכל המערכת.

הטבלאות הללו כוללות מצביע ל-FCB וגם מידע על מיקום הקריאה הנוכחי של התהליך בתוך הקובץ. בסוף, הקרנל דרך PF יודע בדיוק באיזה בלוקים על הדיסק הפיזי נמצא המידע שהתהליך צריך לקרוא. נעיר. המידע בשלב א' שהקרנל מביא הוא המטה-דטה של הקובץ, ה-FCB שלו.

סיכום מערכות הפעלה | © גיא יער-און

שאלה. מדוע העברת קובץ ממחיצה למחיצה בתוך אותו partition לוקחת פחות זמן מהעברת הקובץ בין מחיצות הממוקמות בשני partitions שונים? אם כן – בהעברת קובץ בתוך אותו partition אנחנו רק צריכים לשנות את הרשומה בdirectory (לאן הוא כעת מצביע), בעוד כשאנחנו עוברים בין partitions ישנה העתקה ממש.



דיסק מגנטי: כיצד המחשב מצליח באמת לשמור מידע על רכיב חומרה פיזי? נחשוב על הדיסק כערימה של תקליטים שעשויים ממתכת או זכוכית. הם מסתובבים במהירות גבוהה מאוד, ומצופים בחומר שניתן למגנט אותו. על פני השטח של כל פלטה, יש מעגלים קונצנטריים. זה כמו "מסלולי ריצה באצטדיון", כל מסלול כזה הוא אזור שבו נשמר מידע. כל מסלולים מחולק למקטעים קטנים יותר הנקראים סקטורים – זוהי יחידת האחסון הקטנה ביותר שהדיסק מסוגל לקרוא או לכתוב בפעם אחת. אז מה זה בסוף דיסק מגנטי? אם אנחנו רוצים לשמור מידע לטווח ארוך בצורה פיזית, דיסק מגנטי הוא ספריית תקליטים משוכללת מאוד שנועלת את עצמה בתוך קופסה. הוא נקרא מגנטי כי משתמש בכוח המשיכה המגנטי. בגלל שזה מכשיר מכני – כדי לקרוא קובץ ששמור בצד השני של הפלטה הדיסק צריך להזיז את הזרוע עם הראש, ולחכות שהפלטה תסתובב עד שהנקודה הנכונה תגיע מתחת לראש. לכן: דיסק מגנטי הוא כמו ספר שצריך לדפדף בו פיזית, ולכן הוא יחסית מאוד איטי.

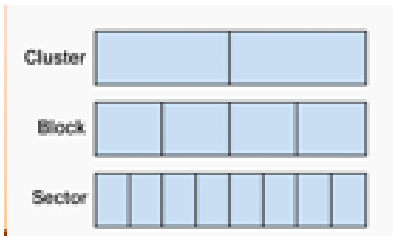
Solid State disks (SSD): כפי שראינו, לדיסק מגנטי יש מגבלות. הטכנולוגיה שהחליפה אותו היא SSD. ה-SSD הוא אמצעי אחסון שלא שוכח את המידע גם כאשר מנתקים את החשמל. אך בניגוד לדיסק מגנטי, אין בו שום חלק שזז. הוא הרבה יותר מהיר מדיסק מגנטי, בגלל שאין זרוע פיזית שצריכה לזוז ואין פלטה שמחכה להסתובב. המחשב פשוט פונה אלקטרונית לכתובת של המידע ומקבל אותו מיידית. הוא עמיד יותר לנפילות ומכות כי אין בו חלקים נעים שעלולים להישבר. הוא יקר הרבה יותר לכל מגה בייט מאשר הדיסק המגנטי. יש לו לפעמים אורך חיים קצר יותר – אך זה פחות מורגש.

רט מגנטי (Magnetic tape): אחד מהפתרונות הראשונים שהיו קיימים לאחסון משני, הסרט המגנטי נחשב לאמצעי אחסון יציב מאוד לאורך שנים ומסוגל להכיל כמות עצומה של דטה. החיסרון הגדול שלו – גישה איטית. בגלל שהמידע נשמר ברצף, הגישה לנתון ספציפי איטית בערך פי 1000 מדיסק קשיח – כי בשביל להגיע לנתון חייבים לעבור על כל הדטה עד שנגיע אליו (כמו שרוצים להגיע לסצנה בסרט – אבל אי אפשר לדלג אליה אלא יש לראות את כולו עד שנגיע אליה). כיום לא משתמשים בסרט מגנטי להרצת מערכות הפעלה או תוכנות אלא בעיקר לגיבויים של מידע. בעת שהסרט כבר הגיע למיקום הנכון מתחת לראש הקריאה, מהירות העברת הנתונים עצמה דומה לזו של דיסק רגיל. הוא זול מאוד ואמין מאוד.

ניהול הדיסק: על מנת שמערכת ההפעלה תוכל בכלל לכתוב מידע על כל דיסק פיזי, ישנם מס' שלבים טכניים מקדימים שצריך לבצע בשביל שמערכת ההפעלה תוכל בכלל לכתוב מידע שכזה.

1. Low level formatting: תהליך חלוקת הדיסק הפיזי למקטעים בסיסיים הנקראים sectors – כך שהבקר של הדיסק הפיזי יוכל לקרוא ולכתוב אליהם. כל סקטור כולל כותרת, את המידע עצמו, וקוד תיקון שגיאות לזיהוי ותיקון תקלות. לרוב סקטור מכיל 512 בתים של נתונים (אך ניתן לבחור גודל שונה).
2. Partitions: חלוקה למחיצות. כדי שמערכת ההפעלה תוכל להשתמש בדיסק פיזי לאחסון קבצים, עליה לרשום עליו מבני נתונים. לכן יש לבצע חלוקה של הדיסק הפיזי לקבוצות של צילינדרים כשכל אחת מהן נחשבת לדיסק לוגי עצמאי.
3. לאחר מכן, יש לבצע פרמוט לוגי: כלומר זה התהליך בו יוצרים מערכת קבצים.
4. כדי לשפר את היעילות, רוב מערכות הקבצים מאגדות בלוקים לקבוצות שנקראות clusters: קלט/פלט לדיסק מתבצע ברמת בלוקים. קלט/פלט לקבצים מתבצע ברמת clusters.

סיכום מערכות הפעלה | © גיא יער-און



- **Sector:** יחידת האחסון הפיזית הבסיסית ביותר על גבי כל דיסק. הגודל המסורתי הוא 512 בתים, כל פעולת קריאה או כתיבת ברמת הדיסק הפיזי מסתיימת תמיד ברמה של סקטור שלם.
- **Block:** יחידת האחסון הבסיסית של מערכת ההפעלה לצורך גישה לדיסק. בלוק יכול להיות שווה לסקטור אחד, אך לרוב הוא אוסף של כמה סקטורים. מערכת ההפעלה מתייחסת לדיסק כאל רצף של בלוקים, ולא של סקטורים בודדים.
- **Cluster:** קבוצה רציפה של בלוקים, מערכת ההפעלה מקצה לקובץ מקום ביחידות של קלאסטרים.

מערכת ההפעלה לא רוצה להתעסק עם "איזו פלטה", איזה "צילינדר" או איזה "ראש קריאה" יש להזיז. זה מאוד מסובך – ואיטי. לכן היא מתייחסת לדיסק כאילו הוא רשימה אחת ארוכה של בלוקים, שממוספרים מאפס ועד max. כלומר היא מתייחסת אל הדיסק כמערכת חד ממדי של בלוקים לוגיים. בפועל – הדיסק הפיזי מבצע תרגום מאחורי הקלעים בין המספרים הלוגיים שביקשה מערכת ההפעלה לבין המיקום הפיזי הממשי על הפלטות. המערכת ממפה את הבלוקים הלוגיים לסדרת הסקטורים הפיזיים בצורה רציפה. מתחילים בסקטור אפס, ממשיכים לאורך המסילה ועוברים לשאר המסילות באותו צילינדר. לבסוף, עוברים פנימה לצילינדרים הבאים. זה הופך את הדיסק ממכשיר מכני מסתובב ליחידת אחסון פשוטה מבחינת מערכת ההפעלה. מערכת ההפעלה רק אומרת לבקר הדיסק: תביא לי בלוק מס' X. החומרה של הדיסק כבר יודעת להזיז את הזרוע המכנית למקום המדויק על הפלטה כדי לשלוף את הנתונים משם.

Allocation Methods

איך נפרוס את הקובץ על גבי הבלוקים הלוגיים שזמינים לנו בדיסק? שיטת הקצאה מתייחסת לאופן שבו בלוקים של דיסק מוקצים לשימוש עבור קבצים. האתגר: איזון בין 2 מטרות שלעיתים סותרות זו את זו –

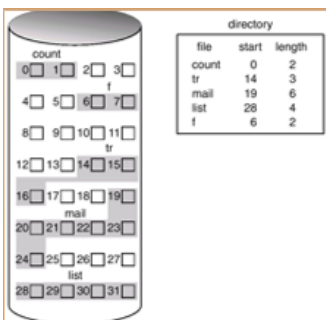
1. ניצול יעיל של הדיסק: לא להשאיר הרבה חורים ריקים או בזבז של מקום.
2. גישה מהירה לקבצים: היכולת לקרוא את הקובץ כמה שיותר מהר, על מנת למנוע תנועה מכנית מיותרת בדיסק.

Contiguous Allocation

שיטת הקצאה שבה כל קובץ תופס סדרה של בלוקים רציפים על הדיסק. כלומר, אם קובץ צריך 3 בלוקים המערכת חייבת למצוא 3 בלוקים פנויים שנמצאים אחד אחרי השני בדיוק.

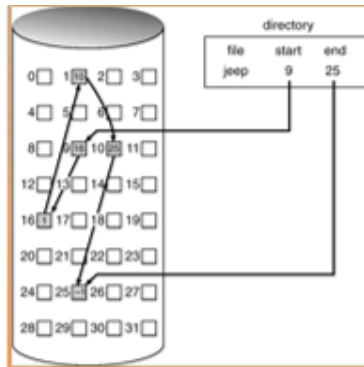
יתרונות: פשוט מאוד, על מנת לדעת היכן קובץ נמצא כל מה שהמערכת צריכה לשמור בתיקיית directory זה את כתובת הבלוק הראשון, ואת אורך הקובץ. וכן, גישה מהירה: אם רוצים להגיע לבלוק ה-2 בתוך קובץ שמתחיל בבלוק 10, ניגשים פשוט לבלוק 10+2.

חסרונות: פרגמנטציה (נוצרים חורים קטנים של מקום פנוי שלא ניתן להשתמש בהם כי הם לא רציפים), כדי לסדר את החורים הללו צריך להזיז את כל הקבצים שעל הדיסק כדי ליצור רצף חדש של מקום פנוי (זה דורש המון זמן! מאוד יקר). וכן הקובץ לא יכול לגדול – אם הקובץ שלך צמוד לקובץ אחר בדיסק, אין לו לאן להתרחב. כדי להגדיל אותו צריך להעתיק את כל הקובץ למקום אחר שבו יש רצף גדול יותר.



סיכום מערכות הפעלה | © גיא יער-און

Extent Based Systems: הקצאה רציפה יעילה לקריאה, אך דורשת שהקובץ יהיה גוש אחד מה שיוצר בעיות פרגמנטציה וקושי בהגדלת קבצים. כעת נרצה לאפשר את היעילות של גוש רציף מבלי לכבול את הקובץ למיקום בודד אחד על הדיסק. מה זה Extent? במקום להקצות את כל הקובץ כגוש אחד רציף, מערכת ההפעלה מקצה Extent: בלוק רציף של סקטורים. קובץ יכול להיות מורכב מרכיב extent אחד או יותר. – זו פשרה חכמה, המערכת עדיין מקבלת את הביצועים של קריאה רציפה כי כל EXTENT הוא רציף, אך אם לקובץ נגמר המקום הוא פשוט מקבל extent חדש במקום אחד בדיסק, מבלי להצטרך להעתיק את כל הקובץ מחדש. החיסרון – פרגמנטציה. הן חיצונית והן פנימית. אם ה-EXTENT שהוקצה גדול מדי ואין בו צורך: המקום מבזבז. וכן, בגלל שמשתמשים ב-EXTENTS בגדלים שונים עלולים להיווצר חורים בין Extents השונים בדיסק.

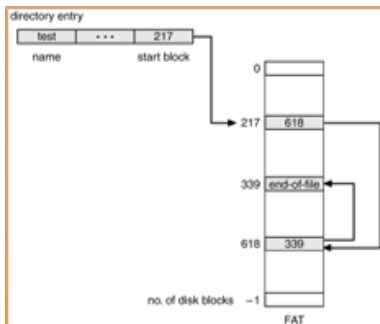


Linked Allocation

שיטת הקצאה נוספת. כעת לא נדרוש רציפות – כל בלוק בדיסק מכיל שני חלקים: הנתונים ומצביע לכתובת של הבלוק הבא באותו קובץ. לכן נקבל מעין רשימה מקושרת. ה-*directory* שומרת רק את הכתובת של הבלוק הראשון (*start*) ואת הכתובת של הבלוק האחרון (*end*).

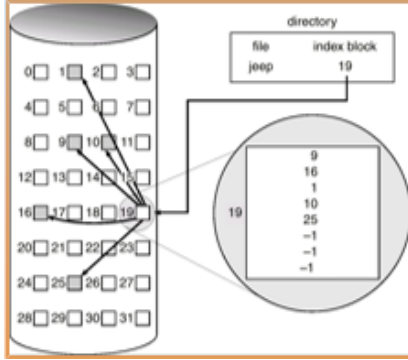
יתרונות: אין בזבוז מקום ופרגמנטציה, כיוון שלא צריך רצף של בלוקים פנויים אפשר להשתמש בכל בלוק פנוי על הדיסק: לא משנה היכן הוא נמצא. וכן ישנה גמישות – קבצים יכולים לגדול בקלות, פשוט מקצים בלוק פנוי כלשהו ומחברים אותו לסוף השרשרת.

חסרונות: אין גישה אקראית – אם רוצים לקרוא את הבלוק העשירי בקובץ, חייבים לקרוא את כל ה-9 לפנינו על מנת לעקוב אחר המצביעים. וכן ישנה בעיית אמינות – אם מצביע אחד נמחק או משתבש, כל המשך הקובץ אובד. כיוון שהבלוקים מפוזרים, קריאת קובץ דורשת הרבה יותר תנועה של ראש הקריאה מה שמאט את הערכת משמעותית.



(FAT) File allocation table: שיפור לשיטה זו – במקום לשמור את המצביעים בתוך הבלוקים של הקובץ (מה שגורם להם להיות מפוזרים), מרכזים את כל המצביעים לטבלה אחת גדולה שנמצאת בזיכרון. כלומר, מדובר במעין מערך אחד גדול שמשקף את הדיסק. במקום שכל בלוק יבזבז מקום על הפוינטר שלו, אנחנו מוציאים את המצביעים הללו החוצה לתוך הטבלה המרכזית הזו. אם בטבלה זו כתוב במיקום 5 את המספר 10 – זה אומר: הבלוק הבא אחרי בלוק 5 הוא בלוק 10 (כלומר 5 מצביע על 10). כעת חסכנו במקום – המידע בתוך הבלוקים עצמם מכיל רק דטה כי המצביעים עברו לטבלה, וכן אם רוצים לדעת מה הבלוק הבא של קובץ לא צריך לקרוא מהדיסק את הבלוק הקודם – אלא מסתכלים ב-FAT שנמצאת ב-RAM, והתשובה זמינה מיידית.

נתבונן בדוגמה משמאל ל-FAT: נניח שנרצה לקרוא קובץ ששמו *test*. המערכת מבצעת את הפעולות הבאות – מתחילה ב-*directory entry*: התיקיה אומרת – הקובץ מתחיל בבלוק מס' 217. משם, בתא 217 כתוב 618: לכן הולכים לתא 618 וכן ממשיכים להתקדם בשרשרת. כשהמערכת מגיעה לתא 339 שם כתוב 1, זהו סימן מוסכם לכך שהגענו לבלוק האחרון של הקובץ.



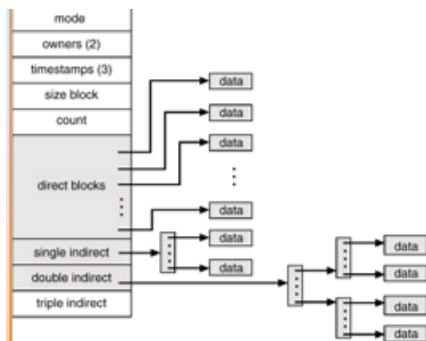
הבעיה בגישה עם הרשימה המקושרת – אם רצינו להגיע לבלוק העשירי, היינו חייבים לעבור את כל 9 שלפניו. כעת נפתור את הבעיה הזו ע"י ריכוז על המצביעים למיקום של בלוקי הקובץ בתוך מבנה נתונים אחד שנקרא index Block. לכל קובץ נחזיר בלוק מיוחד (אינדקס בלוק) שיתפקד כ"תוכן עניינים". בתוכו יש רשימה של מצביעים-הבלוק ה-1 של הקובץ נמצא בכתובת X, הבלוק ה-2 בכתובת Y וכן הלאה. אם נרצה את הבלוק ה-5 של הקובץ, פשוט נפתח את האינדקס בלוק במקום ה-5 בטבלה ונשלוק משם את הכתובת האמיתית.

יתרונות: גישה אקראית מלאה, לא צריך לעבור בלוק בלוק וכמו בשיטה הקודמת – אין פרגמנטציה.

חסרונות: overhead. יש בזבוז מסוים כי חייבים להקצות אינדקס בלוק לכל קובץ, גם אם הוא קטן מאוד.

- אם הקובץ ענק – ורשימת המצביעים שלו לא נכנסת באינדקס בלוק אחד, משתמשים בשיטות משולבות שמשלבות את כל מה שדיברנו עליו קודם. למשל: Linked scheme – מקשרים בלוק אינדקס אחד לאחר.
- בזבזני עבור קבצים קטנים (חייבים לשמור עוד אינדקס בלוק לטבלה תמיד!)
- ניגשים פעמיים – קודם לטבלה ואח"כ לנתונים.
- יש פרגמנטציה פנימית.

זוהי השיטה הגמישה והכי מכירה לגישה אקראית מבין השלוש, והיא הבסיס להרבה ערכות קבצים מודרניות.



Combine scheme: היישום המעשי והנפוץ ביותר של שיטת האינדקס. למה בכלל משתמשים בה? בשיטת האינדקס הפשוטה – הכל תלוי בגודל האינדקס בלוק. אם הוא קטן מדי, אי אפשר להחזיק קבצים גדולים ואם הוא גדול מדי מבזבזים המון מקום על קבצים קטנים. לכן בUNIX משתמשים במבנה מתוחכם (נקרא לרוב inode) שמאפשר גמישות מקסימלית.

ככל שהקובץ ממשיך לגדול, מוסיפים לו מצביעים.

Free space management: דיברנו עד כה כיצד מקצים בלוקים לקבצים קיימים. אך בשביל להקצות בלוק, המערכת חייבת לדעת אילו בלוקים בדיסק פנויים ואילו תפוסים. כיצד? השיטה הנפוצה, הפשוטה והיעילה ביותר היא Bit vector – וקטור סיביות. יוצרים מערך של ביטים שמייצג את כל הבלוקים בדיסק. ביט שערכו אפס: אומר שהבלוק המתאים בדיסק פנוי. ביט שערכו 1: אומר שהבלוק תפוס. בשיטה זו המערכת יכולה לסרוק את הדיסק מהר מאוד, כדי למצוא רצף של בלוקים פנויים (ע"י חיפוש רצף אפסים במערך), וכן המערך תופס מעט מקום יחסית לנפח הכולל של הדיסק (ביט אחד לכל בלוק).

ישנם דרכים נוספות פרט לBit vector:

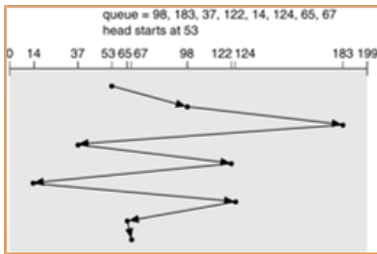
- רשימה מקושרת: כל הבלוקים הפנויים משורשרים זה לזה ברשימה. החיסרון – קשה מאוד למצוא מקום פנוי רציף בשיטה זו.
- Counting: במקום לרשום כל בלוק פנוי בנפרד, שומרים את הכתובת של הבלוק הפנוי הראשון ואת מספר הבלוקים הפנויים הרציפים שבאים אחריו. זה חוסך מקום בטבלאות הניהול כאשר יש הרבה שטחים פנויים רציפים.
- Grouping: גישה דומה לאינדקס. משתמשים בבלוק פנוי אחד כדי לאחסן את הכתובות של N בלוקים פנויים אחרים. יתרון – מאפשר למצוא במהירות כתובות של מס' רב של בלוקים פנויים כשעולה צורך בהקצאה גדולה.

סיכום מערכות הפעלה | © גיא יער-און

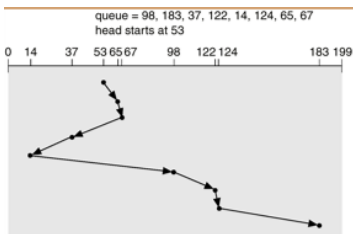
עד כה עסקנו בכיצד לשמור קבצים. כעת נסתכל על הדיסק המגנטי בתור מכונה פיזית – כיוון שהדיסק הוא הרכיב הכי איטי במחשב, למערכת ההפעלה יש אינטרס אדיר לארגן את בקשות הקריאה והכתיבה על מנת לחסוך זמן תנועה מיותר של הזרוע המכנית. ה Seek Time הוא הזמן שלוקח לזרוע המכנית של הדיסק לזוז פיזית לצילינדר הנכון – זהו גורם העיכוב הגדול ביותר.

- Rotational Latency: זמן המתנה סיבובי. הזמן שלוקח לפלטה של הדיסק להסתובב עד שהסקטור המבוקש מגיע בדיוק מתחת לראש הקריאה.
- Disk bandwidth: רוחב פס של הדיסק. סך כמות המידע שעובר בזמן מסוים (כמה בקשות המערכת הצליחה לשרת).

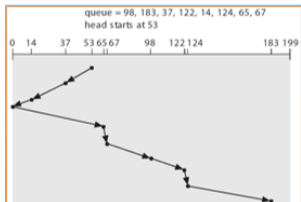
מערכת ההפעלה מקבלת עשרות בקשות לקריאה וכתיבה בבת אחת מתוכנות שונות. היא לא בהכרח תבצע אותן לפי הסדר שבו הגיעו כי זה יהיה לא יעיל – הזרוע תקפוץ הלך וחזר ותבזבז זמן יקר על seek Time. לכן ישנם אלגוריתמי תזמון שמטרתם לצמצם את ה seek time, ולמקסם את ה bandwidth (לשרת כמה שיותר בקשות).



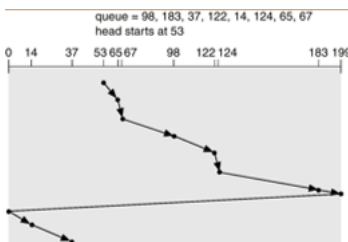
1. FCFS: האלגוריתם הפשוט ביותר. הבקשות משרתות בדיוק לפי הסדר שבו הן הגיעו לתור. נניח שיש לנו תור של בקשות לצילינדרים הבאים – 67, 65, 124, 14, 122, 37, 183, 98 – כאשר ראש הקריאה נמצא כעת במקום 53. תהליך העבודה: הראש נע מ-53 ל-98, משם ל-183, וכן הלאה לפי הסדר בתור. יתרונות: פשוט מאוד, וישנה הגינות של הבקשות. כל אחת מקבלת מענה בזמנה. חיסרון: חוסר יעילות בתנועת הראש, נע בין בקשות רחוקות מאוד ויכול לגרום לזמן חיפוש כולל גבוה מאוד.



2. SSTF: האלגוריתם בוחר בכל שלב את הבקשה הקרובה ביותר למיקום הנוכחי של ראש הדיסק. למשל: אם הראש ב-53, הוא יקפוץ קודם ל-65 (מרחק 12), אח"כ ל-67 (מרחק 2), ואז ל-37 (מרחק 30). הוא מוריד דרמטית את זמן החיפוש הכללי ומגדיל את רוחב הפס. החיסרון – הרעבה. בקשות שנמצאות רחוק מהמיקום הנוכחי עלולות לעולם לא לקבל שירות אם כל הזמן מגיעות בקשות קרובות יותר לראש.



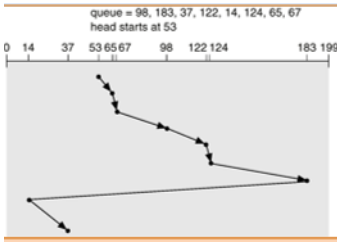
3. Scan: על מנת לפתור את בעיית ההרעבה של SSTF, הSCAN אלגוריתם שעובד כמו מעלית: הוא נע בכיוון אחד (מהצילינדר הפנימי לחיצוני למשל), משרת את כל הבקשות בדרך, מגיע לקצה ואז מסתובב וחוזר חזרה. הראש ינוע מ-53 מטה, יאסוף בדרך את 37 ו-14, ולאחר מכן ילך לאסוף את 65, 67, 98, 122, 123, 183. היתרון שלו הוא שהוא מונע הרעבה – כי בסוף ה"מעלית" מגיעה לכל קומה. החיסרון: הבקשות בקצוות מקבלות שירות פחות תכופות מאשר בקשות במרכז. הערה: הוא בהכרח מגיע לקצה ויורד לאפס (הוא תמיד מגיע לקצה שהלך בו גם אם אין בקשות! בכיוון השני הוא לא חייב להגיע לקצה. רק בכיוון שהוא הולך בו בהתחלה.



4. C-SCAN: הרגיל, המעלית עוברת הלך ושוב. הבעיה היא שהבקשות במרכז מקבלות שירות פעמיים (פעם בדרך למטה ופעם בדרך למעלה). בעוד הבקשות בקצוות מקבלות שירות לעיתים רחוקות יותר. לכן – CSACN מתנהג כמו סיבוב במעגל, הוא נע בכיוון אחד בלבד: כשהוא מגיע לקצה, הוא קופץ מיד חזרה לאפס מבלי לשרת בקשות בדרך ומתחיל שוב את הדרך למעלה. היתרון: זמן ההמתנה הופך ליותר אחיד עבור כל הצילינדרים בדיסק.

סיכום מערכות הפעלה | © גיא יער-און

5. C-LOOK: מדובר באלגוריתם חכם – הוא נע בכיוון מסוים, אך עוצר בעת שהוא מגיע לבקשה האחרונה בתור באותו כיוון, ואז מסתובב מיד. הוא לא מבזבז זמן על הגעה לקצוות של הדיסק אם אין שם מה לעשות. כלומר אם נניח שהוא הולך למעלה לכיוון גבוה – הוא עולה ומטפל בבקשות עד שמגיע ל183, היא הכי גבוהה ואז הוא קופץ לבקשה הכי נמוכה: וחוזר לשרת בסדר עולה.



מתי משתמשים באיזה אלגוריתם?

- SSFT נפוץ ובעל משיכה טבעית.
- SCAN c-scan מתפקדים טוב יותר עבור מערכות המטילות עומס כבד על הדיסק.
- הביצועם תלויים במספר ובסוגים של הבקשות.
- בקשות לשירות דיסק יכולות להיות מושפעות משיטת הקצאת הקבצים.
- אלגוריתם תזמון הדיסק צריך להיכתב כמודול נפרד של מערכת ההפעלה, מה שמאפשר להחליפו באלגוריתם אחר במידת הצורך.

- לא בכל קריאה ל system call מתבצע context switch. Context switch קורה רק אם הקרנל החליט להעביר את המעבד לתהליך אחר. אם ביצוע הקריאה מתבצע מיד – הקרנל פשוט מבצע את הקריאה ומחזיר אותך לאותו תהליך. מה שכן גורר: כל קריאת מערכת גוררת mode switch (החלפת הקשר בין הקרנל והיזר מוד).
- תנסה לבדוק מקרי קצה. אמר N? בדוק N-1 וכו'.
- תוספת זיכרון ל RAM יכולה להגדיל, להקטין, או להשאיר ללא שינוי את מס' page faults (להקטין ברוך, להגדיל אנומליה של בלדי, להשאיר ללא שינוי: הוורקינג סט שלו קטן מגודל ה RAM המקורי לפני ההגדלה).
- האלגוריתם של בנקר לא ישים בסביבה אמיתית בה פועלת מערכות הפעלה מהסיבות הבאות: קשה לחזות את max demand, מהווה overhead משמעותי על המערכת, מס' התהליכים יכול להשתנות לאורך זמן, וכך גם כמות המשאבים מכל משאב יכול להשתנות לאורך זמן (משאב יכול להתקלקל).
- חובה לזכור:

שם היחידה	קיצור	ערך בחזקות של 2
Byte	B	2^0
KiloByte	KB	2^{10}
MegaByte	MB	2^{20}
GigaByte	GB	2^{30}
TeraByte	TB	2^{40}

בכל מעבר בין יחידות (מ KB ל MB, מ MB ל GB – מכפילים ב2 בחזקת 10).

- דוגמה שימושית: אם יש לנו Page table עם 2^{20} כניסות, וכל כניסה היא 4 בייטים בגודלה. אזי גודל הטבלה הוא:
- $2^{20} * 2^4 = 2^{24} \text{ bytes} = 4 \text{ MB}$
- אם אנחנו יודעים את גודל הדף, אנחנו יודעים המון: אם גודל הדף 2^k בייטים, אזי offset בכתובת לדף יהיה בדיוק k ביטים. אם המערכת היא X-ביט, למשל 64-ביט ו גודל דף הוא למשל 4 KB (כלומר 2^{12} בייטים), אזי offset=12, וה-page number=64=52, הוא החלק שאומר לנו איזה דף לחפש בטבלה.
- גודל הטבלה: אם גודל הטבלה למשל הוא 2^{52} בייטים, וכל כניסה היא בגודל $2^4=4$ בייטים אזי גודל הטבלה הוא $2^{52} * 2^4 = 2^{56}$ בייטים.

- לזכור בשאלה 5: הם אוהבים מאוד לשחק על init וחלוקה למקרים. אם האבא מת מוקדם – תהליך הבן מאומץ ע"י init ולכן `getppid()=1`. אם האבא לא מת, הכל כרגיל. אבל יש לחלק למקרים.
- חסרונות של אלגוריתם המתבסס על הפונקציה `TestAndSet()`: יש Busy waiting (בדיקה בלולאה שלא עוצרת, המון IO), ויש Bounded waiting (הוא לא מבטיח שיש הוגנות – תהליך יכול לזכות ב LOCK פעם אחר פעם בעוד שתהליך אחר שמחכה יכול להדחק הצידה).
- הפונקציה `pause` משתחררת רק אם handler הוגדר. אם הברירת מחדל היא IGNORE – התהליך יתקע ב `pause()` לנצח.
- Pipe הוא ערוץ תקשורת חד כיווני, ואילו FIFO הוא ערוץ תקשורת דו כיווני. כלומר ניתן לבצע הן קריאה והן כתיבה ל FIFO דרך אותו descriptor.

סיכום מערכות הפעלה | © גיא יער-און

- Pipe לא מופיע בהיררכיית מערכת הקבצים, FIFO כן.
 - לאחר סיום השימוש בpipe מצד כל התהליכים, מפונים משאביו באופן אוטומטי ואילו בFIFO זה לא כך – אינו משוחרר אוטומטית לאחר שהמשתמש האחרון בו סוגר, יש לסגור אותו בקוד.
 - הוספת זיכרון פיזי לתהליך תוך כדי זמן ריצה – לא משנה את working set (נקבע לפי תבנית הבקשות של התהליכים לפייג'ים השונים), אך כן יקטין את זמן הריצה כי יהיה פחות page faults ויותר זיכרון לכל תהליך.
 - כמעט תמיד יש פרגמנטציה פנימית - בכל פעם ששיטה מחייבת הקצאה בגודל קבוע מראש (כמו דפים בזיכרון או בלוקים בדיסק, נוצרת כמעט תמיד פרגמנטציה פנימית: כי הצרכים האמיתיים של התהליך או הקובץ כמעט אף פעם לא מתאימים בדיוק ליחידת ההקצאה הזו. מתי אין פרגמנטציה פנימית? בשיטת הסגמנטים (לניהול זיכרון) או בהקצאה רציפה (ניהול זיכרון) – כי שם זו הקצאה דינמית של שטחי אחסון רציפים, ולכן אין סיבה לפרגמנטציה פנימית.
 - הוא שחזר את השאלה הזו כמה פעמים:
 - ג. הנח כי ידוע לך כי בדיוק ביט אחד ב-free block bit vector של מערכת ניהול הקבצים כלשהי שגוי אולם אינך יודע באיזה ביט מדובר. תאר אלגוריתם למציאת הביט השגוי (אין צורך לכתוב קוד בתשובה).
- תשובה:**
- הדבר יושג באמצעות בניית הוקטור מחדש והשוואתו לוקטור המקורי. ספציפית:
- צור וקטור חדש ואתחל אותו ל-0 (פנוי).
 - החל ב-root ועבור על כל הקבצים בצורה רקורסיבית.
 - סמן בוקטור החדש ב-1 כל בלוק מוקצה.
 - השווה את הוקטור הקיים עם הוקטור החדש שיצרת. הביט השונה הוא הביט השגוי.
- כאשר זמן ההמתנה עד שהcritical code יתפנה הוא קצר מאוד – עדיף לתהליך לבזבז את זמן המעבד שלו בבדיקת תנאי Busy waiting מאשר לשלם את המחיר של context switch (הכפול: פעם אחת לצאת ממצב חסון, פעם אחת לחזור כשהמשאב מתפנה). כמו כן, כאשר אין עוד תהליכים במצב Ready אזי המעבד היה במצב IDLE של בטלה. במקום שהמעבד סתם יהיה ריק – נעדיף שהוא יעבוד בבדיקת תנאי (ככה לפחות הוא יעבוד).
 - אין partition directory כאשר הוא RAW (ללא מערכת קבצים).
 - שאלה מעצבנת: הפתרון הוא שלא בהכרח. למה? בעת שמחברים דיסק מתבצע Mounting – הדיסק פיזית נכנס לתוך הזיכרון במחשב של משה. יתכן מצב שבו שני הקבצים (או אפילו שלושתם) נכנסים לאותו partition:
- משמעות הדבר – הם יחלקו אותו directory. ולמה זה לא טוב? מה ההגדרה של גיבוי? הרי ברמה הפרקטית – המידע אכן עובר מהדיסקים של דוד למחשב של משה. אך ההגדרה של גיבוי היא לא רק העתקה של חומר, אלא העתקה של חומר שאפשר לשחזר. גיבוי של קבצים – יש לנו דרך לנהל ולשחזר את המידע. מה באמת הבעיה? נניח ושני דיסקים נכנסו לאותה מחיצה. הטבלה directory יושבת פיזית במקום אחד על אחד הדיסקים. כשאנחנו מחברים את הדיסק שיש בו את directory: המערכת של משה רואה את "מפת הדרכים" אך מגלה שהרבה נתונים שכתובים במפה נמצאים על הדיסק השני (שכרגע לא מחובר). לכן המערכת תגיד: אני לא יכולה לגשת לקבצים אלו כי הם חסרים. כאשר מחברים את הדיסק השני בו אין directory: המערכת של משה מסתכלת עליו ולא מבינה מה זה – מבחינתה זה סתם רצף של ביטים כי אין לה את directory (זה RAW). לכן הבעיה היא שאי אפשר לגבות את הקבצים.- ניצולת הדיסק: אחוז הזמן שהדיסק משרת בקשות. ככל שמהירות הסיבוב תהיה גבוהה, כך הניצולת תהיה נמוכה כי נסיים את הבקשות מהר יותר ויהיה IDLE, ככל שמס' התהליכים במערכת גבוה יותר: הניצולת תגדל כי יותר תהליכים ירצו לגשת לדיסק.

סיכום מערכות הפעלה | © גיא יער-און

- בPCB יש Accounting information: עבור המידע של כמה זמן CPU צרך כבר התהליך, מתי התחיל ומתי סיים, כמה IO ביצע, כמה זיכרון תפס לאורך זמן. המערכת צריכה את המידע הזה על מנת לתזמן את התהליך ולבצע BILLING.
 - כלל כשמשמשים בpipe:
 - הקורא צריך בהתחלה לסגור את FD1 (ערוץ הכתיבה), ובסיום לסגור את FD0 (ערוץ הקריאה).
 - הכותב צריך בהתחלה לסגור את FD0 (ערוץ הקריאה) ובסיום לסגור את FD1 (ערוץ הכתיבה).
- מה הסכנה? האבא (הכותב) יעשה PIPE – יש עותק אחד של כתיבה במערכת. ואז הוא עושה FORK – הבן מקבל עותק זהה. עכשיו יש שני עותקים של FD1 (כתיבה) שפתוחים. האבא כותב – מסיים – עושה close(FD1): אבל אצל הבן שירש נשאר עותק אחד של FD1 במערכת! לכן כאשר הבן מנסה לקרוא אחרי שהאבא סיים, המערכת בודקת: האם יש עוד נתיב קריאה פתוח? המערכת רואה שכן – כי הבן קיבל אותו (ושכח לסגור בהתחלה!), ולכן המערכת אומרת לקורא: שומע, יש עדיין כותב פוטנציאלי במערכת ((כי FD1 פתוח מהבן!)), היא אומרת לו אולי הוא כרגע בהפסקה – אבל הדלת עוד פתוחה, לכן אתה תשב ותחכה לו. מה קורה? הבן הוא זה שאצלו פתוח ה fd1 – אבל הוא יושב ומחכה ולא יודע זאת. בום: הבן יתקע לנצח בקריאה.
- הסיבה העיקרית לשימוש בDMA: שיפור ביצועי המערכת ע"י הגדלת המקביליות. **לשים לב:** זה לא לאפשר למעבד לרוץ בקצב מהיר יותר וזה לא לאפשר למעבד להגיב מהר יותר להתקני קלט ופלט.
 - בתוך PCB של תהליך נמצא תמיד: מזהה התהליך, מצב הריצה של התהליך, PC, רגיסטרים, מידע לתזמון (עדיפות, מצביעים לתורי Ready Queue and IO Queue), מידע על הזיכרון (Base/Limit register או Page table), מידע סטטיסטי (זמן CPU שנצרך, זמן התחלה, מזהה המשתמש וכו'). מידע על הקלט והפלט (רשימת הקבצים הפתוחים, התקנים בהם התהליך משתמש, בקשות IO פעילות).
 - האנומליה של בלדי לא יכולה לקרות לא בLRU ולא בOPTIMAL!
 - **שאלה טובה אז נתעכב:**
- נתונה file system עם בלוקים בגודל 2048 bytes ומצביעים לבלוקים בגודל 32-bit. בכל קובץ יש 12 מצביעים ישירים לבלוקים, מצביע מסוג indirect יחיד, מצביע מסוג double-indirect יחיד ומצביע מסוג triply-indirect יחיד.
- א. מהו גודל המקסימלי של דיסקים בהם יכולה לתמוך המערכת?
- ב. מהו הגודל המקסימלי של קובץ במערכת?
- פתרון: נפתור עם הסבר. ישר: נסמן $B = 2^{11}$ כגודל הבלוק. כמו כן, נשים לב כי מצביע לבלוק הוא בגודל $4 \text{ bytes} = 2^2$. $X = \frac{32}{8}$. אלו נתונים קריטיים. מה הכוונה בגודל מקסימלי של דיסקים בהם יכולה לתמוך המערכת? ישנן 2^{32} כתובות אפשריות (32 ביט). מכל כתובת יכול להיות בלוק בגודל B. לכן הפתרון הוא:
- $$\text{Solution} = B * 2^{32} = 2^{43}$$
- ולסעיף הבא. מה הגודל המקסימלי של קובץ במערכת? נשים לב כי הפתרון הוא: מס' הבלוקים האפשריים, כפול גודל של בלוק. ובכן כמה מצביעים נכנסים בבלוק? ובכן אם גודל בלוק הוא B וגודל מצביע הוא 4 אזי מס' המצביעים שנכנסים בבלוק:
- $$\frac{B}{2^2} = 2^9 = N$$
- ובכן. כעת, נספור את מס' הבלוקים:
- 12 מצביעים לבלוקים
 - N מצביעים בבלוק יחיד
 - N^2 מצביעים בשכבה הבאה.

לכן הפתרון הוא:

$$T = B(12 + N + N^2 + N^3) = 2^{11}(12 + 2^9 + 2^{18} + 2^{27})$$

- יתרונות של כל אחת מהשיטות להקצאת זיכרון:
 1. הקצאה רציפה – ביצועים מדהימים בקריאה רציפה, אין צורך להזיז ראש דיסק. טוב להשתמש כאשר יודעים מראש את גודל הקובץ ורוצים מהירות מקסימלית. החיסרון: פרגמנטציה חיצונית
 2. רשימה מקושרת: הכי טוב להשתמש כאשר לא יודעים כמה מידע יכנס לקובץ, והגישה סדרתית. החיסרון הוא גישה אקראית גרועה – כדי להגיע לבלוק 100 יש לקרוא את כל ה-99 שלפניו.
 3. *Index*: הכי טוב להשתמש כאשר אנו דורשים גישה מהירה לחלקים שונים בקובץ, החיסרון הוא בזבוז מקום עבור קבצים קטנים.
- יתרונות של תרדים:
 1. יעילות גבוהה יותר. ניתן להריץ קוד במקביל. בעת חסימה תרדים אחרים ימשיכו לרוץ.
 2. קונטקסט סוויץ' קטן יותר.
 3. קל לחלק משימה גדולה למשימות קטנות.
- חסרונות של תרדים:
 1. הופך סינכרוניזציה למאוד קשה.
 2. מקשה על דיבאגינג (קוד לא דטרמיניסטי).
 3. התרדים זולים אבל יש להם עלות כלשהי, שימוש בהרבה תרדים יגרום למערכת לבצע הרבה החלפות קשר ובכך תקטן נצילות המעבד.
- תהליך שמבצע *thrashing* מוגדר כתהליך *IO-BOUND*.
- *Swap in*: להביא דף מהדיסק לזיכרון, *swap out*: להוציא דף מהזיכרון לדיסק.
- בקבצים תמיד יש פרגמנטציה פנימית.
- תמיד הדפים הראשונים הם *page faults*!!!
- עקרון ההפרדה בין *mechanism policy*: מה שאנחנו רוצים לעשות מופרד מאיך שאנחנו עושים זאת. מפרדים מימוש ומה שרוצים. למשל: אלגוריתם תזמון. *policy* נגדיר מי יקבל זמן *CPU* ובמכניזם נבצע את האלגוריתם. דוגמה נוספת: הרשאות – נחליט מי מקבל הרשאות, ואח"כ נממש כיצד אנחנו בודקים הרשאות ושומרים על כך שרק מי שקיבל הרשאות יוכל לבצע.
- אם אין *multiprogramming* אז אין צורך ב *Long term scheduler* וגם *short*. כמו כן לא יהיו דדלוקים, ניהול הזיכרון ידרוש שינוי מהותי (תהליך אחד שרץ יקבל את כל הזיכרון), בדומה אין צורך לתמוך בסינכרוניזציה. מנגנוני *IPC* יבוטלו (אין יותר תקשורת בין "תהליכים", רץ תהליך אחד. יש שינוי מהותי בכל הקשור באבטחה ובהגנה (הרבה פחות להגן).
- האם ניתן לזהות מצב של *thrashing* באמצעות ניתור רציף של כמות *page faults* שקרו במהלך *X* הפעולות האחרונות? לא. צריך גם לדעת את אחוז הניצולת של המעבד (רק אם הוא נמוך זה באמת *thrashing*).
- האם תהליך יכול לעבור ממצב של *thrashing* שלא כתוצאה בשינוי בכמות הפריימים שמוקצים עבורו ב *RAM*? התשובה היא כן. אם הוא שניה את את *LOCALITY* שלו – אם היא גדולה ממס' הפריימים שמוקצים לו בזיכרון הוא עלול למצוא את עצמו ב *Thrashing*.
- אם מגדילים את גודל *pages* פי 2 – יתכן כי יקטן מס' ה *page faults*. מדוע? כשגודל הפייג' גדל, מביאים יותר זיכרון בכל פעם. לכן יש סיכוי שעמודים שנצטרך בקרוב יהיו כבר בזיכרון. אם מקטינים את גודל *pages* פי 2 – לרוב יגדל. אבל לא תמיד!
- עקרונות של קורס מערכות הפעלה שנרצה להשתמש בהם: הרעבה (זה לא טוב), *overhead* (תקורה) – מריצים קטע קוד יקר בשביל משהו שאמור להיות פשוט, לא טוב.

סיכום מערכות הפעלה | © גיא יער-און

- מדוע גודל של דף הוא חזקה של 2? בשביל שהמחשב יוכל להשתמש בסידור הביטים הטבעי של הכתובת בלי שום צורך בחישובים אריתמטיים בזמן הגישה לזיכרון. אם גודל הדף הוא חזקה של 2, אזי הoffset תמיד יתפוס בדיוק את d הבתים האחרונים של הכתובת (הימניים). אם למשל גודל הדף היה 3000: כדי למצוא את כתובת 7000 המחשב צריך לבצע פעולת חילוק: $7000/3000=2$ עם שארית 1000 (היסט 1000) - זה לא יעיל חילוק!
- תהליך שנמצא במצב *busy waiting* הוא במצב *running* (שכן הוא לוקח זמן CPU) ולא במצב *waiting*!
- הבעיה ב*General Graph Directory*: הוא מכיל מעגלים - מאפשרים קישורים שיוצרים מעגלים או מצב בו קובץ וספרייה מופיעים ביותר ממסלול אחד. קשה מאוד לנהל - אלגוריתמי חיפוש יכולים להכנס ללולאה אינסופית בגלל מעגלים, קשה לדעת מתי למחוק קובץ בפועל: אם יש לו שני קישורים - מחיקת אחד מהם לא צריכה למחוק את הקובץ. כיצד מתמודדים עם זאת? מאשרים קישורים (*LINKS*) רק לקבצים ולא לספריות - *HARD LINK*, משתמשים באלגוריתם שעובר באופן תקופתי על המערכת על מנת לזהות קבצים שאיש לא מצביע עליהם יותר ולמחוק אותם, ומריצים אלגוריתם בבניית כל *LINK* חדש לבדוק שהוא לא יוצר מעגל.
- נשים לב: אלגוריתם *second chance* מחזיק מצביע שמבצע בדיקה מעגלית של הדפים השונים בזיכרון. נשאלת השאלה: האם המצביע צריך לרוץ מעגלית רק על דפים של התהליך עצמו, או שעליו לרוץ מעגלית על כל הדפים, גם של התהליכים שלא גרמו *page fault*. התשובה: חלוקה למקרים! אם ההחלפה גלובלית (מכל הפריימים) אזי שנרוץ על כל הדפים, אם ההחלפה לוקלית (רק מהדפים של התהליך עצמו) - נרוץ עם המצביע רק על הדפים של התהליך עצמו.
- כשאנו עובדים עם *Frames, Pages* - אין בכלל פרגמנטציה חיצונית! לשים לב לזה!
- אם הריצו את הפקודה *umask* עם הרשאות מסוימות להורדה *XYZ* ואז יצרו קובץ חדש עם הרשאות *WTM* אזי יש דרך לחישוב ההרשאות החדשות: לכל סוג הרשאה מבצעים כך: נניח הימנית, כלומר ה"אחרים". נקח את הבסיס *M* - ונעשה לו *AND* בינארי עם *NOTZ* (כלומר *NOT* על *Z* בינארי). נקבל מס' - זה ההרשאות של "אחרים".
- כמובן, *shared memory* הוא רק עבור תהליכים שרצים במכונות זהות. שכן - הם צריכים לחלוק משהו: שטח משותף. תהליכים שרצים במכונות שונות לא יכולים לחלוק.....
- מערכות ההפעלה *Linux, Unix, Mac OS* משתמשות באותו *API* (של קריאות מערכת), *windows* לא.
- קריאה *pthread_exit* מהתרד הראשי - לא הורסת את התהליך כולו! מערכת ההפעלה לא סוגרת את התהליך - היא משאירה את מרחב הכתובות והזיכרון המשותף כל עוד יש לפחות תרד בן אחד חי. רק כאשר התרד הבן האחרון בתהליך יסיים את ריצתו - התהליך כולו ייסגר סופית ע"י הקרנל.
- בשאלות עם *EAT*, אם אנחנו עובדים עם אלפא כאחוז הצלחות והגדילו לנו אותו וכו'. אזי - האלפא החדש (למשל 2 אלפא וכו') צריך להיות קטן מ1.
- הדגל *IPC_RMID* למחיקה: בזיכרון משותף - המקטע נמחק רק אחרי שכל התהליכים מתנתקים ממנו. בסמפורים: קבוצת הסמפורים נמחקת מיד.

סיכום מערכות הפעלה | © גיא יער-און

נגדיר עקרונות כלליים למענה על שאלה מהצורה שני תרדים שרצים ומשנים ערך משתנה משותף.

יהיו שני תרדים שרצים, נסמנם $A:B$ רץ K איטרציות וכן B רץ N איטרציות. נניח כי תרד A מבצע פעולה על משתנה משותף מהצורה $x=x+M$ וכן B מבצע פעולה על משתנה משותף מהצורה $x=x+L$. אזי:

1. $\text{Max}(X)=KM+NL$: מקרה זה מתקבל כאשר אין דריסה של הנתונים.
2. הערך המינימלי – תחת הנחה כי N,K הם לפחות 2 איטרציות. הערך המינימלי האפשרי הוא

$$\text{Min} = \min(M + L, K * M, N * L)$$

מדוע? ישנם 3 מקרים –

$M+L$: המקרה שבו כל תרד תורם איטרציה אחת לתוצאה הסופית בגלל פינג פונג של דריסות.

$K*M$: המקרה בו תרד 1 מבצע כל איטרציות שלו כמעט ברצף, נרדם רגע לפני הכתיבה האחרונה. נותן לתרד 2 לבצע את כל העבודה שלו ואז מתעורר ודורס הכל עם התוצאה שלו.

$N*L$: כמו הקודם, רק הפוך (תרד 2 דורס בסוף את עבודת תרד 1).

3. אם אחת הפעולות הופכת לאטומית. אם הפעולה של B הפכה לאטומית – אזי אי אפשר לדרוס אותה בזמן ביצוע. זה אומר דווקא ההפך – A יכול לרוץ, להקטע: לתת לכל B לרוץ (או לאיטרציה שלמה בו) ואז לדרוס את מה ש B העלה! לכן במקרה זה הערכים האפשריים של x בסיום הריצה הם: $KM, KM+M, KM+2M, KM+3M, \dots, KM+NL$.

מבנה של X-level paging הוא מהצורה הבאה:

- רמה ראשונה, זה חלק שנותן לנו את הטבלה העליונה. הכתובת הלוגית מתחילה בה. היא מצביעה על טבלה ברמה השנייה.
- רמות 2 עד X-1 (עד לפני אחרונה): שכבות ביניים. כל טבלה מכילה מצביעים לטבלאות הבאות בתור.
- הרמה האחרונה: כאן נמצא page Table האמיתי שמכיל את הכתובת הפיזית בזיכרון (Frame number).

כש MMU רוצה לתרגם כתובת לוגית: הוא ניגש לטבלה הראשונה בזיכרון, מוצא מצביע לטבלה השנייה, ניגש לטבלה השלישית וכו'. כשמגיע לטבלה האחרונה הוא מוצא את Page Number.

כאשר אין לנו TLB (או כישלון בחיפוש שם – אגב נזכיר כי TLB יושב ב-CACHE ולכן זו לא עלות גישה ל-RAM שנספרת בשאלות): מס' הגישות ל-RAM שמבצעים הוא X (כמס' הרמות). בסוף – יש גם גישה לנתונים עצמם (יש לשים לב אם בשאלה אנחנו סופרים אותם או לא. אם כן סופרים: זה X+1). כאשר יש לנו TLB – יש 0 גישות עבור תרגום (שנמצא כבר ב-CACHE) – ואז שוב: אם סופרים את הגישה לנתונים עצמם זו גישה אחת בלבד.

2-Level Paging: הכתובת הלוגית מחולקת ל-3 חלקים: הרמה הראשונה, הרמה השנייה (טבלת הדפים), והoffset. עוברים דרך 2 גישות לטבלאות על מנת למצוא את frame.

עכשיו אחרי שעשינו סדר במונחים נגיע לדון בשאלה הכי חשובה: כיצד פותרים שאלה כזו.

- כשאומרים לך מערכת של 32 ביט – משמעות הדבר שאפשר לייצג 2^{32} כתובות שונות (כמובן על כל X ביט שייגיד). כיוון שכל כתובת מצביעה ל-BYTE אחד בזיכרון: **הגודל המקסימלי של הזיכרון הלוגי שאנו יכולים לייצג** (וזהו גם הזיכרון המקסימלי עבור תהליך) הוא 2^{32} Bytes (שוב, להכליל לכל X ביט).
- כל כתובת לוגית תמיד מחולקת ל-2 חלקים (או יותר אם עובדים עם כמה רמות). Offset(d): המיקום המדויק בתוך הדף (כמות הביטים הנדרשת לoffset נגזרת ישירות מגודל הדף – אם גודל הדף הוא 2^k Bytes אזי חייבים k ביטים כדי לייצג בית בתוכו. וכן: Page Number(p): שאר הביטים בכתובת – אלו הביטים שאומרים לנו באיזה דף אנו נמצאים.
- יש 2^p דפים שונים (אם p זה מס' הביטים לייצוג ביט). דרך נוספת הגיונית לחישוב מס' הדפים:

$$\frac{\text{Logical address space}}{\text{Page Size}}$$

- בתוך טבלת הדפים – ישנן המון רשומות. לכל דף לוגי במערכת חייבת להיות רשומה אחת בדיוק. הרשומה הזו שומרת את מס' הפריים הפיזי וביטי בקרה (Valid invalid bit). אם גודל של רשומה הוא PTE SIZE, אזי גודל הטבלה הוא באופן מאוד הגיוני:

$$\text{Page Table Size} = (\text{Number of Pages}) * (\text{PTE Size})$$

- כשיש כמה רמות: לרוב דודי אוהב להגיד במבחן שכל טבלה פנימית תהיה בדיוק בגודל של דף פיזי אחד. איך עובדים החישובים מרגע זה?

אם טבלה פנימית תופסת בדיוק דף אחד, אזי כמות השורות שיכנסו בה הוא בדיוק: $\frac{\text{Page Table (Size of Table)}}{\text{PTE Size}}$

ברגע שאנו יודעים כמה שורות יש בטבלה, אנו יודעים אוטומטית כמה ביטים מוקצים לשדה של אותה שכבה בכתובת הלוגית. על מנת לבחור שורה אחת מתוך טבלה עם 2^k שורות: חייבים בדיוק k ביטים.

במערכת עם X שכבות – אם כל טבלה היא בגודל דף: משמעות הדבר שכל שכבות הביניים והשכבות התחתונות מקבלות בדיוק אותו מס' ביטים בכתובת. השכבה היחידה שיכולה להיות קטנה יותר היא העליונה ביותר שלעיתים לא מנצלת דף שלם אלא פחות (או יותר).

סיכום מערכות הפעלה | © גיא יער-און

כלומר. אם בשכבה התחתונה 10 ביטים - 2^{10} שורות בה וגודל דף הוא KB4 אזי טבלה פנימית אחת כזו מכסה טווח רציף של $2^{10} * 2^{12} = 2^{22} = 4MB$.

כמובן: אם המערכת היא X ביט אזי מס' הבייטים לייצוג כתובת הוא $\frac{X}{8}$.

1. אם אנחנו בהקצאה רציפה: אין מצביעים. אזי, מס' הבלוקים הדרושים הוא:

$$\frac{Disk\ Size}{Block\ Size}$$

2. Linked list: כאן כל בלוק מכיל דטה ומצביע לבלוק הבא. כמות הדטה בכל בלוק היא $Block_Size - Pointer_Size$ לכן מס' הבלוקים שנצטרך:

$$\frac{Disk\ Size}{Block_Size - Pointer_Size}$$

3. אינדקס:

בלוק הדטה מכיל דטה בלבד, ומס' בלוקי הדטה הוא פשוט: $\frac{Disk\ Size}{Block\ Size}$.

בלוק אינדקסים: מכיל מצביעים לבלוקים הבאים. מס' המצביעים שניתן לאחסן בו הוא $\frac{Block\ Size}{Pointer\ Size}$.